

AWS Cloud Handbook — Complete Edition

1 AWS Cloud Handbook — Complete Edition

Complete edition

Beginner → Advanced AWS. Every chapter is included.

Page numbers in the following index match the PDF footer.

2 AWS Cloud Handbook — Complete Edition

This edition: 214 numbered PDF pages (matches the page numbers in this file).

Beginner → Advanced AWS. Every chapter is included.

Companion: [DevOps Handbook](#)

Page numbers in this index match the page number printed at the bottom of each PDF page. They are measured from this edition, not estimates.

Total pages: 214

2.1 Part I — Cloud Foundations (Pages 5-19)

Ch	Title	Pages
1	Introduction to AWS & the Shared Responsibility Model	5-7
2	AWS Global Infrastructure — Regions, AZs, and Edge	8-10
3	AWS Account Setup, Billing, and Cost Basics	11-13
4	AWS CLI, CloudShell, and SDK Overview	14-19

2.2 Part II — Identity & Access Management (Pages 20-36)

Ch	Title	Pages
5	IAM Users, Groups, Roles, and Policies	20-23
6	IAM Best Practices and Permission Boundaries	24-26
7	IAM Identity Center (SSO) & Federation	27-31
8	AWS Organizations & Service Control Policies	32-36

2.3 Part III — Networking (Pages 37-66)

Ch	Title	Pages
9	VPC Fundamentals — CIDR, Subnets, and Route Tables	37-39
10	Internet Gateway, NAT Gateway, and Routing	40-42
11	Security Groups vs Network ACLs	43-48
12	Load Balancers — ALB, NLB, and GWLB	49-54
13	Route 53 — DNS Records and Routing Policies	55-60
14	VPC Peering, Transit Gateway, and PrivateLink	61-66

2.4 Part IV — Compute (Pages 67-92)

Ch	Title	Pages
15	EC2 — Instance Types, AMIs, and Launch Templates	67-69
16	Auto Scaling & Elastic Load Balancing Integration	70-75

17	Lambda — Events, Concurrency, and Layers	76-81
18	ECS & Fargate	82-86
19	EKS — Managed Kubernetes on AWS	87-92

2.5 Part V — Storage (Pages 93-113)

Ch	Title	Pages
20	S3 — Buckets, Objects, Versioning, and Lifecycle	93-95
21	S3 Security, Encryption, and Static Websites	96-101
22	EBS & EFS	102-107
23	AWS Backup & Disaster Recovery Patterns	108-113

2.6 Part VI — Databases (Pages 114-136)

Ch	Title	Pages
24	RDS & Aurora	114-119
25	DynamoDB — Tables, Indexes, and Streams	120-125
26	ElastiCache & Caching Patterns	126-130
27	Database Migration (DMS) & DR	131-136

2.7 Part VII — Application Integration (Pages 137-154)

Ch	Title	Pages
28	API Gateway	137-142
29	SQS, SNS, and EventBridge	143-148
30	Step Functions Workflows	149-154

2.8 Part VIII — DevOps on AWS (Pages 155-177)

Ch	Title	Pages
31	CloudFormation Deep Dive	155-160
32	AWS CDK	161-165
33	CodePipeline, CodeBuild, and CodeDeploy	166-171
34	Systems Manager & Parameter Store	172-177

2.9 Part IX — Observability & Operations (Pages 178-195)

Ch	Title	Pages
35	CloudWatch — Metrics, Alarms, and Dashboards	178-180
36	CloudWatch Logs & Insights	181-185
37	X-Ray & Distributed Tracing	186-190

38	CloudTrail & AWS Config	191-195
----	-------------------------	---------

2.10 Part X — Security & Compliance (Pages 196-214)

Ch	Title	Pages
39	KMS, Secrets Manager, and ACM	196-201
40	GuardDuty, Security Hub, WAF, and Shield	202-208
41	Well-Architected Review & Capstone Design	209-214

2.11 Edition total

This PDF contains **41 chapters** across **214 pages**.

AWS Cloud Handbook — Complete Edition —

Chapter 1

1 Chapter 1: Introduction to AWS & the Shared Responsibility Model

1.1 1.1 What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is a comprehensive **cloud computing platform** offering on-demand compute, storage, databases, networking, machine learning, and 200+ other services. Instead of buying physical servers, you provision resources in minutes and pay primarily for what you use.

1.1.1 Why AWS matters for DevOps

- **API-driven** — Every service is controllable via CLI, SDK, and IaC.
- **Global scale** — Deploy in multiple regions for resilience and latency.
- **Managed services** — Offload undifferentiated heavy lifting (RDS, EKS, Lambda).
- **Ecosystem** — Largest cloud market share; extensive documentation and community.

1.2 1.2 Cloud service models

Model	You manage	Provider manages	AWS examples
IaaS	OS, apps, data	Hardware, network, hypervisor	EC2, VPC, EBS
PaaS	Apps, data	Runtime, OS, infra	Elastic Beanstalk, App Runner
SaaS	Configuration, data	Everything else	WorkSpaces, Chime
FaaS	Function code	Everything else	Lambda

Containers blur lines: ECS/EKS give you orchestration (between IaaS and PaaS).

1.3 1.3 The Shared Responsibility Model

Security and compliance are **shared** between AWS and the customer. The boundary shifts by service type.

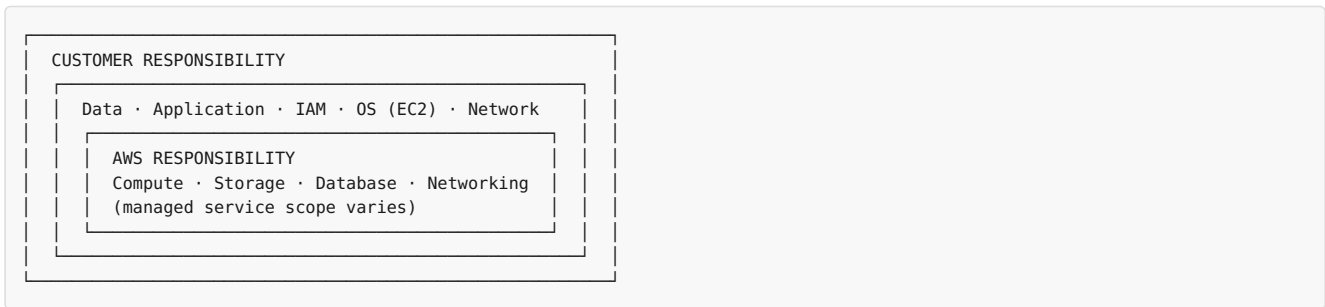
1.3.1 AWS is responsible for Security OF the Cloud

- Physical data centers
- Hardware and global network
- Virtualization layer
- Managed service patching (for fully managed services)

1.3.2 You are responsible for Security IN the Cloud

- Data classification and encryption choices
- IAM users, roles, and policies
- Operating system patches (on EC2)

- Network configuration (security groups, NACLs)
- Application code security



1.3.3 Example: RDS vs EC2 MySQL

Task	EC2 + self-managed MySQL	Amazon RDS
Patch MySQL	You	AWS
Patch OS	You	AWS
Configure security groups	You	You
Encrypt data at rest	You (configure)	You (enable)
Physical security	AWS	AWS

1.4 1.4 Core AWS concepts

Concept	Definition
Region	Geographic area (e.g., <code>us-east-1</code> , <code>eu-west-1</code>)
Availability Zone (AZ)	Isolated data center within a region
Account	Billing and security boundary (12-digit ID)
Resource	Any AWS entity (EC2 instance, S3 bucket, etc.)
ARN	Amazon Resource Name — unique identifier
Tag	Key-value metadata on resources

Example ARN:

```
arn:aws:s3:::my-bucket
arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234
```

1.5 1.5 AWS console, CLI, and IaC

Three ways to interact with AWS:

1. **Management Console** — Web UI; great for learning and ad-hoc tasks.
2. **AWS CLI** — Scriptable; essential for automation.
3. **Infrastructure as Code** — CloudFormation, CDK, Terraform for reproducible environments.

DevOps engineers spend most time in **CLI + IaC**, using the console for debugging.

1.6 1.6 Pricing fundamentals

- **On-Demand** — Pay per hour/second; no commitment.
- **Reserved Instances / Savings Plans** — 1–3 year commitment; lower cost.
- **Spot Instances** — Unused capacity; cheap but can be interrupted.
- **Free Tier** — Limited free usage for 12 months (and some always-free services).

Always set billing alarms (AWS Budgets) on new accounts.

1.7 1.7 Well-Architected Framework (preview)

AWS defines six pillars for good architecture (covered throughout this handbook):

1. Operational Excellence
 2. Security
 3. Reliability
 4. Performance Efficiency
 5. Cost Optimization
 6. Sustainability
-

1.8 1.8 Chapter summary

- AWS provides on-demand cloud infrastructure and managed services.
 - **Shared responsibility** splits security between AWS and you—know your side.
 - Learn **regions, accounts, ARNs, and tags** early.
 - Use CLI and IaC for production; console for exploration.
-

1.9 Lab 1.1 — Create your AWS account

1. Sign up at <https://aws.amazon.com> (use a dedicated email).
 2. Enable **MFA** on the root account; do not use root daily.
 3. Create an **IAM admin user** for daily work.
 4. Set a **billing budget** alert at \$10 or your comfort threshold.
 5. Explore the console: EC2, S3, IAM dashboards.
-

1.10 Review questions

1. What is the difference between IaaS and PaaS? Give one AWS example of each.
 2. Who patches the OS on an EC2 instance?
 3. What is an ARN used for?
 4. Name two AWS pricing models.
-

Next: [Chapter 2 — Global Infrastructure](#)

AWS Cloud Handbook — Complete Edition —

Chapter 2

1 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

1.1 2.1 Global infrastructure overview

AWS operates a worldwide network of data centers organized into:

```
Region (e.g., us-east-1)
├─ Availability Zone A (us-east-1a)
├─ Availability Zone B (us-east-1b)
├─ Availability Zone C (us-east-1c)
└─ ... (typically 3+ AZs per region)
```

Edge Locations (200+) – CloudFront, Route 53, etc.

1.2 2.2 Regions

A **Region** is a separate geographic area. Each region is **fully isolated** from others for most services.

1.2.1 Choosing a region

Factor	Guidance
Latency	Place resources near users
Compliance	Data residency (GDPR, etc.) may require EU regions
Service availability	New services launch in select regions first
Cost	Pricing varies by region (us-east-1 often cheapest)
Disaster recovery	Use a second region for critical workloads

Naming: us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

1.2.2 Regional vs global services

Regional (must specify region)	Global (single endpoint)
EC2, RDS, VPC, Lambda	IAM, Route 53, CloudFront
S3 (bucket tied to region)	AWS Organizations

1.3 2.3 Availability Zones (AZs)

An **AZ** is one or more discrete data centers with independent power, networking, and cooling within a region. AZs are connected by **low-latency private fiber**.

- Deploy across **multiple AZs** for high availability.
- AZ names like `us-east-1a` are **account-specific mappings**— `1a` in your account may not be the same physical DC as `1a` in another account.

1.3.1 Multi-AZ patterns

Pattern	Use case
Active-passive	RDS Multi-AZ failover
Active-active	ALB spanning AZs with EC2 in each
Stretch cluster	EKS node groups per AZ

1.4 2.4 Local Zones, Wavelength, and Outposts

Type	Purpose
Local Zones	Extend region closer to metro areas (ultra-low latency)
Wavelength	5G edge at carrier locations
Outposts	AWS hardware on-premises; hybrid cloud

These are specialized; start with standard regions + AZs.

1.5 2.5 Edge locations and CloudFront

Edge locations cache content close to users via **Amazon CloudFront** (CDN). Route 53 and AWS Shield also use the edge network.

Benefits: - Lower latency for static assets - Reduced origin load - DDoS protection at edge (with Shield)

1.6 2.6 Fault isolation design principles

1. **Never rely on a single AZ** for production.
2. **Design for regional failure** for mission-critical systems (multi-region active-active or pilot light).
3. **Use managed services** that handle AZ redundancy (RDS Multi-AZ, S3 cross-AZ durability).
4. **Test failover** regularly.

1.6.1 S3 durability vs availability

- **S3 Standard** — 99.999999999% (11 nines) durability across AZs in a region.
- **Availability** — Different metric; rare regional outages can still occur.

1.7 2.7 Service quotas (limits)

Each account has **default quotas** per region (e.g., EC2 vCPUs, VPCs per region). Request increases via **Service Quotas** console. Plan capacity before large deployments.

1.8 2.8 Chapter summary

- **Regions** are geographic; **AZs** are isolated DCs within a region.
 - Deploy production workloads across **multiple AZs**.
 - **Edge locations** power CloudFront and global services.
 - Know **regional vs global** services when designing architecture.
-

1.9 Lab 2.1 — Explore regions in CLI

```
aws configure                                # Set default region
aws ec2 describe-availability-zones --region us-east-1
aws ec2 describe-regions --output table
```

Document which region you will use for labs and why.

1.10 Review questions

1. How many AZs should a highly available web app use at minimum?
 2. Is IAM a regional or global service?
 3. What AWS service uses edge locations for caching?
 4. Why might you choose `eu-central-1` over `us-east-1`?
-

Next: Part II — [Chapter 5: IAM Fundamentals](#)

AWS Cloud Handbook — Complete Edition —

Chapter 3

1 Chapter 3: AWS Account Setup, Billing, and Cost Basics

1.1 3.1 Creating a secure AWS account

1.1.1 Step-by-step

1. **Sign up** at <https://aws.amazon.com> with a dedicated email (not personal if possible for work).
2. **Root account** — Full admin access. Enable **MFA immediately**.
3. **Do not use root** for daily tasks.
4. Create an **IAM admin user** or use **IAM Identity Center** for workforce access.
5. Set up **billing alerts** before launching resources.

1.1.2 Root account checklist

- ☐ MFA enabled (hardware key preferred)
- ☐ Strong unique password in password manager
- ☐ No access keys on root
- ☐ Contact info and alternate contacts set
- ☐ AWS Budgets alert configured

1.2 3.2 AWS Free Tier

Type	Examples
12-month free	750 hrs/month t2/t3.micro EC2, 5 GB S3, RDS limits
Always free	Lambda 1M requests/month, DynamoDB 25 GB
Short trials	Some services have limited trials

Warning: Free tier expires; resources left running can incur charges. Always set billing alarms.

1.3 3.3 Billing and Cost Management

Tool	Purpose
Billing Dashboard	Current month spend
Cost Explorer	Historical analysis, forecasts, filters
AWS Budgets	Email/SNS alerts at thresholds
Cost & Usage Report (CUR)	Granular export to S3 for analysis
Pricing Calculator	Estimate architecture costs before building

1.3.1 Budget example

Create a monthly budget of \$20 with alerts at 50%, 80%, and 100%:

```
aws budgets create-budget \
  --account-id 123456789012 \
  --budget file:///budget.json \
  --notifications-with-subscribers file:///notifications.json
```

1.4 3.4 Cost optimization fundamentals

1. **Tag everything** — Environment, Project, Owner, CostCenter
2. **Right-size** — Use Compute Optimizer recommendations
3. **Stop dev resources** — Schedule EC2/RDS shutdown nights/weekends
4. **Use Spot** for fault-tolerant batch workloads
5. **S3 lifecycle** — Move old data to IA/Glacier
6. **Reserved capacity** — 1–3 year Savings Plans for steady workloads
7. **Delete unused** — EBS volumes, Elastic IPs, old snapshots

1.5 3.4 Consolidated billing with Organizations

AWS Organizations lets you manage multiple accounts under one payer account:

- **Consolidated billing** — Single invoice, volume discounts
- **SCPs** — Guardrails on what accounts can do
- **Separate accounts** per environment (dev/staging/prod) or per team

Recommended for any serious AWS usage.

1.6 3.5 Support plans (overview)

Plan	Best for
Basic	Free; account and billing support
Developer	Business hours email support
Business	24/7 phone, < 1 hr response for production down
Enterprise	TAM, < 15 min critical response

1.7 3.6 Chapter summary

- Secure the **root account** with MFA; use IAM for daily work.
- Set **budgets and alerts** on day one.
- **Tag** resources and review Cost Explorer weekly during learning.

1.8 Lab 3.1

1. Enable MFA on root.
 2. Create IAM admin user `admin-yourname`.
 3. Create a \$10 monthly budget with email alert.
 4. Enable Cost Explorer (may take 24h for data).
-

Next: [*Chapter 6 — IAM Best Practices*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 4

1 Chapter 4: AWS CLI, CloudShell, and SDK Overview

1.1 4.1 Why the CLI matters for DevOps

The **AWS Management Console** is excellent for exploration, but production workflows demand **repeatability**, **version control**, and **automation**. The **AWS Command Line Interface (CLI)** is the universal tool that every AWS service exposes through a consistent interface. Combined with **CloudShell** (browser-based shell) and **SDKs** (programmatic libraries), you can script deployments, integrate CI/CD pipelines, and build applications that call AWS APIs directly.

DevOps engineers typically spend 70–80% of their AWS interaction time in CLI and IaC tools, reserving the console for debugging and one-off investigations.

1.2 4.2 Installing and configuring the AWS CLI

1.2.1 Installation options

Platform	Method
Linux/macOS	<code>curl</code> installer or package manager
Windows	MSI installer or <code>winget</code>
CI/CD	Pre-installed on GitHub Actions runners; use in Docker
CloudShell	Pre-installed; no setup required

1.2.2 AWS CLI v2 vs v1

Always use **CLI v2**. It supports SSO, improved pagination, `aws s3` performance improvements, and interactive `aws configure sso`.

1.2.3 Initial configuration

```
# Interactive setup with long-term access keys (avoid for daily use; prefer SSO)
aws configure
# AWS Access Key ID: AKIA...
# AWS Secret Access Key: ...
# Default region name: us-east-1
# Default output format: json

# Verify identity
aws sts get-caller-identity
```

Configuration files live at:

File	Purpose
<code>~/.aws/credentials</code>	Access keys per profile

~/aws/config	Region, output format, SSO settings
--------------	-------------------------------------

1.2.4 Named profiles

```
aws configure --profile dev
aws s3 ls --profile dev

# Environment variable override
export AWS_PROFILE=prod
aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId'
```

1.2.5 SSO configuration (recommended for humans)

```
aws configure sso
# SSO session name: my-org
# SSO start URL: https://my-org.awsapps.com/start
# SSO region: us-east-1
# CLI default client Region: us-east-1

aws sso login --profile dev-admin
aws sts get-caller-identity --profile dev-admin
```

1.3 4.3 AWS CloudShell

AWS CloudShell is a browser-based shell environment pre-authenticated with your console credentials. It includes the AWS CLI v2, Python, Node.js, Git, and 1 GB persistent storage per region.

1.3.1 When to use CloudShell

Use CloudShell	Use local CLI
Quick API calls from console	Daily development
No local install available	CI/CD pipelines
Temporary troubleshooting	Scripts in version control
Learning and labs	Large file transfers

1.3.2 CloudShell features

- **Automatic credentials** — Uses your current console session IAM permissions.
- **Persistent home directory** — Files survive across sessions (per region).
- **Network access** — Can reach public AWS APIs; VPC access requires additional setup.
- **Limitations** — 1 GB storage, no inbound connections, session timeout after inactivity.

```
# Example: list S3 buckets from CloudShell
aws s3 ls

# Install a Python package (persists in home directory)
pip install --user boto3
```

1.4 4.4 CLI essentials: structure and patterns

Every AWS CLI command follows this pattern:

```
aws <service> <operation> [--parameters] [--global-options]
```

1.4.1 Global options

Option	Purpose
<code>--region</code>	Override default region
<code>--profile</code>	Use named profile
<code>--output</code>	<code>json</code> , <code>yaml</code> , <code>text</code> , <code>table</code>
<code>--query</code>	JMESPath filter on response
<code>--dry-run</code>	Validate without executing (where supported)
<code>--no-cli-pager</code>	Disable pager for scripts

1.4.2 JMESPath query examples

```
# List running EC2 instance IDs only
aws ec2 describe-instances \
  --filters "Name=instance-state-name,Values=running" \
  --query 'Reservations[].Instances[].InstanceId' \
  --output text

# Get bucket names and creation dates as a table
aws s3api list-buckets \
  --query 'Buckets[].{Name:Name,Created:CreationDate}' \
  --output table
```

1.4.3 Pagination

```
# CLI v2 auto-paginates most list operations
aws ec2 describe-instances --output json

# Manual control
aws s3api list-objects-v2 --bucket my-bucket --max-items 100
```

1.4.4 Waiters

```
aws ec2 run-instances --image-id ami-0c55b159cbfafa1f0 --instance-type t3.micro ...

aws ec2 wait instance-running --instance-ids i-0abc123
aws ec2 wait instance-status-ok --instance-ids i-0abc123
```

1.5 4.5 AWS SDK overview

SDKs wrap AWS APIs in language-native libraries. They handle request signing, retries, and pagination.

SDK	Language	Package
Boto3	Python	<code>pip install boto3</code>
AWS SDK for JavaScript v3	Node.js	<code>@aws-sdk/client-s3</code>
AWS SDK for Go v2	Go	<code>github.com/aws/aws-sdk-go-v2</code>
AWS SDK for .NET	C#	<code>AWSSDK.S3</code>

1.5.1 Credential chain (SDK and CLI)

Both resolve credentials in this order:

1. Environment variables (`AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY`)

2. Shared credentials file (~/.aws/credentials)
3. IAM role (EC2 instance profile, ECS task role, Lambda execution role)
4. SSO cached credentials

Best practice: Applications on AWS use **IAM roles**, never hard-coded keys.

1.5.2 Boto3 example

```
import boto3

s3 = boto3.client("s3", region_name="us-east-1")
response = s3.list_buckets()
for bucket in response["Buckets"]:
    print(bucket["Name"])
```

1.5.3 SDK with assumed role

```
import boto3

sts = boto3.client("sts")
assumed = sts.assume_role(
    RoleArn="arn:aws:iam::123456789012:role/CrossAccountRole",
    RoleSessionName="my-session",
)
creds = assumed["Credentials"]
client = boto3.client(
    "s3",
    aws_access_key_id=creds["AccessKeyId"],
    aws_secret_access_key=creds["SecretAccessKey"],
    aws_session_token=creds["SessionToken"],
)
```

1.6 4.6 Terraform provider configuration

Terraform uses the AWS provider, which shares the same credential chain as the CLI.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
  profile = "dev"
}

resource "aws_s3_bucket" "logs" {
  bucket = "my-app-logs-${data.aws_caller_identity.current.account_id}"
}

data "aws_caller_identity" "current" {}
```

1.7 4.7 CLI security best practices

Practice	Rationale
Use SSO, not long-term access keys	Keys can leak; SSO sessions expire
Never commit credentials to Git	Use IAM roles in CI/CD

Use <code>--profile</code> or <code>AWS_PROFILE</code>	Separate dev/staging/prod
Enable MFA for sensitive operations	Extra layer for destructive actions
Rotate any remaining access keys	Limit blast radius
Use <code>aws sts get-caller-identity</code> before destructive commands	Confirm correct account

1.8 4.8 Useful CLI productivity tips

```
# Auto-complete (bash)
complete -C '/usr/local/bin/aws_completer' aws

# Generate CLI skeleton for complex commands
aws ec2 run-instances --generate-cli-skeleton input > run-instance.json
# Edit JSON, then:
aws ec2 run-instances --cli-input-json file://run-instance.json

# Filter help by service
aws ec2 help | grep describe

# Export credentials for third-party tools (short-lived)
eval "$(aws configure export-credentials --profile dev --format env)"
```

1.9 4.9 Chapter summary

- The **AWS CLI** is the foundation of automation; configure profiles and prefer **SSO**.
- **CloudShell** provides instant CLI access from the console without local setup.
- **SDKs** (Boto3, etc.) use the same credential chain; applications on AWS should use **IAM roles**.
- Master **JMESPath queries**, **waiters**, and **pagination** for efficient scripting.
- Terraform and other IaC tools inherit CLI credential configuration.

1.10 Lab 4.1 — CLI and CloudShell workflow

1. Install AWS CLI v2 locally (or open **CloudShell** in the console).
2. Configure an SSO profile: `aws configure sso`.
3. Run `aws sts get-caller-identity` and note your account ID and ARN.
4. List all regions: `aws ec2 describe-regions --query 'Regions[].RegionName' --output table`.
5. Create a test S3 bucket (unique name): `aws s3 mb s3://cli-lab-$(aws sts get-caller-identity --query Account --output text)-$(date +%s)`.
6. Upload a file: `echo "hello" > test.txt && aws s3 cp test.txt s3://YOUR-BUCKET/`.
7. Delete the bucket and objects when done.

1.11 Lab 4.2 — Boto3 script

1. On an EC2 instance with an instance profile (or locally with configured credentials), write a Python script that lists all EC2 instances and prints ID, type, and state.
2. Run it with `python3 list_instances.py`.
3. Extend the script to tag instances missing an `Owner` tag.

1.12 Review questions

1. What is the difference between `~/.aws/credentials` and `~/.aws/config`?
 2. How does the SDK credential chain resolve credentials on an EC2 instance with an instance profile?
 3. What JMESPath query would return only S3 bucket names from `aws s3api list-buckets`?
 4. When should you use CloudShell instead of a local CLI installation?
 5. Why is SSO preferred over long-term access keys for human users?
-

Next: [*Chapter 5 — IAM Fundamentals*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 5

1 Chapter 5: IAM Users, Groups, Roles, and Policies

1.1 5.1 What is IAM?

AWS Identity and Access Management (IAM) controls **who** can access **which** AWS resources and **how**. Every API call to AWS is authenticated and authorized through IAM (except public resources like anonymous S3 access).

IAM is **global**—users and roles exist at the account level, not per region.

1.2 5.2 IAM identity types

Identity	Use case
Root user	Account owner; created at signup. Avoid for daily use.
IAM user	Long-term credentials for humans or legacy apps
IAM role	Temporary credentials; assumed by users, services, or other accounts
IAM group	Collection of users with shared permissions

Best practice: Humans use **IAM Identity Center (SSO)** or roles with MFA. Applications and EC2/Lambda use **roles**, not access keys on users.

1.3 5.3 Policies — the core of permissions

A **policy** is a JSON document defining **Allow** or **Deny** for **actions** on **resources** under **conditions**.

1.3.1 Policy structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Read",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-app-bucket",
        "arn:aws:s3::my-app-bucket/*"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

```
}  
]  
}
```

1.3.2 Policy types

Type	Attached to
Identity-based	Users, groups, roles
Resource-based	S3 buckets, SNS topics, etc.
Permission boundaries	Max permissions for identity
SCP	AWS Organizations — account ceiling
Session policies	Limit role session scope

Evaluation logic: Explicit **Deny** always wins. Default is implicit deny until Allow.

1.4 5.4 Users and groups

1.4.1 Create a developer group (example workflow)

1. Create group `Developers`.
2. Attach policy `PowerUserAccess` (or custom least-privilege policy).
3. Create users; add to group.
4. Enable **MFA** for console access.
5. Provide **programmatic access** only if needed (prefer roles).

```
aws iam create-group --group-name Developers  
aws iam attach-group-policy --group-name Developers \  
--policy-arn arn:aws:iam::aws:policy/PowerUserAccess  
aws iam create-user --user-name alice  
aws iam add-user-to-group --user-name alice --group-name Developers
```

1.5 5.5 Roles and trust policies

A **role** has two policies:

1. **Trust policy** — Who can assume the role
2. **Permission policy** — What the role can do

1.5.1 EC2 instance role example

Trust policy (allows EC2 service to assume):

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Principal": { "Service": "ec2.amazonaws.com" },  
      "Action": "sts:AssumeRole"  
    }  
  ]  
}
```

Attach permission policy allowing `s3:GetObject` on a specific bucket. The EC2 instance receives **temporary credentials** via instance metadata—no hardcoded keys.

1.5.2 Assume role (cross-account or CLI)

```
aws sts assume-role \  
  --role-arn arn:aws:iam::123456789012:role/DeployRole \  
  --role-session-name deploy-session
```

1.6 5.6 AWS managed vs customer managed policies

Type	Description
AWS managed	Maintained by AWS (<code>ReadOnlyAccess</code> , <code>AdministratorAccess</code>)
Customer managed	You create and reuse across identities
Inline	Embedded in single user/group/role; harder to reuse

Prefer **customer managed** policies for team standards; avoid `AdministratorAccess` except break-glass accounts.

1.7 5.7 Least privilege in practice

1. Start with **read-only** access; add permissions as needed.
2. Use **IAM Access Analyzer** to find overly broad access.
3. Scope resources with ARNs, not `"Resource": "*" .`
4. Use **conditions** (MFA present, source IP, tag-based).
5. Rotate or eliminate **access keys**; prefer roles.

1.7.1 Tag-based access (ABAC)

```
"Condition": {  
  "StringEquals": {  
    "aws:PrincipalTag/Department": "${aws:ResourceTag/Department}"  
  }  
}
```

Users tagged `Department=Engineering` access only resources tagged the same.

1.8 5.8 Common IAM actions for DevOps

Action prefix	Service
<code>ec2:*</code>	EC2
<code>s3:*</code>	S3
<code>iam:PassRole</code>	Allow attaching roles to services
<code>sts:AssumeRole</code>	Switch roles
<code>cloudformation:*</code>	Stack operations

`iam:PassRole` is often required when CI/CD deploys resources that need an instance/task role.

1.9 5.9 IAM Access Analyzer & CloudTrail

- **Access Analyzer** — Identifies resources shared externally.
- **CloudTrail** — Logs every IAM API call for audit.

Enable CloudTrail in all regions for security accounts.

1.10 5.10 Chapter summary

- IAM controls authentication and authorization globally.
 - Use **groups** for humans, **roles** for services and temporary access.
 - Policies are JSON with Effect, Action, Resource, Condition.
 - Apply **least privilege**; avoid root and long-lived access keys.
-

1.11 Lab 5.1 — IAM hands-on

1. Create IAM user `lab-user` with MFA.
 2. Create customer managed policy allowing `s3:ListBucket` and `s3:GetObject` on one bucket.
 3. Create role `EC2-S3-ReadRole` with trust for EC2 and the S3 read policy.
 4. Launch EC2 with the role; verify `aws s3 ls` works without configured keys.
 5. Review effective permissions in **IAM Policy Simulator**.
-

1.12 Review questions

1. What is the difference between an IAM user and an IAM role?
 2. What happens when Allow and Deny conflict?
 3. Why should EC2 instances use roles instead of access keys?
 4. What does `sts:AssumeRole` do?
-

Continue: Chapter 6 — IAM Best Practices (outline in TOC)

AWS Cloud Handbook — Complete Edition —

Chapter 6

1 Chapter 6: IAM Best Practices and Permission Boundaries

1.1 6.1 The IAM security baseline

Follow these rules in every AWS account:

1. **Root account** — MFA only; no access keys; break-glass use only
 2. **No long-lived access keys** for humans — use SSO + temporary credentials
 3. **Least privilege** — Grant minimum permissions required
 4. **MFA** — Required for console access and sensitive API calls
 5. **Rotate keys** — If keys must exist, rotate every 90 days max
 6. **Audit** — CloudTrail enabled in all regions; review Access Analyzer findings
-

1.2 6.2 Prefer roles over users for applications

Identity	Use for
IAM user + keys	Legacy; avoid for new projects
IAM role	EC2, Lambda, ECS, cross-account, CI/CD
IAM Identity Center	Human workforce access

1.2.1 CI/CD role pattern

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:*"
      }
    }
  }]
}
```

GitHub Actions assumes this role—no static AWS keys in GitHub Secrets.

1.3 6.3 Permission boundaries

A **permissions boundary** sets the **maximum** permissions an identity can have—even if policies grant more.

Use for:

- Delegating IAM admin to developers without full account access
- Sandbox accounts with caps

```
aws iam put-user-permissions-boundary \  
  --user-name developer \  
  --permissions-boundary arn:aws:iam::aws:policy/PowerUserAccess
```

1.4 6.4 Policy writing tips

1.4.1 Start with AWS managed, refine to customer managed

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "DeployToSpecificBucket",  
    "Effect": "Allow",  
    "Action": ["s3:PutObject", "s3:GetObject"],  
    "Resource": "arn:aws:s3::my-app-artifacts-prod/*"  
  }]  
}
```

1.4.2 Use IAM Policy Simulator

Test policies before deploying: AWS Console → IAM → Policy Simulator.

1.4.3 Avoid these anti-patterns

- "Action": "*" on production roles
- "Resource": "*" when ARNs are known
- Attaching AdministratorAccess to CI roles
- Sharing one IAM user across a team

1.5 6.5 Service-linked roles

AWS creates **service-linked roles** for services (e.g., AWSServiceRoleForElasticLoadBalancing). Do not delete these unless AWS docs say it's safe.

1.6 6.6 Cross-account access

Pattern: Account A (workload) trusts Account B (CI/CD) via role assumption.

```
# From CI in account B  
aws sts assume-role \  
  --role-arn arn:aws:iam::WORKLOAD_ACCOUNT:role/DeployRole \  
  --role-session-name github-deploy
```

Use **AWS Organizations** SCPs to prevent privilege escalation across accounts.

1.7 6.7 Chapter summary

- Humans: **SSO + MFA**. Apps: **roles**. CI: **OIDC federation**.
 - Write **scoped customer-managed policies**; test with Policy Simulator.
 - Use **permission boundaries** and **SCPs** for guardrails.
-

1.8 Lab 6.1

1. Create a CI/CD role trust policy for GitHub OIDC (or a second AWS account).
 2. Attach a least-privilege policy for S3 deploy only.
 3. Run Access Analyzer and remediate any `Public` findings.
-

Next: [*Chapter 9 — VPC Fundamentals*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 7

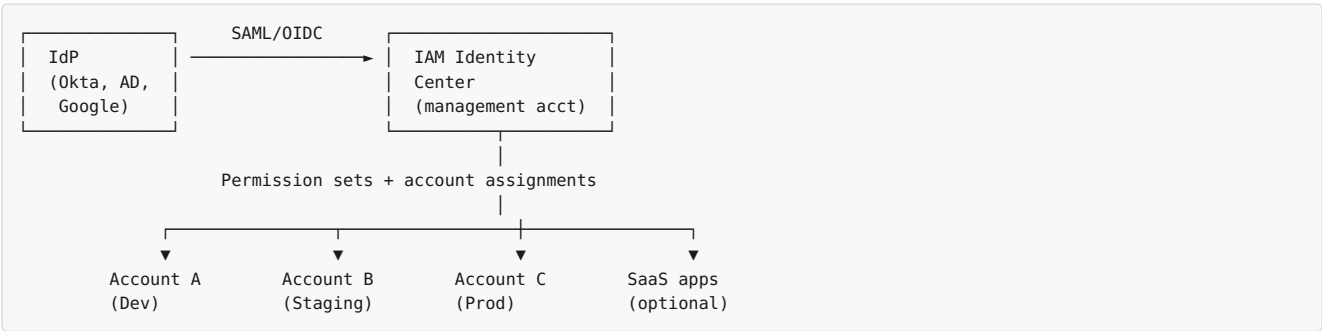
1 Chapter 7: IAM Identity Center (SSO) & Federation

1.1 7.1 The problem with IAM users at scale

Managing hundreds of **IAM users** across dozens of AWS accounts does not scale. Each user needs credentials, MFA enrollment, password rotation, and offboarding when they leave. **IAM Identity Center** (formerly AWS Single Sign-On) centralizes workforce access: users authenticate once through your corporate identity provider and receive temporary credentials to AWS accounts and applications.

For organizations with multiple accounts, Identity Center is the **recommended** human access pattern, replacing per-account IAM users.

1.2 7.2 IAM Identity Center architecture



1.2.1 Key components

Component	Description
Identity Center instance	Deployed in a home region; one per organization
Identity source	Built-in directory, or federation with external IdP
Permission set	Template of IAM policies assigned to users/groups
Account assignment	Maps user/group + permission set → AWS account
Access portal	<code>https://d-xxxxxxxxx.awsapps.com/start</code>

1.3 7.3 Permission sets

A **permission set** is a collection of IAM policies that define what a user can do in a target account. When assigned, Identity Center creates an **IAM role** in the target account (prefixed `AWSReservedSSO_`).

1.3.1 Common permission sets

Permission set	
----------------	--

	Typical use
AdministratorAccess	Platform team, break-glass
PowerUserAccess	Developers (no IAM changes)
ReadOnlyAccess	Auditors, observers
Custom (least privilege)	Application-specific access

1.3.2 Custom permission set example (Terraform)

```
resource "aws_ssoadmin_permission_set" "developer" {
  name           = "DeveloperAccess"
  instance_arn   = tolist(data.aws_ssoadmin_instances.main.arns)[0]
  session_duration = "PT4H" # ISO 8601 duration: 4 hours
}

resource "aws_ssoadmin_managed_policy_attachment" "developer_poweruser" {
  instance_arn       = tolist(data.aws_ssoadmin_instances.main.arns)[0]
  permission_set_arn = aws_ssoadmin_permission_set.developer.arn
  managed_policy_arn = "arn:aws:iam::aws:policy/PowerUserAccess"
}

resource "aws_ssoadmin_account_assignment" "dev_team" {
  instance_arn       = tolist(data.aws_ssoadmin_instances.main.arns)[0]
  permission_set_arn = aws_ssoadmin_permission_set.developer.arn
  principal_id       = aws_identitystore_group.developers.group_id
  principal_type      = "GROUP"
  target_id          = "123456789012" # Dev account ID
  target_type        = "AWS_ACCOUNT"
}
```

1.4 7.4 Federation with external identity providers

1.4.1 SAML 2.0 federation

Most common for enterprise IdPs (Okta, Azure AD, PingFederate):

1. Configure SAML application in your IdP.
2. In Identity Center, set identity source to **External identity provider**.
3. Exchange metadata (IdP → AWS and AWS → IdP).
4. Map IdP attributes to Identity Center users and groups.

1.4.2 Attribute mapping

IdP attribute	Identity Center field
email	Username
givenName	First name
sn	Last name
memberOf	Group membership (via SCIM or manual)

1.4.3 SCIM provisioning

System for Cross-domain Identity Management (SCIM) automates user and group lifecycle:

- User created in Okta → automatically appears in Identity Center
- User deactivated in Okta → access revoked in AWS
- Group membership changes → permission set assignments update

Enable SCIM in Identity Center settings and configure your IdP's provisioning connector.

1.5 7.5 Accessing AWS with Identity Center

1.5.1 AWS access portal

Users visit the portal URL, authenticate via IdP, and see assigned accounts and roles. Clicking an account/role opens the console or provides CLI credentials.

1.5.2 CLI access

```
aws configure sso
# SSO start URL: https://d-xxxxxxxxx.awsapps.com/start
# SSO region: us-east-1
# Account: 123456789012
# Role: DeveloperAccess

aws sso login --profile dev
aws sts get-caller-identity --profile dev
```

1.5.3 Programmatic access from CI/CD

CI/CD should **not** use human SSO sessions. Instead:

- Use **IAM roles** with OIDC federation (GitHub Actions → AWS)
- Use **IAM roles** for EC2/ECS/Lambda
- Use short-lived credentials via `sts:AssumeRole`

1.6 7.6 IAM federation without Identity Center

For single-account or legacy setups, you can federate directly:

Method	Use case
SAML 2.0	Enterprise IdP → IAM role
Web Identity (OIDC)	Google, Facebook, Amazon login
Cognito Identity Pools	Mobile/web apps with federated identities

1.6.1 SAML role trust policy example

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:saml-provider/MyCorp"
      },
      "Action": "sts:AssumeRoleWithSAML",
      "Condition": {
        "StringEquals": {
          "SAML:aud": "https://signin.aws.amazon.com/saml"
        }
      }
    }
  ]
}
```

1.7 7.7 Multi-account access patterns

Pattern	Description
Account per environment	dev, staging, prod as separate accounts
Account per team	Team A owns accounts for their services
Account per workload	PCI workloads isolated from general workloads
Management account	Identity Center, Organizations, billing only

1.7.1 Session duration

Permission sets support session duration from **1 hour to 12 hours**. Shorter sessions reduce risk; longer sessions improve developer experience. Production admin access should use shorter durations (1-2 hours).

1.8 7.8 Security best practices

Practice	Why
Use Identity Center for all human access	Centralized audit, MFA, offboarding
Assign permissions via groups, not individuals	Easier lifecycle management
Enable SCIM provisioning	Automatic deprovisioning
Use least-privilege permission sets	Limit blast radius per account
Require MFA at the IdP level	Defense in depth
Audit assignments regularly	Remove stale access
Separate break-glass admin	Emergency access with extra logging

1.8.1 Auditing SSO access

```
# CloudTrail logs AssumeRoleWithSAML and SSO portal logins
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRoleWithSAML \
  --max-results 10
```

1.9 7.9 Chapter summary

- **IAM Identity Center** replaces per-account IAM users with centralized SSO across accounts.
- **Permission sets** define what users can do; **account assignments** map them to specific accounts.
- **SAML/OIDC federation** connects corporate IdPs; **SCIM** automates user lifecycle.
- CLI access via `aws configure sso` and `aws sso login`.
- CI/CD and services should use **IAM roles**, not SSO sessions.

1.10 Lab 7.1 — Identity Center setup

1. In the management account, enable **IAM Identity Center** (if not already enabled).
2. Create a permission set `LabReadOnly` with the `ViewOnlyAccess` AWS managed policy.
3. Create a user (or use your IdP) and assign them to a dev account with `LabReadOnly`.
4. Log in via the access portal and verify you can view but not modify resources.

5. Configure CLI SSO: `aws configure sso` and run `aws s3 ls` with the new profile.

1.11 Lab 7.2 — Audit permission assignments

1. List all account assignments in Identity Center (console or CLI).
 2. Identify any assignments with `AdministratorAccess` to production accounts.
 3. Document a plan to replace them with least-privilege permission sets.
-

1.12 Review questions

1. What is the difference between a permission set and an account assignment?
 2. How does SCIM improve security compared to manual user management?
 3. Why should CI/CD pipelines not use Identity Center SSO sessions?
 4. What IAM role prefix does Identity Center create in target accounts?
 5. Where should MFA be enforced: the IdP, Identity Center, or both?
-

Next: [Chapter 8 — AWS Organizations & SCPs](#)

AWS Cloud Handbook — Complete Edition —

Chapter 8

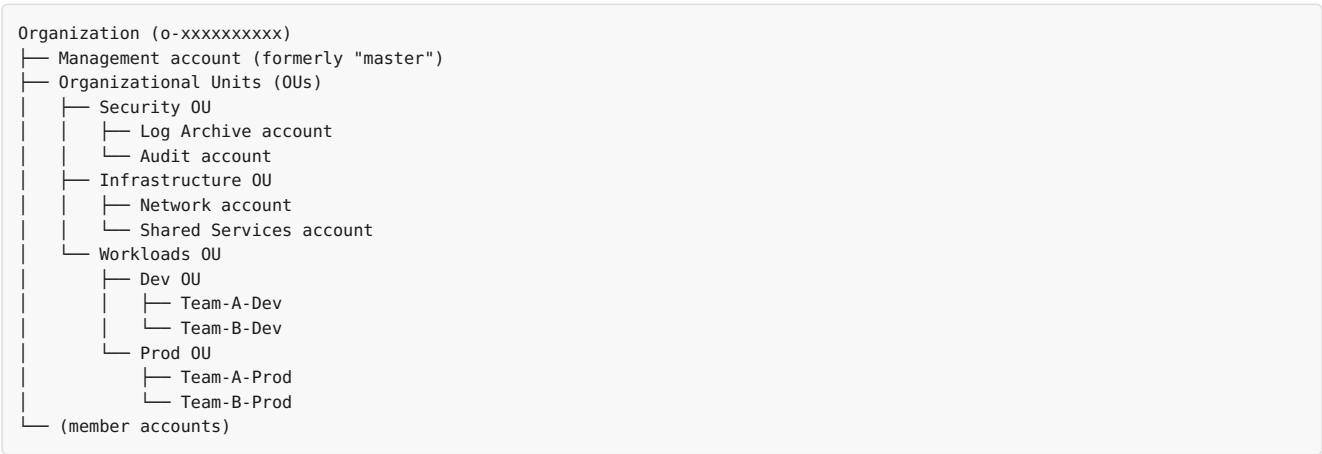
1 Chapter 8: AWS Organizations & Service Control Policies

1.1 8.1 Why AWS Organizations?

A single AWS account works for small teams, but enterprises need **multiple accounts** for security isolation, blast-radius containment, billing separation, and compliance boundaries. **AWS Organizations** is a free service that centrally manages a hierarchy of accounts under one **organization**.

Without Organizations, you cannot use Service Control Policies, consolidated billing, or automated account vending.

1.2 8.2 Organization structure



1.2.1 Key concepts

Concept	Description
Management account	Pays for all member accounts; hosts Organizations, Identity Center
Member account	Standard AWS account within the organization
Organizational Unit (OU)	Grouping of accounts for policy application
Root	Top of the hierarchy; contains all OUs and accounts

1.3 8.3 Consolidated billing

Organizations provides a **single bill** for all accounts:

- **Volume discounts** aggregate across accounts (e.g., S3, Data Transfer).

- **Cost allocation tags** propagate for chargeback.
- **AWS Budgets** and **Cost Explorer** work across the organization.
- **Reserved Instances and Savings Plans** can be shared within the org.

```
# List all accounts in the organization
aws organizations list-accounts \
  --query 'Accounts[].{Id:Id,Name:Name,Status:Status}' \
  --output table
```

1.4 8.4 Service Control Policies (SCPs)

SCPs are guardrails that set the **maximum permissions** for accounts or OUs. They do not grant permissions—they **filter** what identity-based policies can allow.

1.4.1 SCP evaluation

Effective permissions = Identity policies $\cap$ SCPs (and permission boundaries)

- **Explicit Deny** in an SCP always wins.
- If no SCP allows an action, it is implicitly denied (for SCP scope).
- SCPs affect all principals **except** the management account root user and certain service-linked roles.

1.4.2 Example: Deny leaving the organization

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyLeaveOrganization",
      "Effect": "Deny",
      "Action": "organizations:LeaveOrganization",
      "Resource": "*"
    }
  ]
}
```

1.4.3 Example: Restrict regions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyUnapprovedRegions",
      "Effect": "Deny",
      "NotAction": [
        "iam:*",
        "organizations:*",
        "route53:*",
        "support:*",
        "cloudfront:*",
        "globalaccelerator:*"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["us-east-1", "eu-west-1"]
        }
      }
    }
  ]
}
```

1.4.4 Example: Deny root user actions

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyRootUser",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringLike": {
          "aws:PrincipalArn": "arn:aws:iam::*:root"
        }
      }
    }
  ]
}

```

1.5 8.5 Common SCP strategies

SCP purpose	Approach
Region lock	Deny all actions outside approved regions
Prevent privilege escalation	Deny <code>iam:CreateUser</code> , <code>iam:AttachUserPolicy</code> in workload accounts
Require encryption	Deny <code>S3 PutObject</code> without <code>s3:x-amz-server-side-encryption</code>
Block expensive services	Deny <code>ec2:RunInstances</code> for large instance types in dev
Protect security services	Deny disabling GuardDuty, CloudTrail, Config

1.5.1 Terraform SCP attachment

```

resource "aws_organizations_policy" "deny_root" {
  name     = "DenyRootUser"
  type     = "SERVICE_CONTROL_POLICY"
  content = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid      = "DenyRootUser"
      Effect   = "Deny"
      Action   = "*"
      Resource = "*"
      Condition = {
        StringLike = {
          "aws:PrincipalArn" = "arn:aws:iam::*:root"
        }
      }
    }]
  })
}

resource "aws_organizations_policy_attachment" "workloads" {
  policy_id = aws_organizations_policy.deny_root.id
  target_id = aws_organizations_organizational_unit.workloads.id
}

```

1.6 8.6 Account vending with Control Tower

AWS Control Tower builds on Organizations to provide:

- **Account Factory** — Automated account creation with baseline guardrails.

- **Guardrails** — Pre-built SCPs (mandatory and elective).
- **Account Factory for Terraform (AFT)** — Terraform-based account vending.

Typical landing zone accounts:

Account	Purpose
Management	Organizations, Identity Center, billing
Log Archive	Centralized CloudTrail, Config logs
Audit	Security tooling (GuardDuty admin, Security Hub)
Network	Transit Gateway, VPN, Direct Connect
Shared Services	CI/CD, artifact repos, DNS

1.7 8.7 Delegated administration

Certain security services can be administered from a **delegated admin account** instead of the management account:

Service	Delegated admin use
GuardDuty	Central threat detection
Security Hub	Aggregated findings
AWS Config	Aggregator for compliance
CloudTrail	Organization trail management
Firewall Manager	WAF rules across accounts

```
aws organizations enable-aws-service-access \
  --service-principal guardduty.amazonaws.com

aws guardduty enable-organization-admin-account \
  --admin-account-id 111111111111
```

1.8 8.8 SCP troubleshooting

Symptom	Likely cause
AccessDeniedException for allowed IAM policy	SCP denies the action
Works in management account, fails in member	SCP attached to member OU
Service-linked role fails	SCP may need exception for that role
Cannot enable a service	SCP blocks the enable API

1.8.1 Testing SCP impact

```
# Use IAM Policy Simulator with Organizations context
# Or: create a test role in a sandbox account with the SCP attached
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:role/TestRole \
  --action-names ec2:RunInstances \
  --resource-arns "*" "
```

1.9 8.9 Chapter summary

- **AWS Organizations** manages multi-account hierarchies with consolidated billing.
 - **OUs** group accounts for policy application.
 - **SCPs** set permission guardrails; they filter but do not grant access.
 - **Control Tower** provides a landing zone with automated account vending.
 - **Delegated administration** centralizes security services in a dedicated account.
-

1.10 Lab 8.1 — Explore your organization

1. Run `aws organizations describe-organization` and note the organization ID.
2. List all accounts: `aws organizations list-accounts`.
3. Identify which OUs exist and which accounts belong to each.
4. List SCPs: `aws organizations list-policies --filter SERVICE_CONTROL_POLICY`.

1.11 Lab 8.2 — Create a sandbox SCP

1. In a **sandbox OU** (never production first), create an SCP that denies `ec2:RunInstances` for instance types larger than `xlarge`.
 2. Attach the SCP to the sandbox OU.
 3. Attempt to launch a `m5.2xlarge` — verify denial.
 4. Launch a `t3.micro` — verify success.
 5. Detach and delete the SCP when done.
-

1.12 Review questions

1. Do SCPs grant permissions or restrict them?
 2. Which account is exempt from SCP restrictions?
 3. How do consolidated billing volume discounts work across accounts?
 4. What is the difference between an OU and an AWS account?
 5. Why should you test SCPs in a sandbox before applying to production OUs?
-

Next: [Chapter 9 — VPC Fundamentals](#)

AWS Cloud Handbook — Complete Edition —

Chapter 9

1 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables

1.1 9.1 What is a VPC?

A **Virtual Private Cloud (VPC)** is your isolated network in AWS. You control IP ranges, subnets, routing, and firewalls.

Every AWS account has a **default VPC** per region (with public subnets). Production workloads use **custom VPCs** with explicit design.

1.2 9.2 CIDR and IP planning

CIDR (Classless Inter-Domain Routing) defines IP ranges:

CIDR	Addresses	Usable (AWS reserves 5)
10.0.0.0/16	65,536	~65,531
10.0.1.0/24	256	~251
10.0.1.0/28	16	~11

1.2.1 Example VPC design

VPC: 10.0.0.0/16

Public subnets (ALB, NAT Gateway):
10.0.1.0/24 — us-east-1a
10.0.2.0/24 — us-east-1b

Private subnets (app servers, ECS, Lambda):
10.0.10.0/24 — us-east-1a
10.0.11.0/24 — us-east-1b

Database subnets (RDS, no internet):
10.0.20.0/24 — us-east-1a
10.0.21.0/24 — us-east-1b

Tip: Plan for growth; changing VPC CIDR later is painful. Use /16 for VPC, /24 for subnets.

1.3 9.3 Subnets

Subnets live in **one Availability Zone**. Types:

Type	Route to IGW	Typical workloads
Public	Yes (via route table)	ALB, NAT Gateway, bastion
Private	No (via NAT Gateway)	App servers, ECS, Lambda

Private	No direct IGW	App servers, workers
Isolated	No NAT either	Databases

`map_public_ip_on_launch = true` assigns public IPs to instances in public subnets.

1.4 9.4 Route tables

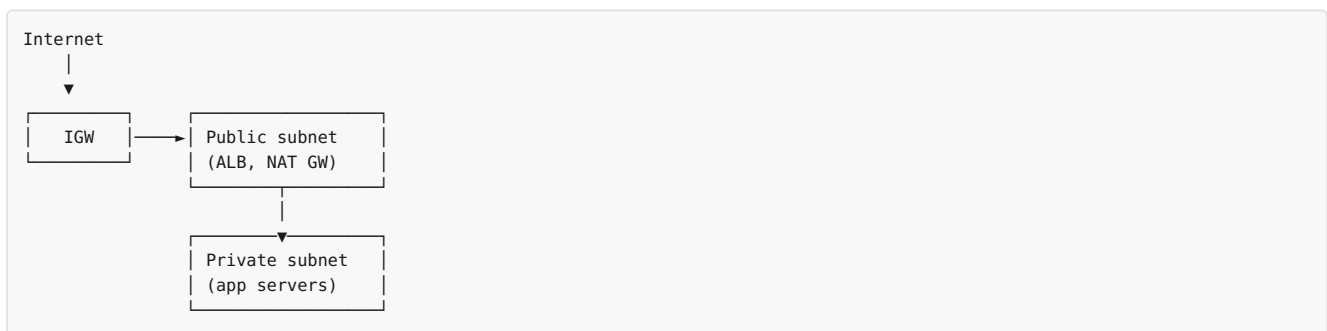
A **route table** defines where traffic goes:

Destination	Target	Meaning
10.0.0.0/16	local	Traffic within VPC
0.0.0.0/0	igw-xxxxx	Internet via Internet Gateway
0.0.0.0/0	nat-xxxxx	Internet via NAT Gateway (private subnet)
pl-xxxxx	VPC Endpoint	Private AWS service access

Each subnet associates with **one** route table.

1.5 9.5 Internet Gateway and NAT (preview)

- **Internet Gateway (IGW)** — VPC component for bidirectional internet access
- **NAT Gateway** — Allows private subnet instances to reach internet (outbound only); place in public subnet



1.6 9.6 Security Groups vs NACLs

	Security Group	NACL
Level	Instance/ENI	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny
Default	Deny all inbound	Allow all

Best practice: Use security groups as primary firewall; NACLs for subnet-level deny rules if needed.

Example security group for web tier:

- Inbound: 443 from ALB security group
- Outbound: 5432 to database security group

1.7 9.7 Terraform VPC snippet

```
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "prod-vpc" }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
  map_public_ip_on_launch = true
}
```

1.8 9.8 Chapter summary

- Design VPCs with **public**, **private**, and **database** subnet tiers across **multiple AZs**.
- **Route tables** control internet and local routing.
- **Security groups** are stateful instance firewalls; default deny inbound.

1.9 Lab 9.1

1. Create a custom VPC with 2 public and 2 private subnets in different AZs.
2. Attach an Internet Gateway and configure route tables.
3. Launch an EC2 in a private subnet (no public IP) and verify no direct internet access.

Next: [Chapter 10 — NAT and Routing](#)

AWS Cloud Handbook — Complete Edition —

Chapter 10

1 Chapter 10: Internet Gateway, NAT Gateway, and Routing

1.1 10.1 Internet Gateway (IGW)

An **Internet Gateway** is a horizontally scaled, redundant VPC component that allows:

- Instances with public IPs to communicate with the internet
- Internet to initiate connections to instances with public IPs

Attach one IGW per VPC (for internet-connected VPCs).

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id igw-xxx --vpc-id vpc-xxx
```

Public subnet route table:

Destination	Target
0.0.0.0/0	igw-xxxxxxx
10.0.0.0/16	local

1.2 10.2 NAT Gateway

Private subnet instances should **not** have public IPs. They reach the internet (for updates, API calls) via a **NAT Gateway** in a public subnet.

```
Private subnet instance (10.0.10.5)
→ NAT Gateway (public subnet, elastic IP)
→ Internet Gateway
→ Internet
```

1.2.1 NAT Gateway facts

- **Managed** by AWS (no patching)
- **Charged** per hour + per GB processed
- **AZ-specific** — deploy one NAT per AZ for HA, or accept cross-AZ failover tradeoffs
- **Alternative:** NAT instances (self-managed, not recommended)

Private subnet route table:

Destination	Target
0.0.0.0/0	nat-xxxxxxx
10.0.0.0/16	local

1.3 10.3 VPC Endpoints — avoid NAT costs

For AWS service traffic (S3, DynamoDB, etc.), use **VPC endpoints** to keep traffic on the AWS network—no NAT charges.

Type	Services	Cost
Gateway	S3, DynamoDB	Free
Interface	Most other services	Per hour + data

```
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  route_table_ids = [aws_route_table.private.id]
}
```

1.4 10.4 Elastic IP (EIP)

A **static public IPv4** address. Required for NAT Gateway.

- **Charge** applies if EIP is allocated but not attached to a running instance
- Release unused EIPs to avoid fees

1.5 10.5 Troubleshooting connectivity

Problem	Check
Can't SSH to public instance	SG allows 22; has public IP; IGW attached; correct route
Private instance can't reach internet	NAT in public subnet; private RT has 0.0.0.0/0 → NAT
Can't reach S3 from private subnet	VPC endpoint or NAT route
Inter-subnet blocked	NACL rules; SG rules

1.5.1 Useful commands

```
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-xxx"
aws ec2 describe-network-interfaces --filters "Name=subnet-id,Values=subnet-xxx"
# From instance:
curl -s https://checkip.amazonaws.com # Public IP test
```

1.6 10.6 High availability networking

Production checklist:

- ☐ Multi-AZ subnets
- ☐ NAT Gateway per AZ (or accept risk)
- ☐ ALB spans multiple AZs
- ☐ VPC endpoints for S3/DynamoDB/ECR/API
- ☐ No single-AZ dependencies for critical paths

1.7 10.7 Chapter summary

- **IGW** enables public internet for public subnets.
 - **NAT Gateway** provides outbound-only internet for private subnets.
 - Use **VPC endpoints** to reduce NAT costs and improve security for AWS APIs.
-

1.8 Lab 10.1

1. Deploy NAT Gateway in public subnet with EIP.
 2. Configure private subnet route table.
 3. Launch EC2 in private subnet; `curl` an external API successfully.
 4. Add S3 gateway endpoint; verify S3 access without NAT.
-

Next: [Chapter 15 — EC2](#)

AWS Cloud Handbook — Complete Edition —

Chapter 11

1 Chapter 11: Security Groups vs Network ACLs

1.1 11.1 Two layers of VPC security

Amazon VPC provides **two complementary firewall mechanisms** for controlling traffic:

1. **Security Groups (SGs)** — Stateful, instance-level (or ENI-level) firewalls.
2. **Network Access Control Lists (NACLs)** — Stateless, subnet-level firewalls.

Understanding when and how to use each is essential for designing secure, compliant network architectures. Most day-to-day rules are security groups; NACLs add a subnet-wide safety net.

1.2 11.2 Security groups in depth

A **security group** acts as a virtual firewall for EC2 instances, RDS databases, Lambda functions in a VPC, ELBs, and other resources. It controls **inbound** and **outbound** traffic at the **elastic network interface (ENI)** level.

1.2.1 Key characteristics

Property	Security Group
State	Stateful — return traffic automatically allowed
Scope	ENI / resource level
Rules	Allow only (no Deny rules)
Evaluation	All rules evaluated before decision
Default	Deny all inbound; allow all outbound
Attachment	One or more SGs per ENI

1.2.2 Stateful behavior example

If you allow inbound TCP 443, the response traffic on ephemeral ports is **automatically permitted** without an explicit outbound rule. This simplifies rule management significantly.

1.2.3 Security group rules

Each rule specifies:

Field	Example
Type	HTTP, HTTPS, SSH, Custom TCP
Protocol	TCP, UDP, ICMP, all
Port range	443, 22, 1024-65535
Source (inbound)	CIDR, another SG, prefix list

Destination (outbound)	CIDR, another SG, prefix list
------------------------	-------------------------------

1.2.4 Referencing security groups

Instead of IP addresses, reference another security group:

```
Inbound: TCP 3306 from sg-0abc123 (app-tier SG)
```

This allows any instance in the app-tier SG to reach the database, regardless of IP address. When instances scale, rules remain valid.

1.2.5 CLI example

```
# Create a web tier security group
aws ec2 create-security-group \
  --group-name web-tier-sg \
  --description "Allow HTTP/HTTPS from ALB" \
  --vpc-id vpc-0abc123

# Allow HTTPS from ALB security group
aws ec2 authorize-security-group-ingress \
  --group-id sg-0web123 \
  --protocol tcp \
  --port 443 \
  --source-group sg-0alb456

# Allow SSH from bastion only
aws ec2 authorize-security-group-ingress \
  --group-id sg-0web123 \
  --protocol tcp \
  --port 22 \
  --source-group sg-0bastion789
```

1.2.6 Terraform example

```
resource "aws_security_group" "web" {
  name      = "web-tier"
  description = "Web server security group"
  vpc_id    = aws_vpc.main.id

  ingress {
    description = "HTTPS from ALB"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = { Name = "web-tier-sg" }
}
```

1.3 11.3 Network ACLs in depth

A **Network ACL (NACL)** is an optional layer of security that acts as a firewall for **subnets**. Each subnet is associated with exactly one NACL; a NACL can be associated with multiple subnets.

1.3.1 Key characteristics

--	--

Property	NACL
State	Stateless — must allow both directions explicitly
Scope	Subnet level
Rules	Allow and Deny
Evaluation	Lowest rule number first; first match wins
Default	Allow all inbound and outbound
Attachment	One NACL per subnet

1.3.2 Stateless behavior

If you allow inbound TCP 443, you must **also** allow outbound traffic on ephemeral ports (1024-65535) for the response to return. Forgetting the outbound rule is the most common NACL mistake.

1.3.3 Rule numbering

NACL rules are numbered (e.g., 100, 200, 300). Lower numbers are evaluated first. A ***** rule at the end denies everything not explicitly allowed (in custom NACLs).

1.3.4 Example: Restrict SSH to corporate IP

Rule #	Type	Protocol	Port	Source	Allow/Deny
100	Inbound	TCP	22	203.0.113.0/24	ALLOW
110	Inbound	TCP	22	0.0.0.0/0	DENY
100	Outbound	TCP	1024-65535	0.0.0.0/0	ALLOW

1.3.5 CLI example

```
aws ec2 create-network-acl --vpc-id vpc-0abc123

aws ec2 create-network-acl-entry \
  --network-acl-id acl-0abc123 \
  --rule-number 100 \
  --protocol tcp \
  --port-range From=22,To=22 \
  --cidr-block 203.0.113.0/24 \
  --rule-action allow \
  --ingress

aws ec2 replace-network-acl-association \
  --association-id aclassoc-0abc123 \
  --network-acl-id acl-0abc123
```

1.4 11.4 Security groups vs NACLs — comparison

Feature	Security Group	NACL
Level	Instance/ENI	Subnet
Stateful	Yes	No
Deny rules	No	Yes
Rule evaluation	All rules	Ordered (first match)
Default action	Deny inbound	Allow all
Affects	Resources with ENI	All traffic to/from subnet

Use case	Primary access control	Subnet-wide guardrails
----------	------------------------	------------------------

1.4.1 When to use which

Scenario	Use
Allow app → database on port 3306	Security group referencing app SG
Block a known malicious IP range	NACL Deny rule
Allow ALB → web tier on 443	Security group
Block all inbound except specific CIDRs at subnet level	NACL
Lambda in VPC accessing RDS	Security group on both

1.5 11.5 Defense in depth pattern

Production architectures typically use **both**:

Internet → NACL (subnet) → Security Group (ALB) → Security Group (web) → NACL (private subnet) → Security Group (DB)

1. **Public subnet NACL** — Deny known bad IPs, allow 80/443 from internet.
2. **ALB security group** — Allow 80/443 from 0.0.0.0/0.
3. **Web tier SG** — Allow 443 only from ALB SG.
4. **Private subnet NACL** — Deny all inbound from internet CIDRs.
5. **Database SG** — Allow 3306 only from web tier SG.

1.6 11.6 Common mistakes and troubleshooting

Mistake	Symptom	Fix
NACL missing outbound ephemeral rule	Connection hangs after SYN	Add outbound allow for 1024-65535
SG allows wrong source CIDR	Unexpected access or blocked access	Use SG references instead of CIDRs
Default SG used in production	Overly permissive (same-SG traffic allowed)	Create dedicated SGs per tier
NACL deny blocks health checks	ALB marks targets unhealthy	Allow health check IPs/CIDRs
Forgot IPv6 rules	IPv6 traffic blocked	Add ::/0 rules where needed

1.6.1 Connectivity troubleshooting order

1. Check **security group** inbound rules on the target.
2. Check **security group** outbound rules on the source.
3. Check **NACL** inbound on the target subnet.
4. Check **NACL** outbound on the source subnet.
5. Check **route tables** (covered in Chapter 10).
6. Use **VPC Reachability Analyzer** for automated path analysis.

```
aws ec2 describe-security-groups --group-ids sg-0abc123
```

```
aws ec2 describe-network-acls --filters "Name=association.subnet-id,Values=subnet-0abc123"
```

1.7 11.7 Security group limits and quotas

Resource	Default limit
Security groups per VPC	2,500
Rules per security group	60 inbound + 60 outbound
Security groups per ENI	5 (can request increase)
Referenced security groups per rule	1

Use **prefix lists** to reduce rule count when allowing access from many CIDRs.

1.8 11.8 Chapter summary

- **Security groups** are stateful, instance-level, allow-only firewalls—the primary access control mechanism.
- **NACLs** are stateless, subnet-level, support deny rules—a secondary guardrail.
- Reference security groups instead of IP addresses for scalable architectures.
- Always allow **return traffic** in NACLs (ephemeral ports outbound).
- Use both layers for defense in depth; troubleshoot connectivity systematically.

1.9 Lab 11.1 — Security group tiers

1. Create three security groups in your VPC: `alb-sg`, `web-sg`, `db-sg`.
2. Configure: ALB allows 80/443 from internet; web allows 443 from ALB SG; db allows 3306 from web SG.
3. Launch a test EC2 in a private subnet with `web-sg`.
4. Verify you cannot SSH directly from the internet.
5. Verify web tier can reach a test database port using `nc` or `telnet`.

1.10 Lab 11.2 — NACL deny rule

1. Create a custom NACL for a public subnet.
2. Add a Deny rule for a specific IP (your test IP).
3. Verify HTTP access is blocked from that IP but works from another.
4. Remove the deny rule and confirm access is restored.

1.11 Review questions

1. Why are security groups called “stateful” and NACLs “stateless”?
2. Can you create a Deny rule in a security group?
3. If a security group allows inbound TCP 80, is an outbound rule needed for the response?
4. What happens when NACL rule 100 allows traffic but rule 110 denies it?
5. Why should you reference security groups instead of IP addresses between tiers?

Next: [Chapter 12 — Load Balancers](#)

AWS Cloud Handbook — Complete Edition —

Chapter 12

1 Chapter 12: Load Balancers — ALB, NLB, and GWLB

1.1 12.1 Why load balancers?

In production, a single server cannot handle all traffic or survive failures. **Elastic Load Balancing (ELB)** distributes incoming traffic across multiple targets (EC2 instances, containers, IP addresses, Lambda functions) and performs health checks to route around unhealthy targets.

AWS offers three load balancer types, each optimized for different layers of the OSI model and use cases.

1.2 12.2 Load balancer comparison

Feature	ALB	NLB	GWLB
Layer	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP/TLS)	Layer 3 (IP)
Protocols	HTTP, HTTPS, gRPC, WebSocket	TCP, UDP, TLS	IP packets
Targets	Instances, IPs, Lambda	Instances, IPs, ALB	Appliance instances
Static IP	No (use Global Accelerator)	Yes (per AZ)	N/A
Cross-zone LB	Optional (default on)	Optional (default off)	N/A
Use case	Web apps, microservices, path routing	Extreme performance, static IP, TLS passthrough	Inline security appliances
Pricing	Per LCU-hour	Per NLCU-hour	Per GLCU-hour

1.3 12.3 Application Load Balancer (ALB)

The **ALB** operates at Layer 7, making routing decisions based on HTTP content: host header, path, query string, and headers.

1.3.1 Key features

- **Listener rules** — Route by path (`/api/*` → API target group), host (`api.example.com`), or headers.
- **Target groups** — Collections of targets with health checks and stickiness settings.
- **SSL/TLS termination** — Decrypt at the ALB using ACM certificates.
- **WebSocket and HTTP/2** — Native support.
- **AWS WAF integration** — Attach WAF Web ACLs for application-layer protection.

- **Authentication** — Built-in OIDC/Cognito authentication at the ALB.

1.3.2 ALB architecture

```

Internet → ALB (public subnets, multi-AZ)
├── Listener :443 (HTTPS)
│   ├── Rule: /api/* → Target Group A (ECS tasks)
│   ├── Rule: /admin/* → Target Group B (EC2 instances)
│   └── Default      → Target Group C (static site)
└── Listener :80 (redirect to 443)

```

1.3.3 Health checks

Setting	Typical value
Protocol	HTTP
Path	/health
Interval	30 seconds
Healthy threshold	2 consecutive successes
Unhealthy threshold	3 consecutive failures
Success codes	200

1.3.4 Terraform ALB example

```

resource "aws_lb" "app" {
  name            = "app-alb"
  internal        = false
  load_balancer_type = "application"
  security_groups = [aws_security_group.alb.id]
  subnets        = aws_subnet.public[*].id

  tags = { Name = "app-alb" }
}

resource "aws_lb_target_group" "web" {
  name     = "web-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id

  health_check {
    path            = "/health"
    healthy_threshold = 2
    unhealthy_threshold = 3
    timeout         = 5
    interval        = 30
    matcher         = "200"
  }
}

resource "aws_lb_listener" "https" {
  load_balancer_arn = aws_lb.app.arn
  port              = 443
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
  certificate_arn    = aws_acm_certificate.app.arn

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.web.arn
  }
}

resource "aws_lb_listener_rule" "api" {
  listener_arn = aws_lb_listener.https.arn
  priority     = 100

  action {

```

```

    type = "forward"
    target_group_arn = aws_lb_target_group.api.arn
  }

  condition {
    path_pattern { values = ["/api/*"] }
  }
}

```

1.3.5 CLI example

```

aws elbv2 create-load-balancer \
  --name app-alb \
  --subnets subnet-aaa subnet-bbb \
  --security-groups sg-alb \
  --scheme internet-facing \
  --type application

aws elbv2 create-target-group \
  --name web-tg \
  --protocol HTTP \
  --port 80 \
  --vpc-id vpc-0abc123 \
  --health-check-path /health

```

1.4 12.4 Network Load Balancer (NLB)

The **NLB** operates at Layer 4, handling millions of requests per second with ultra-low latency. It preserves the client source IP and supports static IP addresses per Availability Zone.

1.4.1 When to choose NLB

Scenario	Why NLB
Non-HTTP protocols (MQTT, gaming)	Layer 4, any TCP/UDP
Extreme throughput requirements	Millions of RPS
Static IP needed	Elastic IP per AZ
TLS passthrough	NLB forwards encrypted traffic to targets
PrivateLink service provider	NLB as endpoint service
ALB as target	NLB → ALB for static IP + L7 routing

1.4.2 NLB characteristics

- **Connection-based** — Not request-based like ALB.
- **Cross-zone load balancing** — Off by default (enable for even distribution).
- **Preserve source IP** — Targets see the real client IP (unlike ALB which adds `X-Forwarded-For`).
- **Target types** — Instance, IP, ALB.

1.4.3 Health checks for NLB

NLB supports TCP, HTTP, and HTTPS health checks. TCP checks only verify port openness; HTTP/HTTPS checks validate response codes.

1.5 12.5 Gateway Load Balancer (GWLB)

The **GWLB** distributes traffic to **third-party virtual appliances** (firewalls, IDS/IPS, deep packet inspection) in a

transparent, bump-in-the-wire fashion.

1.5.1 Use case

VPC traffic → GWLB Endpoint → GWLB → Firewall appliance → GWLB → destination

- Deploy security appliances as GWLB targets.
- Traffic is encapsulated using **GENEVE** protocol (port 6081).
- Appliances inspect and forward traffic transparently.
- Common with vendors: Palo Alto, Fortinet, Check Point.

1.5.2 GWLB components

Component	Role
GWLB	Distributes flows to appliance fleet
Target group	Appliance instances (IP or instance type)
GWLB endpoint	VPC endpoint that routes traffic through GWLB
Route table	Directs subnet traffic to the GWLB endpoint

1.6 12.6 Shared ELB concepts

1.6.1 Listeners and rules

- **Listener** — Checks for connection requests on a port/protocol.
- **Rules** — ALB only; evaluated by priority (lower number first).
- **Default action** — Catches traffic not matched by any rule.

1.6.2 Target groups

Setting	Description
Target type	instance, ip, lambda, alb
Protocol/port	How the LB communicates with targets
Deregistration delay	Drain time before removing unhealthy targets (default 300s)
Stickiness	Session affinity via cookie (ALB) or source IP (NLB)

1.6.3 Stickiness

LB	Method
ALB	Application cookie (LB cookie or app cookie)
NLB	Source IP hash (5-tuple for TCP, 3-tuple for UDP)

1.6.4 Connection draining

When a target is deregistered or becomes unhealthy, the LB stops sending **new** connections but allows **in-flight** connections to complete during the deregistration delay.

1.7 12.7 SSL/TLS with load balancers

Approach	LB	Description
Terminate at LB	ALB, NLB	LB decrypts; backend can use HTTP
TLS passthrough	NLB	Encrypted traffic forwarded to targets
Mutual TLS (mTLS)	ALB	Client certificate validation
Certificate	ACM (free)	Attach to listener

Best practice: Terminate TLS at the ALB with ACM certificates. Use HTTPS between ALB and targets for end-to-end encryption if required.

1.8 12.8 Monitoring and troubleshooting

Tool	Purpose
CloudWatch metrics	RequestCount, TargetResponseTime, HealthyHostCount
Access logs	S3-stored per-request logs (enable explicitly)
Connection logs	NLB connection-level logs
Target health	<code>aws elbv2 describe-target-health</code>

```
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/web-tg/abc123

aws elbv2 describe-load-balancer-attributes \
  --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/app-alb/abc123
```

1.8.1 Common issues

Symptom	Cause
502 Bad Gateway	Target not responding or security group blocks ALB
503 Service Unavailable	No healthy targets in target group
Health check failing	Wrong path, port, or security group
Slow responses	Target overloaded; check TargetResponseTime metric

1.9 12.9 Chapter summary

- **ALB** — Layer 7; path/host routing, SSL termination, WAF, authentication.
- **NLB** — Layer 4; extreme performance, static IPs, non-HTTP protocols.
- **GWLB** — Inline security appliances with transparent traffic inspection.
- Configure **health checks**, **security groups**, and **target groups** carefully.
- Use **ACM** for certificates; enable **access logs** for troubleshooting.

1.10 Lab 12.1 — Deploy an ALB

1. Create an ALB in public subnets with an HTTPS listener (ACM certificate).
2. Create a target group pointing to EC2 instances in private subnets.
3. Configure a health check on `/health`.
4. Add a listener rule: `/api/*` forwards to a separate target group.

5. Verify routing with `curl` against both paths.

1.11 Lab 12.2 — NLB with static IP

1. Create an NLB with TCP listener on port 5432.
 2. Note the static Elastic IP assigned per AZ.
 3. Register RDS or an EC2 PostgreSQL instance as target.
 4. Connect from a client using the NLB static IP.
-

1.12 Review questions

1. At which OSI layer does an ALB operate, and what routing decisions can it make?
 2. When would you choose NLB over ALB?
 3. What is the purpose of a GWLB?
 4. Why might ALB health checks fail even when the application is running?
 5. What is connection draining, and why does it matter during deployments?
-

Next: [Chapter 13 — Route 53 DNS Routing](#)

AWS Cloud Handbook — Complete Edition —

Chapter 13

1 Chapter 13: Route 53 — DNS Records and Routing Policies

1.1 13.1 What is Amazon Route 53?

Amazon Route 53 is AWS’s highly available and scalable **Domain Name System (DNS)** web service. It translates human-readable domain names (e.g., `www.example.com`) into IP addresses and provides advanced routing policies for traffic management, failover, and latency optimization.

Route 53 also offers **domain registration**, **health checks**, and **traffic flow** visual policies.

1.2 13.2 Core DNS concepts

Term	Definition
Hosted zone	Container for DNS records for a domain
Record	Maps a name to a value (A, CNAME, MX, etc.)
TTL	Time-to-live; how long resolvers cache the answer
Name server (NS)	Authoritative DNS servers for the zone
Resolver	Client-side DNS lookup (Route 53 Resolver in VPC)

1.2.1 Hosted zone types

Type	Cost	Use case
Public hosted zone	\$0.50/month + queries	Internet-facing domains
Private hosted zone	\$0.50/month + queries	Internal VPC DNS (associated with VPCs)

1.3 13.3 DNS record types

Type	Purpose	Example value
A	IPv4 address	<code>203.0.113.10</code>
AAAA	IPv6 address	<code>2001:db8::1</code>
CNAME	Alias to another name	<code>app.example.com → d123.cloudfront.net</code>
MX	Mail server	<code>10 mail.example.com</code>
TXT	Text (SPF, DKIM, verification)	<code>"v=spf1 include:..."</code>
NS	Name servers	<code>ns-123.awsdns-45.com</code>

SOA	Start of authority	Zone metadata
SRV	Service location	_http._tcp.example.com
Alias	Route 53-specific; maps to AWS resource	ALB, CloudFront, S3, API Gateway

1.3.1 Alias records (Route 53 specific)

Alias records are Route 53's extension of CNAME with advantages:

- Can be used at the **zone apex** (example.com — CNAME cannot).
- No charge for alias queries to AWS resources.
- Automatically tracks IP changes of the target resource.

Supported alias targets: ALB, CloudFront, S3 website, API Gateway, Elastic Beanstalk, VPC interface endpoints, another Route 53 record.

1.4 13.4 Routing policies

Route 53 supports multiple routing policies beyond simple DNS:

1.4.1 Simple routing

One record with one or more values. Route 53 returns **all** values in random order (client picks one).

```
aws route53 change-resource-record-sets \
  --hosted-zone-id Z1234567890ABC \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "www.example.com",
        "Type": "A",
        "TTL": 300,
        "ResourceRecords": [{"Value": "203.0.113.10"}]
      }
    }]
  }'
```

1.4.2 Weighted routing

Distribute traffic by **weight** (percentage). Useful for A/B testing and blue/green deployments.

Record	Weight	Traffic share
blue.example.com → 1.2.3.4	90	90%
green.example.com → 5.6.7.8	10	10%

```
resource "aws_route53_record" "blue" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.example.com"
  type    = "A"
  ttl     = 60
  records = ["203.0.113.10"]

  set_identifier = "blue"
  weighted_routing_policy {
    weight = 90
  }
}

resource "aws_route53_record" "green" {
  zone_id = aws_route53_zone.main.zone_id
  name    = "app.example.com"
```

```

type    = "A"
ttl     = 60
records = ["203.0.113.20"]

set_identifier = "green"
weighted_routing_policy {
  weight = 10
}
}

```

1.4.3 Latency-based routing

Route traffic to the region with the **lowest latency** for the user. Requires one record per region with a `region` identifier.

Region	Record	Latency
us-east-1	203.0.113.10	~20ms (US East users)
eu-west-1	198.51.100.10	~15ms (EU users)

1.4.4 Failover routing

Active-passive failover. Pair a primary record with a secondary; Route 53 health checks determine which is active.

```

Primary (active) → ALB in us-east-1 (health check: /health)
Secondary (passive) → S3 static failover page (no health check needed)

```

1.4.5 Geolocation routing

Route based on the user's **geographic location** (continent, country, or US state). Useful for content localization and compliance (e.g., EU data stays in EU).

1.4.6 Geoproximity routing

Route based on **geographic location and bias**. Unlike geolocation, you can shift traffic by applying a **bias** value to expand or shrink a region's coverage area. Requires Route 53 Traffic Flow.

1.4.7 Multi-value answer routing

Return **multiple healthy** IP addresses. Similar to simple routing but only returns values that pass health checks. Up to 8 healthy records.

1.5 Health checks

Route 53 health checks monitor endpoint health and integrate with routing policies (failover, weighted, multi-value).

Type	Monitors
Endpoint	HTTP, HTTPS, or TCP to an IP/hostname
CloudWatch alarm	Any CloudWatch metric (e.g., DynamoDB throttling)
Calculated	Combination of child health checks (AND/OR)

1.5.1 Health check settings

Setting	Typical value
Protocol	HTTPS

Path	/health
Interval	30 seconds (10 seconds for fast)
Failure threshold	3
Regions	Minimum 3 for reliable failover

```
aws route53 create-health-check \
  --health-check-config '{
    "IPAddress": "203.0.113.10",
    "Port": 443,
    "Type": "HTTPS",
    "ResourcePath": "/health",
    "FullyQualifiedDomainName": "app.example.com",
    "RequestInterval": 30,
    "FailureThreshold": 3
  }'
```

1.5.2 Health check billing note

Health checks cost \$0.50/month each (first 50 AWS-endpoint checks free). Fast interval checks cost more.

1.6 13.6 Private DNS and hybrid resolution

1.6.1 Private hosted zones

Associate a hosted zone with one or more VPCs. Internal names (e.g., `db.internal.example.com`) resolve only within those VPCs.

1.6.2 Route 53 Resolver

For hybrid cloud (on-premises + AWS):

Feature	Purpose
Inbound endpoints	On-premises resolves AWS private DNS
Outbound endpoints	AWS VPC resolves on-premises DNS
Resolver rules	Forward specific domains to on-premises

On-premises DNS → Inbound Resolver Endpoint → Private hosted zone (`db.internal`)
 VPC instances → Outbound Resolver Endpoint → On-premises DNS (`corp.example.com`)

1.7 13.7 Route 53 with other AWS services

Integration	Pattern
ALB/NLB	Alias A record → load balancer DNS name
CloudFront	Alias A/AAAA → CloudFront distribution
S3 static site	Alias A → S3 website endpoint
API Gateway	Alias A → API Gateway custom domain
Global Accelerator	Alias A → accelerator DNS name
ACM	DNS validation via CNAME records

1.8 13.8 DNS security

Feature	Purpose
DNSSEC	Cryptographic signing of DNS responses
Route 53 Resolver DNS Firewall	Block malicious domains
Query logging	Log all DNS queries to CloudWatch Logs

1.9 13.9 Chapter summary

- **Route 53** provides DNS hosting, domain registration, and advanced routing policies.
 - **Alias records** map to AWS resources without CNAME limitations at the zone apex.
 - **Routing policies** — simple, weighted, latency, failover, geolocation, geoproximity, multi-value.
 - **Health checks** enable automatic failover and traffic shifting.
 - **Private hosted zones** and **Resolver** support hybrid DNS architectures.
-

1.10 Lab 13.1 — Public hosted zone

1. Create a public hosted zone for a domain you control.
2. Update your registrar's NS records to point to Route 53 name servers.
3. Create an A record (alias) pointing to an ALB or CloudFront distribution.
4. Verify resolution: `dig www.yourdomain.com`.

1.11 Lab 13.2 — Weighted routing

1. Create two EC2 instances in different AZs with simple web pages ("Server A" / "Server B").
2. Create weighted A records (50/50) for the same hostname.
3. Run `dig` or `curl` multiple times and observe distribution.
4. Shift weight to 90/10 and verify the change.

1.12 Lab 13.3 — Failover routing

1. Set up a primary record with a health check on a healthy endpoint.
 2. Create a secondary record pointing to an S3 static failover page.
 3. Stop the primary endpoint and verify DNS failover within 1–2 minutes.
-

1.13 Review questions

1. What is the advantage of an alias record over a CNAME at the zone apex?
 2. How does weighted routing differ from latency-based routing?
 3. What happens when a failover primary record's health check fails?
 4. How do private hosted zones differ from public hosted zones?
 5. Why should health checks use at least three AWS regions?
-

Next: [Chapter 14 — VPC Peering, Transit Gateway, PrivateLink](#)

AWS Cloud Handbook — Complete Edition —

Chapter 14

1 Chapter 14: VPC Peering, Transit Gateway, and PrivateLink

1.1 14.1 Connecting VPCs and services

As organizations grow, workloads spread across multiple VPCs, accounts, and regions. AWS provides several connectivity options, each with different trade-offs for complexity, scalability, and cost.

Option	Scope	Transitive	Use case
VPC Peering	Same or cross-region	No	Simple 1:1 VPC connections
Transit Gateway	Hub-and-spoke	Yes (via TGW)	Many VPCs, on-premises, multi-account
PrivateLink	Service consumer → provider	N/A	Private access to SaaS/services
VPC Endpoints	VPC → AWS service	N/A	Private access to AWS APIs (S3, DynamoDB)

1.2 14.2 VPC Peering

A **VPC peering connection** is a networking connection between two VPCs that enables routing using private IP addresses.

1.2.1 Peering rules

- Peering is **non-transitive**: if A↔B and B↔C, A cannot reach C through B.
- **No overlapping CIDR blocks** between peered VPCs.
- Peering works **same-region** and **cross-region**.
- **No single point of failure** — AWS backbone handles the connection.
- You control access via **route tables** and **security groups** (cross-VPC SG referencing supported in same region).

1.2.2 Peering setup steps

1. Create peering connection (requester → acceptor).
2. Accept the peering connection (same or different account).
3. Update **route tables** in both VPCs to route peer CIDR through the peering connection.
4. Update **security groups** to allow traffic from peer VPC CIDR or SG.

VPC A (10.0.0.0/16) ← Peering → VPC B (10.1.0.0/16)

Route table A: 10.1.0.0/16 → pcx-0abc123

Route table B: 10.0.0.0/16 → pcx-0abc123

1.2.3 CLI example

```
# Create peering connection
aws ec2 create-vpc-peering-connection \
  --vpc-id vpc-0aaa \
  --peer-vpc-id vpc-0bbb \
  --peer-region us-west-2 # omit for same region

# Accept (in acceptor account/region)
aws ec2 accept-vpc-peering-connection \
  --vpc-peering-connection-id pcx-0abc123

# Add routes
aws ec2 create-route \
  --route-table-id rtb-0aaa \
  --destination-cidr-block 10.1.0.0/16 \
  --vpc-peering-connection-id pcx-0abc123
```

1.2.4 Terraform example

```
resource "aws_vpc_peering_connection" "peer" {
  vpc_id      = aws_vpc.a.id
  peer_vpc_id = aws_vpc.b.id
  peer_region = "us-west-2"
  auto_accept = false

  tags = { Name = "a-to-b-peering" }
}

resource "aws_route" "a_to_b" {
  route_table_id      = aws_route_table.a.id
  destination_cidr_block = aws_vpc.b.cidr_block
  vpc_peering_connection_id = aws_vpc_peering_connection.peer.id
}
```

1.2.5 When peering breaks down

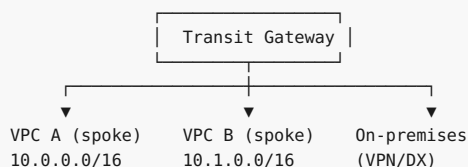
VPCs	Peering connections needed
2	1
5	10 ($n \times (n-1) / 2$)
10	45
50	1,225

Peering does not scale beyond a handful of VPCs. Use **Transit Gateway** instead.

1.3 14.3 AWS Transit Gateway

Transit Gateway (TGW) is a regional network hub that connects VPCs, VPN connections, and Direct Connect gateways in a **hub-and-spoke** model.

1.3.1 Architecture



1.3.2 Key concepts

Component	Description
Transit Gateway	Regional hub
Attachment	VPC, VPN, DX, peering, Connect
Route table	Controls which attachments can reach which
Association	Links attachment to a route table
Propagation	Auto-adds routes from attachment to route table

1.3.3 Route table segmentation

Use multiple TGW route tables for isolation:

Route table	Attachments	Routes to
Production	Prod VPCs, prod VPN	Other prod VPCs, shared services
Development	Dev VPCs	Dev VPCs only (not prod)
Shared Services	Shared VPC	All environments (one-way)

1.3.4 Cross-region peering

Connect TGWs in different regions with **inter-region peering** for global connectivity.

1.3.5 Terraform TGW example

```
resource "aws_ec2_transit_gateway" "main" {
  description           = "Organization TGW"
  default_route_table_association = "disable"
  default_route_table_propagation = "disable"
  auto_accept_shared_attachments = "enable"

  tags = { Name = "org-tgw" }
}

resource "aws_ec2_transit_gateway_vpc_attachment" "app" {
  subnet_ids      = aws_subnet.private[*].id
  transit_gateway_id = aws_ec2_transit_gateway.main.id
  vpc_id          = aws_vpc.app.id

  tags = { Name = "app-vpc-attachment" }
}

resource "aws_ec2_transit_gateway_route_table" "prod" {
  transit_gateway_id = aws_ec2_transit_gateway.main.id
  tags = { Name = "prod-rt" }
}

resource "aws_ec2_transit_gateway_route_table_association" "app" {
  transit_gateway_attachment_id = aws_ec2_transit_gateway_vpc_attachment.app.id
  transit_gateway_route_table_id = aws_ec2_transit_gateway_route_table.prod.id
}
```

1.3.6 TGW pricing

- **Attachment hourly charge** per VPC/VPN/DX attachment.
- **Data processing charge** per GB transited.
- For many VPCs, TGW is more cost-effective and manageable than full-mesh peering.

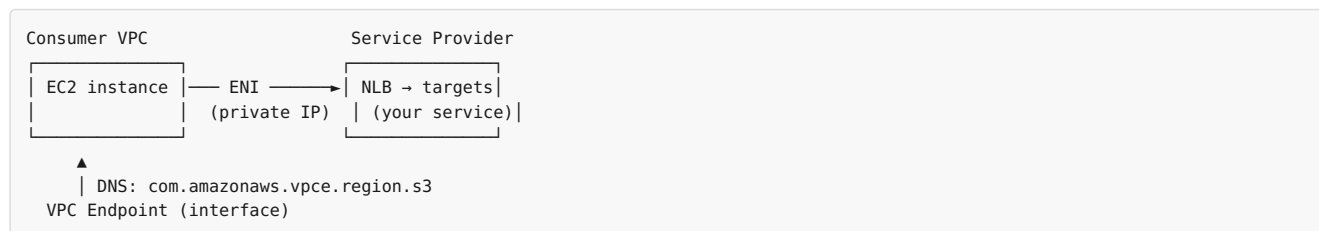
1.4 14.4 AWS PrivateLink

PrivateLink enables private connectivity from your VPC to services (AWS, partner, or your own) without traversing the public internet.

1.4.1 Two flavors

Type	Direction	Example
Interface VPC Endpoint	Your VPC → AWS service or partner service	S3 (interface), third-party SaaS
Endpoint Service (Provider)	Consumer VPC → your service	Expose internal API to customers

1.4.2 Interface endpoint architecture



1.4.3 Key points

- Creates **Elastic Network Interfaces (ENIs)** in your subnets.
- Uses **private IP addresses** from your subnet CIDR.
- Security groups apply to the endpoint ENIs.
- **Endpoint policies** control which principals can use the endpoint.

1.4.4 Gateway vs Interface endpoints

Feature	Gateway Endpoint	Interface Endpoint
Services	S3, DynamoDB	Most other AWS services + partner services
Cost	Free	Hourly + data charges
Access control	Endpoint policy + bucket policy	SG + endpoint policy
DNS	Route table entry	Private DNS (optional)
IP	Prefix list in route table	ENI with private IP

```
# Create interface endpoint for a service
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc123 \
  --service-name com.amazonaws.us-east-1.execute-api \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-0aaa subnet-0bbb \
  --security-group-ids sg-0endpoint

# Create gateway endpoint for S3
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0abc123 \
  --service-name com.amazonaws.us-east-1.s3 \
  --route-table-ids rtb-0aaa rtb-0bbb
```

1.5 14.5 Choosing the right connectivity

Requirement	Solution

Two VPCs, same region, simple	VPC Peering
10+ VPCs, hub-and-spoke	Transit Gateway
Cross-account service consumption	PrivateLink (endpoint service)
Private S3/DynamoDB access	Gateway VPC Endpoint
Private access to AWS APIs (EC2, SSM, etc.)	Interface VPC Endpoint
On-premises to AWS	VPN or Direct Connect → TGW
SaaS provider private access	PrivateLink (consumer)

1.6 14.6 Security considerations

Control	Applies to
Route tables	Peering, TGW — control which CIDRs are reachable
Security groups	Peering (same region SG ref), endpoints, TGW attachments
NACLs	All traffic paths
Endpoint policies	VPC endpoints — IAM-style access control
TGW route tables	Segment environments (prod cannot reach dev)
Resource policies	S3 bucket policies for gateway endpoints

1.7 14.7 Chapter summary

- **VPC Peering** connects two VPCs directly; non-transitive; does not scale beyond a few VPCs.
- **Transit Gateway** provides hub-and-spoke connectivity for many VPCs, VPN, and Direct Connect.
- **PrivateLink** enables private access to services without public internet exposure.
- **Gateway endpoints** (S3, DynamoDB) are free; **interface endpoints** create ENIs with hourly charges.
- Choose connectivity based on scale, transitivity needs, and security segmentation requirements.

1.8 Lab 14.1 — VPC Peering

1. Create two VPCs with non-overlapping CIDRs (10.0.0.0/16 and 10.1.0.0/16).
2. Establish a peering connection and accept it.
3. Update route tables in both VPCs.
4. Launch an instance in each VPC and verify ping/SSH across the peering connection.

1.9 Lab 14.2 — S3 Gateway Endpoint

1. Create a gateway VPC endpoint for S3 in your VPC.
2. Attach it to private subnet route tables.
3. From a private instance (no NAT), run `aws s3 ls` and verify it works without internet access.

1.10 Lab 14.3 — Transit Gateway (advanced)

1. Create a Transit Gateway with two VPC attachments.

2. Create a TGW route table and associate both attachments.
 3. Verify instances in both VPCs can communicate through the TGW.
-

1.11 Review questions

1. Why is VPC peering non-transitive, and what problem does Transit Gateway solve?
 2. Can two peered VPCs have overlapping CIDR blocks?
 3. What is the difference between a gateway endpoint and an interface endpoint?
 4. How does Transit Gateway route table segmentation improve security?
 5. When is VPC peering preferred over Transit Gateway?
-

Next: [*Chapter 15 — EC2 Fundamentals*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 15

1 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates

1.1 15.1 Amazon EC2 overview

Elastic Compute Cloud (EC2) provides resizable virtual machines in the cloud. You choose:

- **Instance type** — CPU, memory, network, storage
 - **AMI** — Operating system image
 - **Networking** — VPC, subnet, security groups
 - **Storage** — EBS volumes
 - **IAM role** — Permissions without access keys
-

1.2 15.2 Instance families

Family	Use case	Examples
T (burstable)	Dev, low-traffic web	t3.micro, t3.small
M (general)	Balanced workloads	m6i.large
C (compute)	CPU-intensive	c6i.xlarge
R (memory)	Databases, caches	r6i.large
G/P (GPU)	ML, graphics	g5.xlarge
I (storage)	High IOPS databases	i3.large

Naming: `m6i.large` = family **m**, gen **6**, Intel **i**, size **large**.

1.2.1 Purchasing options

Option	Cost	Interruption
On-Demand	Highest	Never
Reserved / Savings Plans	30–70% less	Never
Spot	Up to 90% less	Can be terminated with 2-min notice
Dedicated Host	Compliance	Never

1.3 15.3 AMIs and launch templates

AMI (Amazon Machine Image) — Template for root volume: OS + optional software.

Sources:

- AWS-provided (Amazon Linux 2023, Ubuntu)
- Marketplace
- Custom (built with EC2 Image Builder or Packer)

Launch Template — Reusable EC2 configuration (instance type, AMI, SG, user data, IAM role). Preferred over launch configurations.

```
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafelf0 \
  --instance-type t3.micro \
  --key-name my-key \
  --security-group-ids sg-xxx \
  --subnet-id subnet-xxx \
  --iam-instance-profile Name=EC2-S3-ReadRole \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-01}]'
```

1.4 15.4 User data — bootstrap scripts

Run commands on first boot:

```
#!/bin/bash
yum update -y
yum install -y nginx
systemctl enable nginx
systemctl start nginx
echo "<h1>Hello from ${hostname}</h1>" > /usr/share/nginx/html/index.html
```

Pass via console, CLI `--user-data`, or launch template. Logs: `/var/log/cloud-init-output.log`.

1.5 15.5 Auto Scaling Groups (ASG)

ASG maintains desired instance count and replaces unhealthy instances.

```
resource "aws_autoscaling_group" "web" {
  name                = "web-asg"
  vpc_zone_identifier = [aws_subnet.public_a.id, aws_subnet.public_b.id]
  target_group_arns   = [aws_lb_target_group.web.arn]
  health_check_type   = "ELB"
  min_size            = 2
  max_size            = 10
  desired_capacity    = 2

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
}
```

Pair with **ALB** for traffic distribution and **scaling policies** on CPU or request count.

1.6 15.6 EC2 best practices

- Use **IAM roles**, never embed access keys
- **Session Manager** instead of SSH where possible (no open port 22)
- **IMDSv2** required (`HttpTokens: required`) — prevents SSRF credential theft
- **Encrypted EBS** volumes by default
- **Patch** via SSM Patch Manager or golden AMI pipelines
- **Right-size** with CloudWatch metrics and Compute Optimizer

1.7 15.7 Chapter summary

- Choose instance **family and size** based on workload profile.
 - Use **launch templates** and **user data** for repeatable deployments.
 - **ASG + ALB** provides scalable, self-healing web tiers.
-

1.8 Lab 15.1

1. Launch Amazon Linux 2023 t3.micro with nginx user data.
 2. Attach IAM role for S3 read.
 3. Create launch template and ASG (min 1, max 2) behind an ALB.
-

Next: [Chapter 20 — S3](#)

AWS Cloud Handbook — Complete Edition —

Chapter 16

1 Chapter 16: Auto Scaling & Elastic Load Balancing Integration

1.1 16.1 Why Auto Scaling?

Manual capacity management cannot keep pace with variable demand. **Amazon EC2 Auto Scaling** automatically adjusts the number of EC2 instances (or other resources) based on conditions you define. Combined with **Elastic Load Balancing**, you get a self-healing, elastic application tier that scales out during traffic spikes and scales in during quiet periods—optimizing both performance and cost.

1.2 16.2 Auto Scaling components

Component	Description
Launch template / configuration	AMI, instance type, security groups, user data
Auto Scaling group (ASG)	Collection of EC2 instances with min, max, desired capacity
Scaling policies	Rules that trigger scale-out or scale-in
Health checks	EC2 status checks and/or ELB health checks
Lifecycle hooks	Custom actions during instance launch/terminate

1.2.1 ASG capacity settings

Setting	Meaning
Minimum	Floor; ASG never goes below this
Desired	Target number of instances
Maximum	Ceiling; ASG never exceeds this

1.3 16.3 Launch templates vs launch configurations

Launch templates are the modern approach (launch configurations are legacy):

Feature	Launch Template	Launch Configuration
Multiple versions	Yes	No
Mix instance types (Spot + On-Demand)	Yes	No
Latest features	Yes	Deprecated
Modify without replacement	Yes	No

1.3.1 Launch template example (Terraform)

```
resource "aws_launch_template" "web" {
  name_prefix   = "web-"
  image_id      = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"

  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = base64encode(<<-EOF
    #!/bin/bash
    yum install -y httpd
    systemctl start httpd
    echo "<h1>$(hostname)</h1>" > /var/www/html/index.html
  EOF
)

  tag_specifications {
    resource_type = "instance"
    tags = { Name = "web-asg-instance" }
  }
}
```

1.4 16.4 Auto Scaling group with ALB

```
Internet → ALB → Target Group ← ASG registers instances
                        ↑
                Health checks (HTTP /health)
```

1.4.1 Terraform ASG + ALB integration

```
resource "aws_autoscaling_group" "web" {
  name                = "web-asg"
  vpc_zone_identifier = aws_subnet.private[*].id
  target_group_arns   = [aws_lb_target_group.web.arn]
  health_check_type   = "ELB"
  health_check_grace_period = 300

  min_size      = 2
  max_size      = 10
  desired_capacity = 2

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }

  tag {
    key      = "Name"
    value    = "web-asg"
    propagate_at_launch = true
  }
}
```

Critical settings:

- `health_check_type = "ELB"` — ASG replaces instances that fail ALB health checks, not just EC2 status checks.
- `health_check_grace_period` — Time (seconds) before health checks count after launch. Allow your app to start.

1.5 16.5 Scaling policies

1.5.1 Target tracking (recommended)

Maintain a metric at a target value. AWS automatically creates scale-out and scale-in policies.

Metric	Use case
ASGAverageCPUUtilization	General compute workloads
ALBRequestCountPerTarget	Request-driven scaling
Custom CloudWatch metric	Application-specific (queue depth, latency)

```
resource "aws_autoscaling_policy" "cpu_target" {
  name           = "cpu-target-tracking"
  autoscaling_group_name = aws_autoscaling_group.web.name
  policy_type     = "TargetTrackingScaling"

  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
    target_value = 60.0
  }
}
```

1.5.2 Step scaling

Scale by steps based on alarm breach magnitude:

CPU utilization	Action
50-70%	Add 1 instance
70-90%	Add 2 instances
> 90%	Add 4 instances

1.5.3 Simple scaling

Single adjustment per alarm. Less flexible; prefer target tracking or step scaling.

1.5.4 Scheduled scaling

Pre-define capacity for known patterns:

```
aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name web-asg \
  --scheduled-action-name scale-up-morning \
  --recurrence "0 8 * * MON-FRI" \
  --desired-capacity 10 \
  --time-zone "America/New_York"
```

1.5.5 Predictive scaling

Machine learning predicts traffic patterns and proactively scales. Works alongside dynamic policies.

1.6 16.6 Instance refresh

Instance refresh replaces instances in an ASG with new ones (e.g., after AMI update) without manual intervention.

Setting	Description

Min healthy percentage	Minimum healthy instances during refresh (e.g., 90%)
Instance warmup	Time before new instances count toward capacity
Checkpoint	Pause refresh at percentage milestones

```
aws autoscaling start-instance-refresh \
  --auto-scaling-group-name web-asg \
  --preferences '{
    "MinHealthyPercentage": 90,
    "InstanceWarmup": 300,
    "CheckpointPercentages": [50, 100]
  }'
```

1.7 16.7 Lifecycle hooks

Lifecycle hooks pause instance launch or termination for custom actions:

Hook	Use case
Launching	Install software, pull config, register with service mesh
Terminating	Drain connections, deregister from service discovery

Instance launching → Hook (wait) → Custom action (Lambda/SSM) → Continue → InService
 Instance terminating → Hook (wait) → Drain logs, deregister → Continue → Terminated

Default heartbeat timeout: 3600 seconds (1 hour). Extend or complete via `complete-lifecycle-action`.

1.8 16.8 Mixed instances and Spot

ASGs support **mixed instance policies** combining On-Demand and Spot:

```
resource "aws_autoscaling_group" "mixed" {
  mixed_instances_policy {
    launch_template {
      launch_template_specification {
        launch_template_id = aws_launch_template.web.id
        version            = "$Latest"
      }
      override { instance_type = "t3.micro" }
      override { instance_type = "t3.small" }
    }
    instances_distribution {
      on_demand_base_capacity          = 2
      on_demand_percentage_above_base_capacity = 25
      spot_allocation_strategy        = "capacity-optimized"
    }
  }
}
```

This keeps 2 On-Demand instances as baseline, with 25% On-Demand above that and the rest Spot.

1.9 16.9 Warm pools

Warm pools pre-initialize instances in `Stopped` or `Running:Wait` state for faster scale-out:

- Instances in `Stopped` state incur EBS charges but no compute charges.

- Instances in `Running:Wait` incur compute charges but scale out in seconds.

1.10 16.10 Monitoring and troubleshooting

Metric	Alarm threshold idea
<code>GroupDesiredCapacity</code>	Track scaling events
<code>GroupInServiceInstances</code>	Should match desired
<code>GroupTerminatingInstances</code>	Spike during scale-in
<code>WarmPoolDesiredCapacity</code>	Warm pool sizing

```
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names web-asg

aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name web-asg \
  --max-records 10
```

1.10.1 Common issues

Symptom	Cause
Instances launch then terminate	Failing health checks; check grace period
ASG won't scale out	Max capacity reached; scaling policy cooldown
Uneven AZ distribution	Check <code>AZRebalance</code> and subnet configuration
Spot interruptions	Use capacity-optimized allocation; diversify instance types

1.11 16.11 Chapter summary

- **Auto Scaling groups** maintain desired capacity with min/max bounds.
- Integrate with **ALB** using `health_check_type = "ELB"` and target group registration.
- **Target tracking** scaling policies are the simplest and most effective for most workloads.
- **Instance refresh** enables rolling AMI updates without downtime.
- **Mixed instances** and **warm pools** optimize cost and scale-out speed.

1.12 Lab 16.1 — ASG with ALB

1. Create a launch template with a simple web server and `/health` endpoint.
2. Create an ASG (min 2, max 6) in private subnets attached to an ALB target group.
3. Add a target tracking policy on `ASGAverageCPUUtilization` at 50%.
4. Generate load with `ab` or `hey` and watch instances scale out.
5. Stop load and observe scale-in after cooldown.

1.13 Lab 16.2 — Instance refresh

1. Update the launch template AMI or user data.
2. Start an instance refresh with 50% checkpoint.

3. Monitor `describe-scaling-activities` until complete.
 4. Verify all instances serve the updated content.
-

1.14 Review questions

1. What is the difference between `health_check_type = "EC2"` and `"ELB"`?
 2. Why is a health check grace period important?
 3. How does target tracking scaling differ from step scaling?
 4. What is the purpose of lifecycle hooks?
 5. How do warm pools improve scale-out speed?
-

Next: [Chapter 17 — Lambda](#)

AWS Cloud Handbook — Complete Edition —

Chapter 17

1 Chapter 17: Lambda — Events, Concurrency, and Layers

1.1 17.1 What is AWS Lambda?

AWS Lambda is a serverless compute service that runs code in response to events without provisioning or managing servers. You upload your function code, configure triggers, and AWS handles scaling, patching, and availability. You pay only for compute time consumed (per millisecond) and number of requests.

Lambda is ideal for event-driven workloads: API backends, data processing, scheduled tasks, and microservices glue.

1.2 17.2 Lambda fundamentals

Concept	Description
Function	Your code + runtime + configuration
Handler	Entry point (e.g., <code>index.handler</code>)
Runtime	Language environment (Python 3.12, Node.js 20, etc.)
Trigger	Event source that invokes the function
Execution role	IAM role granting AWS service permissions
Deployment package	.zip or container image (up to 10 GB for containers)

1.2.1 Limits (selected)

Resource	Limit
Memory	128 MB – 10,240 MB (CPU scales with memory)
Timeout	900 seconds (15 minutes)
Deployment package (zip)	50 MB (direct upload), 250 MB (S3)
Environment variables	4 KB total
/tmp storage	512 MB – 10,240 MB
Concurrent executions (default)	1,000 per region (can increase)

1.3 17.3 Creating a Lambda function

1.3.1 AWS CLI

```
# Create execution role
aws iam create-role \
  --role-name lambda-basic-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "lambda.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'

aws iam attach-role-policy \
  --role-name lambda-basic-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

# Create function
aws lambda create-function \
  --function-name hello-world \
  --runtime python3.12 \
  --handler index.handler \
  --role arn:aws:iam::123456789012:role/lambda-basic-role \
  --zip-file fileb://function.zip
```

1.3.2 Python handler example

```
import json

def handler(event, context):
    name = event.get("name", "World")
    return {
        "statusCode": 200,
        "body": json.dumps(f"Hello, {name}!"),
    }
```

1.3.3 Terraform

```
resource "aws_lambda_function" "api" {
  function_name = "api-handler"
  runtime       = "python3.12"
  handler       = "app.handler"
  role          = aws_iam_role.lambda.arn
  filename      = "lambda.zip"
  source_code_hash = filebase64sha256("lambda.zip")

  memory_size = 256
  timeout      = 30

  environment {
    variables = {
      TABLE_NAME = aws_dynamodb_table.items.name
    }
  }

  vpc_config {
    subnet_ids      = aws_subnet.private[*].id
    security_group_ids = [aws_security_group.lambda.id]
  }
}
```

1.4 17.4 Event sources and triggers

Lambda integrates with 200+ AWS services and custom applications:

Trigger	Pattern
API Gateway	HTTP API requests
S3	Object created/deleted events

SQS	Message queue processing
SNS	Pub/sub notifications
EventBridge	Event bus rules
DynamoDB Streams	Change data capture
Kinesis	Stream record processing
CloudWatch Events	Scheduled (cron) execution
ALB	HTTP requests to Lambda targets

1.4.1 S3 trigger example

```
resource "aws_s3_bucket_notification" "trigger" {
  bucket = aws_s3_bucket.uploads.id

  lambda_function {
    lambda_function_arn = aws_lambda_function.processor.arn
    events              = ["s3:ObjectCreated:*"]
    filter_prefix       = "uploads/"
    filter_suffix       = ".csv"
  }
}
```

1.4.2 Scheduled execution

```
resource "aws_cloudwatch_event_rule" "daily" {
  name           = "daily-cleanup"
  schedule_expression = "cron(0 6 * * ? *)" # 6 AM UTC daily
}

resource "aws_cloudwatch_event_target" "lambda" {
  rule      = aws_cloudwatch_event_rule.daily.name
  target_id = "cleanup"
  arn       = aws_lambda_function.cleanup.arn
}
```

1.5 17.5 Concurrency

Concurrency is the number of simultaneous executions of your function.

1.5.1 Types

Type	Description
Unreserved	Shared pool; competes with other functions
Reserved	Guaranteed concurrency for a specific function
Provisioned	Pre-warmed execution environments (no cold start)

1.5.2 Concurrency formula

```
Account concurrency limit = 1,000 (default, per region)
Reserved concurrency for function X = 100
Unreserved pool = 1,000 - sum(all reserved)
```

1.5.3 Reserved concurrency

```
aws lambda put-function-concurrency \
```

```
--function-name api-handler \  
--reserved-concurrent-executions 50
```

Use reserved concurrency to: - Guarantee capacity for critical functions. - Limit maximum concurrency (acts as a throttle). - Prevent one function from consuming all account concurrency.

1.5.4 Provisioned concurrency

Eliminates cold starts by keeping execution environments warm:

```
aws lambda put-provisioned-concurrency-config \  
--function-name api-handler \  
--provisioned-concurrent-executions 10 \  
--qualifier live
```

Costs: provisioned concurrency is billed even when idle. Use for latency-sensitive APIs.

1.6 17.6 Cold starts and performance

A **cold start** occurs when Lambda creates a new execution environment:

Factor	Impact
Runtime	Java/.NET slower than Python/Node.js
Package size	Larger = slower init
VPC	Adds ENI creation time (improved with Hyperplane)
Provisioned concurrency	Eliminates cold starts

1.6.1 Optimization tips

- Keep deployment packages small; use **Lambda layers** for dependencies.
- Initialize SDK clients **outside** the handler (reused across invocations).
- Use **ARM64 (Graviton2)** for 20% better price-performance.
- Choose the right memory (more memory = more CPU = potentially faster).
- Use **SnapStart** for Java functions.

1.7 17.7 Lambda layers

Layers package libraries, custom runtimes, or other dependencies separately from your function code:

```
Function code (handler.py) + Layer (boto3, pandas, numpy) = Deployment
```

1.7.1 Benefits

- Share dependencies across multiple functions.
- Keep deployment packages small.
- Separate dependency updates from function code changes.

1.7.2 Creating a layer

```
mkdir -p python/lib/python3.12/site-packages  
pip install requests -t python/lib/python3.12/site-packages/  
zip -r layer.zip python  
  
aws lambda publish-layer-version \  
--function-name api-handler \  
--provisioned-concurrent-executions 50
```

```
--layer-name python-requests \  
--zip-file fileb://layer.zip \  
--compatible-runtimes python3.12
```

1.7.3 Attaching a layer

```
resource "aws_lambda_function" "api" {  
  # ...  
  layers = [  
    aws_lambda_layer_version.deps.arn,  
    "arn:aws:lambda:us-east-1:770693421928:layer:Klayers-p312-requests:1" # public layer  
  ]  
}
```

1.8 17.8 Lambda in a VPC

Place Lambda in a VPC to access private resources (RDS, ElastiCache, internal APIs):

```
vpc_config {  
  subnet_ids      = aws_subnet.private[*].id  
  security_group_ids = [aws_security_group.lambda.id]  
}
```

Requirements: - Subnets must have routes to the resource (not necessarily NAT for VPC-internal traffic). - Security groups on Lambda and target must allow communication. - Hyperplane ENIs reduce cold start penalty but still add some latency.

1.9 17.9 Error handling and retries

Trigger type	Retry behavior
Synchronous (API GW, ALB)	Caller handles retry
Asynchronous (S3, SNS, EventBridge)	2 automatic retries, then DLQ
Stream (Kinesis, DynamoDB)	Retry until data expires; bisect on failure
SQS	Retry based on visibility timeout; DLQ after max receives

1.9.1 Dead letter queue (DLQ)

```
resource "aws_lambda_function_event_invoke_config" "api" {  
  function_name = aws_lambda_function.api.function_name  
  
  destination_config {  
    on_failure {  
      destination = aws_sqs_queue.dlq.arn  
    }  
  }  
}
```

1.10 17.10 Monitoring

Tool	Purpose
CloudWatch Metrics	Invocations, Duration, Errors, Throttles, ConcurrentExecutions

CloudWatch Logs	Function stdout/stderr (auto-created log group)
X-Ray	Distributed tracing (enable active tracing)
Lambda Insights	Enhanced metrics (init duration, memory usage)

```
aws logs filter-log-events \
  --log-group-name /aws/lambda/api-handler \
  --filter-pattern "ERROR" \
  --start-time $(date -d '1 hour ago' +%s000)
```

1.11 17.11 Chapter summary

- **Lambda** runs event-driven code without server management; pay per invocation and duration.
- **Triggers** connect Lambda to 200+ event sources (API Gateway, S3, SQS, schedules).
- **Concurrency** controls — reserved (guarantee/limit), provisioned (no cold start).
- **Layers** separate dependencies from function code for reuse and smaller packages.
- Configure **DLQs**, **VPC access**, and **monitoring** for production reliability.

1.12 Lab 17.1 — S3-triggered processor

1. Create a Lambda function that processes CSV files uploaded to S3.
2. Configure an S3 event trigger for `uploads/*.csv`.
3. Upload a test CSV and verify the function logs and processes it.
4. Add error handling and a DLQ for failures.

1.13 Lab 17.2 — API with provisioned concurrency

1. Create a Lambda function behind API Gateway HTTP API.
2. Measure cold start latency with `curl -w "%{time_total}"`.
3. Enable provisioned concurrency (5) and measure again.
4. Compare p50 and p99 latency.

1.14 Review questions

1. What is the maximum Lambda timeout, and when would you need it?
2. How does reserved concurrency differ from provisioned concurrency?
3. Why should SDK clients be initialized outside the handler function?
4. What happens to failed asynchronous invocations after retries are exhausted?
5. What are the trade-offs of running Lambda in a VPC?

Next: [Chapter 18 — ECS & Fargate](#)

AWS Cloud Handbook — Complete Edition —

Chapter 18

1 Chapter 18: ECS & Fargate

1.1 18.1 Container orchestration on AWS

Amazon Elastic Container Service (ECS) is AWS’s native container orchestration platform. It schedules, deploys, and manages Docker containers across a cluster of EC2 instances or serverlessly via **AWS Fargate**. ECS integrates deeply with AWS networking, IAM, load balancing, and monitoring.

For teams that want AWS-native container management without the complexity of Kubernetes, ECS (especially with Fargate) is the recommended choice.

1.2 18.2 ECS architecture



1.2.1 Key components

Component	Description
Cluster	Logical grouping of tasks/services
Task definition	Blueprint: image, CPU, memory, ports, env vars, IAM role
Task	Running instance of a task definition (1+ containers)
Service	Maintains desired count of tasks; integrates with ALB
Container instance	EC2 host running the ECS agent (EC2 launch type only)

1.3 18.3 EC2 vs Fargate launch types

Feature	EC2 Launch Type	Fargate Launch Type
Infrastructure	You manage EC2 instances	AWS manages infrastructure
Pricing	EC2 instance hours	Per vCPU/memory per task hour
Scaling	ASG for instances + service scaling	Service scaling only

Control	Host access, GPU, custom AMIs	No host access
Best for	Cost optimization, GPU, daemon tasks	Simplicity, variable workloads

Recommendation: Start with Fargate. Move to EC2 if you need GPU, persistent host storage, or significant cost optimization at scale.

1.4 18.4 Task definitions

A **task definition** is a JSON blueprint specifying how containers run:

```
{
  "family": "web-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
  "taskRoleArn": "arn:aws:iam::123456789012:role/webAppTaskRole",
  "containerDefinitions": [
    {
      "name": "web",
      "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/web-app:latest",
      "portMappings": [
        { "containerPort": 8080, "protocol": "tcp" }
      ],
      "environment": [
        { "name": "ENV", "value": "production" }
      ],
      "secrets": [
        { "name": "DB_PASSWORD", "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:db-pass" }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/web-app",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "web"
        }
      },
      "healthCheck": {
        "command": ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"],
        "interval": 30,
        "timeout": 5,
        "retries": 3
      }
    }
  ]
}
```

1.4.1 IAM roles

Role	Purpose
Task execution role	Pull images from ECR, write logs, fetch secrets
Task role	Permissions for the application (S3, DynamoDB, etc.)

1.5 18.5 ECS services and load balancing

An **ECS service** maintains a desired number of running tasks and optionally registers them with a load balancer.

1.5.1 Terraform ECS Fargate service

```

resource "aws_ecs_service" "web" {
  name           = "web-service"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.web.arn
  desired_count  = 3
  launch_type    = "FARGATE"

  network_configuration {
    subnets          = aws_subnet.private[*].id
    security_groups    = [aws_security_group.ecs_tasks.id]
    assign_public_ip   = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.web.arn
    container_name   = "web"
    container_port    = 8080
  }

  deployment_configuration {
    maximum_percent      = 200
    minimum_healthy_percent = 100
  }

  depends_on = [aws_lb_listener.https]
}

```

1.5.2 Deployment types

Type	Description
Rolling update	Replace tasks incrementally (default)
Blue/green	Via CodeDeploy; shift traffic between target groups
Canary	Via CodeDeploy; gradual traffic shift

1.6 18.6 Auto Scaling ECS services

Scale tasks based on CloudWatch metrics:

```

resource "aws_appautoscaling_target" "ecs" {
  max_capacity   = 10
  min_capacity   = 2
  resource_id    = "service/${aws_ecs_cluster.main.name}/${aws_ecs_service.web.name}"
  scalable_dimension = "ecs:service:DesiredCount"
  service_namespace = "ecs"
}

resource "aws_appautoscaling_policy" "cpu" {
  name           = "cpu-scaling"
  policy_type    = "TargetTrackingScaling"
  resource_id    = aws_appautoscaling_target.ecs.resource_id
  scalable_dimension = aws_appautoscaling_target.ecs.scalable_dimension
  service_namespace = aws_appautoscaling_target.ecs.service_namespace

  target_tracking_scaling_policy_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ECSServiceAverageCPUUtilization"
    }
    target_value = 70.0
  }
}

```

1.7 18.7 Amazon ECR

Elastic Container Registry (ECR) stores Docker images:

```
# Authenticate
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com

# Build and push
docker build -t web-app .
docker tag web-app:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/web-app:latest
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/web-app:latest
```

1.7.1 ECR features

Feature	Purpose
Image scanning	Vulnerability scanning on push
Lifecycle policies	Auto-delete old images
Cross-account/region replication	DR and multi-account
Pull-through cache	Cache upstream registries (Docker Hub, etc.)

1.8 18.8 Networking modes

Mode	Use case
awsvpc	Required for Fargate; each task gets its own ENI and IP
bridge	EC2 only; Docker bridge networking (legacy)
host	EC2 only; task uses host network directly

Fargate requires `awsvpc` — plan IP address capacity in subnets (each task consumes one IP).

1.9 18.9 Service Connect and Service Discovery

Feature	Purpose
Cloud Map service discovery	DNS-based discovery (<code>api.local</code>)
ECS Service Connect	Built-in service mesh; observability, retries, traffic splitting

Service Connect simplifies inter-service communication without managing a full service mesh.

1.10 18.10 ECS Exec

ECS Exec provides interactive shell access to running containers (like `kubectl exec`):

```
aws ecs execute-command \
  --cluster my-cluster \
  --task abc123 \
  --container web \
  --interactive \
  --command "/bin/bash"
```

Requires SSM permissions on the task role and `enableExecuteCommand` on the service.

1.11 18.11 Chapter summary

- **ECS** orchestrates containers on EC2 or **Fargate** (serverless).
 - **Task definitions** define container images, resources, and IAM roles.
 - **Services** maintain desired task count and integrate with ALB.
 - **ECR** stores container images with scanning and lifecycle policies.
 - Use **awsvpc** networking, **auto scaling**, and **ECS Exec** for production operations.
-

1.12 Lab 18.1 — Fargate web service

1. Create an ECR repository and push a simple web container.
2. Create an ECS cluster, task definition, and Fargate service (2 tasks).
3. Attach an ALB and verify HTTP access.
4. Scale to 4 tasks and observe load distribution.

1.13 Lab 18.2 — ECS Exec debugging

1. Enable ECS Exec on your service.
 2. Connect to a running container and inspect logs, env vars, and network.
 3. Use ECS Exec to troubleshoot a misconfigured environment variable.
-

1.14 Review questions

1. What is the difference between a task definition and a task?
 2. When would you choose EC2 launch type over Fargate?
 3. What are the two IAM roles in an ECS task definition?
 4. Why does Fargate require awsvpc networking mode?
 5. How does ECS service auto scaling differ from EC2 Auto Scaling?
-

Next: [Chapter 19 — EKS](#)

AWS Cloud Handbook — Complete Edition —

Chapter 19

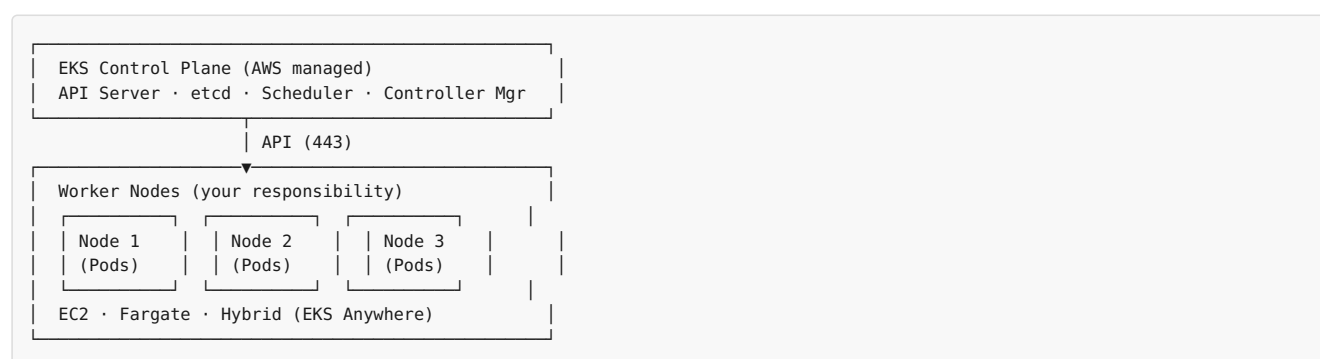
1 Chapter 19: EKS — Managed Kubernetes on AWS

1.1 19.1 Why Amazon EKS?

Amazon Elastic Kubernetes Service (EKS) is AWS's managed Kubernetes offering. Kubernetes has become the de facto standard for container orchestration at scale, and EKS provides a certified, managed control plane with deep AWS integration.

Choose EKS when you need Kubernetes portability, a rich ecosystem (Helm, operators, service mesh), multi-cloud strategy, or your team already has Kubernetes expertise.

1.2 19.2 EKS architecture



1.2.1 What AWS manages

- Kubernetes control plane (API server, etcd, scheduler, controllers)
- Control plane high availability across 3 AZs
- Control plane patching and upgrades
- Integrated logging and monitoring

1.2.2 What you manage

- Worker nodes (EC2, Fargate, or hybrid)
- Node scaling and patching
- Application deployments
- Networking (CNI plugin)
- Add-ons and cluster configuration

1.3 19.3 Cluster creation

1.3.1 eksctl (recommended for getting started)

```
# Install eksctl
curl -sLO "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz"
```

```
tar -xzf eksctl_*.tar.gz -C /tmp && sudo mv /tmp/eksctl /usr/local/bin

# Create cluster with managed node group
eksctl create cluster \
  --name production \
  --region us-east-1 \
  --version 1.29 \
  --nodegroup-name standard-workers \
  --node-type t3.medium \
  --nodes 3 \
  --nodes-min 2 \
  --nodes-max 6 \
  --managed
```

1.3.2 Terraform

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name = "production"
  cluster_version = "1.29"

  vpc_id = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnets

  eks_managed_node_groups = {
    general = {
      min_size = 2
      max_size = 10
      desired_size = 3
      instance_types = ["t3.medium"]
    }
  }

  enable_cluster_creator_admin_permissions = true
}
```

1.4 19.4 Node groups

Type	Description
Managed node group	AWS manages node lifecycle (launch, drain, terminate)
Self-managed	You manage ASG and node provisioning
Fargate profile	Serverless pods; no nodes to manage

1.4.1 Managed node group features

- Automatic AMI updates via EKS optimized AMIs
- Integrated with Cluster Autoscaler and Karpenter
- Rolling updates with drain and cordon
- Launch templates for customization

1.4.2 Karpenter (recommended autoscaler)

Karpenter provisions nodes dynamically based on pending pod requirements—faster and more cost-efficient than Cluster Autoscaler:

```
apiVersion: karpenter.sh/v1beta1
kind: NodePool
metadata:
  name: default
spec:
  template:
    spec:
```

```

requirements:
  - key: karpenter.sh/capacity-type
    operator: In
    values: ["spot", "on-demand"]
  - key: node.kubernetes.io/instance-type
    operator: In
    values: ["t3.medium", "t3.large", "m5.large"]
limits:
  cpu: 100
disruption:
  consolidationPolicy: WhenUnderutilized

```

1.5 19.5 Networking

1.5.1 VPC CNI

EKS uses the **Amazon VPC CNI plugin** — each pod gets a real VPC IP address:

- Plan IP capacity: each pod consumes an IP from the subnet.
- Use **prefix delegation** or **custom networking** for large clusters.
- Security groups can be applied directly to pods (Security Groups for Pods).

1.5.2 Service types

Type	AWS integration
ClusterIP	Internal only
NodePort	Exposes on node port
LoadBalancer	Creates ALB/NLB via AWS Load Balancer Controller
Ingress	ALB Ingress Controller for HTTP routing

1.5.3 AWS Load Balancer Controller

```

helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=production \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

```

1.6 19.6 IAM integration

1.6.1 IRSA (IAM Roles for Service Accounts)

Pods assume IAM roles via OIDC federation — no need for node-level credentials:

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/app-role
---
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      serviceAccountName: app-sa

```

```
containers:
  - name: app
    image: my-app:latest
```

1.6.2 EKS Pod Identity (newer)

Simpler alternative to IRSA; assigns IAM roles to pods via the EKS API without OIDC provider setup.

1.7 19.7 Essential add-ons

Add-on	Purpose
CoreDNS	Cluster DNS
kube-proxy	Network proxy on nodes
VPC CNI	Pod networking
EBS CSI Driver	Persistent volumes (EBS)
EFS CSI Driver	Shared persistent volumes (EFS)
Metrics Server	HPA resource metrics
Cluster Autoscaler / Karpenter	Node autoscaling

```
aws eks create-addon \
  --cluster-name production \
  --addon-name aws-ebs-csi-driver \
  --service-account-role-arn arn:aws:iam::123456789012:role/AmazonEKS_EBS_CSI_DriverRole
```

1.8 19.8 Storage

Type	Use case	Provisioner
EBS (gp3)	Single-pod persistent storage	EBS CSI Driver
EFS	Shared storage across pods	EFS CSI Driver
FSx for Lustre	High-performance computing	FSx CSI Driver
emptyDir	Ephemeral pod storage	Built-in

1.8.1 PersistentVolumeClaim example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources:
    requests:
      storage: 20Gi
```

1.9 19.9 Security

Layer	Control
Control plane	Private endpoint, public endpoint, or both
Network policies	Calico or VPC CNI network policies
Pod security	Pod Security Standards (restricted, baseline)
Secrets	External Secrets Operator + Secrets Manager
Image scanning	ECR scanning, admission controllers
Audit logging	EKS control plane logs to CloudWatch

```
aws eks update-cluster-config \
  --name production \
  --logging '{"clusterLogging":[{"types":["api","audit","authenticator"],"enabled":true}]}'
```

1.10 19.10 Upgrades and operations

1.10.1 Cluster upgrade path

1. Upgrade control plane to target Kubernetes version.
2. Upgrade add-ons (CoreDNS, kube-proxy, VPC CNI).
3. Upgrade node groups (rolling replacement).
4. Update application manifests for deprecated APIs.

```
# Check deprecated APIs
kubectl pluto detect-deprecated-apis --target-versions k8s=v1.29.0

# Upgrade control plane
aws eks update-cluster-version --name production --kubernetes-version 1.29
```

1.10.2 Useful kubectl commands

```
kubectl get nodes -o wide
kubectl top nodes
kubectl get pods --all-namespaces -o wide
kubectl describe pod <name> -n <namespace>
kubectl logs -f <pod> -c <container>
```

1.11 19.11 EKS vs ECS decision matrix

Factor	Choose EKS	Choose ECS
Team Kubernetes expertise	Yes	No
Multi-cloud portability	Yes	No
AWS-native simplicity	No	Yes
Ecosystem (Helm, operators)	Yes	Limited
Operational overhead	Higher	Lower
Fargate support	Yes	Yes

1.12 19.12 Chapter summary

- **EKS** provides a managed Kubernetes control plane with AWS integrations.
 - **Managed node groups**, **Fargate**, and **Karpenter** handle worker capacity.
 - **IRSA** and **Pod Identity** grant fine-grained IAM permissions to pods.
 - **AWS Load Balancer Controller** and **EBS/EFS CSI drivers** are essential add-ons.
 - Plan **IP capacity**, **upgrades**, and **security** from day one.
-

1.13 Lab 19.1 — EKS cluster deployment

1. Create an EKS cluster with eksctl (3-node managed node group).
2. Deploy a sample application with a LoadBalancer service.
3. Configure IRSA for an S3-accessing pod.
4. Install Metrics Server and verify `kubectl top nodes`.

1.14 Lab 19.2 — Persistent storage

1. Install the EBS CSI driver add-on.
 2. Create a StorageClass, PVC, and Pod mounting the volume.
 3. Write data to the volume, delete the pod, recreate it, and verify data persists.
-

1.15 Review questions

1. What components of Kubernetes does AWS manage in EKS?
 2. How does IRSA enable pods to access AWS services securely?
 3. Why is IP address planning critical for EKS clusters?
 4. What is the recommended order for EKS cluster upgrades?
 5. When would you choose ECS over EKS?
-

Next: [Chapter 20 — S3 Fundamentals](#)

AWS Cloud Handbook — Complete Edition —

Chapter 20

1 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle

1.1 20.1 Amazon S3 overview

Simple Storage Service (S3) is object storage: store files (objects) in containers (buckets). Virtually unlimited scale; 99.999999999% durability.

Use cases:

- Static websites and assets
- Data lakes and backups
- Application uploads
- Log archives
- Terraform state files

1.2 20.2 Core concepts

Concept	Description
Bucket	Global unique name; lives in a region
Object	File + metadata; up to 5 TB
Key	Object path (e.g., <code>images/photo.jpg</code>)
Prefix	Simulated folders via key prefixes

```
aws s3 mb s3://my-unique-bucket-name-12345
aws s3 cp report.pdf s3://my-bucket/reports/2026/
aws s3 ls s3://my-bucket/reports/
aws s3 sync ./local-dir s3://my-bucket/backup/
aws s3 rm s3://my-bucket/old/ --recursive
```

1.3 20.3 Storage classes

Class	Use case	Retrieval
Standard	Frequent access	Immediate
Standard-IA	Infrequent access	Immediate, min storage charge
Glacier Instant	Archive, instant retrieval	Immediate
Glacier Flexible	Archive	Minutes to hours
Glacier Deep Archive	Long-term archive	12–48 hours

Intelligent-Tiering	Unknown access patterns	Auto-moves tiers
---------------------	-------------------------	------------------

1.4 20.4 Versioning and lifecycle

1.4.1 Versioning

Protects against accidental deletes; keeps all versions of objects.

```
aws s3api put-bucket-versioning \
  --bucket my-bucket \
  --versioning-configuration Status=Enabled
```

1.4.2 Lifecycle rules

Automatically transition or expire objects:

```
{
  "Rules": [{
    "ID": "archive-old-logs",
    "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [{
      "Days": 30,
      "StorageClass": "STANDARD_IA"
    }, {
      "Days": 90,
      "StorageClass": "GLACIER"
    }],
    "Expiration": { "Days": 365 }
  }]
}
```

1.5 20.5 Security

1.5.1 Block Public Access (account and bucket level)

Enable by default on all buckets unless hosting a public static site intentionally.

1.5.2 Encryption

Method	Description
SSE-S3	AWS-managed keys
SSE-KMS	KMS keys; audit trail
SSE-C	Customer-provided keys
Client-side	Encrypt before upload

1.5.3 Bucket policies vs ACLs

- **Bucket policies** (JSON) — Preferred; resource-based IAM
- **ACLs** — Legacy; avoid for new projects

Example: allow CloudFront OAC to read objects (not public internet).

1.6 20.6 Static website hosting

```
# Enable static website hosting on bucket
aws s3 website s3://my-bucket/ --index-document index.html
```

Front with **CloudFront** + **ACM certificate** for HTTPS. Do not expose bucket directly to public.

1.7 20.7 S3 performance tips

- Use **multipart upload** for files > 100 MB
 - Prefix keys for parallelism (S3 scales automatically)
 - **S3 Transfer Acceleration** for long-distance uploads
 - **S3 Inventory** for auditing object metadata
-

1.8 20.8 Chapter summary

- S3 stores **objects in buckets** with global-unique names.
 - Enable **versioning** and **lifecycle** for data protection and cost control.
 - **Block public access**; use bucket policies and CloudFront for controlled delivery.
-

1.9 Lab 20.1

1. Create bucket with versioning and default encryption (SSE-S3).
 2. Upload files; delete one; restore previous version.
 3. Add lifecycle rule to transition `logs/` to IA after 30 days.
 4. Host static `index.html` behind CloudFront.
-

Next: [Chapter 35 — CloudWatch](#)

AWS Cloud Handbook — Complete Edition —

Chapter 21

1 Chapter 21: S3 Security, Encryption, and Static Websites

1.1 21.1 S3 security model

Amazon S3 security operates at multiple layers: **identity-based policies** (IAM), **resource-based policies** (bucket policies), **ACLs** (legacy), **block public access** settings, and **encryption**. Understanding how these layers interact is critical for preventing data breaches—the majority of S3 incidents involve misconfigured public access.

1.2 21.2 Block Public Access

S3 Block Public Access is the first line of defense. Enable it at the **account level** and verify it on every bucket:

Setting	Effect
BlockPublicAcls	Blocks new public ACLs
IgnorePublicAcls	Ignores existing public ACLs
BlockPublicPolicy	Blocks bucket policies granting public access
RestrictPublicBuckets	Restricts access to buckets with public policies

```
aws s3control put-public-access-block \
  --account-id 123456789012 \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Best practice: Enable account-level block public access. Only disable for buckets that intentionally serve public content (static websites, CDN origins).

1.3 21.3 Bucket policies

Bucket policies are **resource-based** JSON policies attached to buckets:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOAC",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-static-site/*",
      "Condition": {
        "StringEquals": {
```

```

    "AWS:SourceArn": "arn:aws:cloudfront::123456789012:distribution/E1234567890"
  }
}
]
}

```

1.3.1 Common policy patterns

Pattern	Principal	Use case
Cross-account access	Another account's IAM role	Shared data lake
CloudFront OAC	cloudfront.amazonaws.com	Static website via CDN
VPC endpoint restriction	"*" with aws:SourceVpce condition	Private S3 access
Deny unencrypted uploads	"*" with Deny + s3:x-amz-server-side-encryption	Compliance

1.3.2 Deny insecure transport

```

{
  "Sid": "DenyInsecureTransport",
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": [
    "arn:aws:s3::my-bucket",
    "arn:aws:s3::my-bucket/*"
  ],
  "Condition": {
    "Bool": {
      "aws:SecureTransport": "false"
    }
  }
}

```

1.4 21.4 IAM policies for S3

Identity-based policies attach to IAM users, groups, or roles:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3::app-data-${aws:username}/*"
    }
  ]
}

```

1.4.1 Policy evaluation for S3

Request → Block Public Access → Bucket Policy → IAM Policy → ACL → Allow/Deny

Explicit **Deny** in any policy wins. Both identity and resource policies must allow for cross-account access.

1.5 21.5 Encryption

S3 supports encryption at rest and in transit.

1.5.1 Server-side encryption (SSE)

Type	Key management	Use case
SSE-S3	AWS-managed (AES-256)	Default; no key management
SSE-KMS	AWS KMS customer-managed or AWS-managed key	Audit trail, key rotation, cross-account
SSE-C	Customer-provided key	You manage keys outside AWS
DSSE-KMS	Dual-layer KMS encryption	Highest security requirements

1.5.2 Default encryption

```
aws s3api put-bucket-encryption \
  --bucket my-bucket \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "aws:kms",
        "KMSEMasterKeyID": "arn:aws:kms:us-east-1:123456789012:key/abc-123"
      },
      "BucketKeyEnabled": true
    }]
  }'
```

Bucket Key reduces KMS API calls (and costs) by using a bucket-level key that wraps object keys.

1.5.3 Client-side encryption

Encrypt data before uploading with the AWS Encryption SDK or your own encryption library. You manage keys entirely.

1.5.4 Encryption in transit

- All S3 API calls use HTTPS by default.
- Enforce with bucket policy `aws:SecureTransport` condition.
- TLS 1.2+ required.

1.6 21.6 Access Points and Object Lambda

1.6.1 S3 Access Points

Named network endpoints with dedicated bucket policies—simplify access for shared buckets:

```
Shared bucket → Access Point "finance" (finance team policy)
               → Access Point "engineering" (engineering team policy)
```

1.6.2 S3 Object Lambda

Transform objects on retrieval (resize images, redact PII, convert formats) without modifying the stored object.

1.7 21.7 Static website hosting

S3 can host static websites (HTML, CSS, JS, images):

1.7.1 Setup steps

1. Enable static website hosting on the bucket.
2. Set `index.html` as index document and `error.html` as error document.
3. Upload website files.
4. Configure bucket policy for public read (or use CloudFront OAC for private bucket).

```
aws s3 website s3://my-static-site/ \  
--index-document index.html \  
--error-document error.html
```

1.7.2 Production architecture (recommended)

```
User → Route 53 → CloudFront → S3 (private bucket via OAC)  
      |  
      v  
    ACM certificate (HTTPS)  
    WAF (optional)
```

Never expose S3 directly to the internet in production. Use CloudFront with Origin Access Control (OAC) to keep the bucket private.

1.7.3 Terraform static site

```
resource "aws_s3_bucket" "site" {  
  bucket = "my-static-site"  
}  
  
resource "aws_s3_bucket_website_configuration" "site" {  
  bucket = aws_s3_bucket.site.id  
  index_document { suffix = "index.html" }  
  error_document { key = "error.html" }  
}  
  
resource "aws_cloudfront_distribution" "site" {  
  origin {  
    domain_name      = aws_s3_bucket.site.bucket_regional_domain_name  
    origin_id        = "s3"  
    origin_access_control_id = aws_cloudfront_origin_access_control.site.id  
  }  
  
  enabled          = true  
  default_root_object = "index.html"  
  
  default_cache_behavior {  
    allowed_methods      = ["GET", "HEAD"]  
    cached_methods      = ["GET", "HEAD"]  
    target_origin_id     = "s3"  
    viewer_protocol_policy = "redirect-to-https"  
    forwarded_values {  
      query_string = false  
      cookies { forward = "none" }  
    }  
  }  
}  
  
viewer_certificate {  
  acm_certificate_arn      = aws_acm_certificate.site.arn  
  ssl_support_method       = "sni-only"  
  minimum_protocol_version = "TLSv1.2_2021"  
}  
  
restrictions {  
  geo_restriction { restriction_type = "none" }  
}  
}
```

1.8 21.8 S3 access logging and monitoring

Feature	Purpose
Server access logging	Log all bucket requests to another bucket
CloudTrail data events	API-level logging (GetObject, PutObject)
S3 Storage Lens	Organization-wide storage analytics
S3 Inventory	Scheduled reports of objects and metadata
Macie	ML-based sensitive data discovery

```
aws cloudtrail put-event-selectors \  
  --trail-name org-trail \  
  --event-selectors '[{  
    "ReadWriteType": "All",  
    "IncludeManagementEvents": true,  
    "DataResources": [{  
      "Type": "AWS::S3::Object",  
      "Values": ["arn:aws:s3:::sensitive-bucket/"]  
    }]  
}]'
```

1.9 21.9 Pre-signed URLs and temporary access

Grant time-limited access without making objects public:

```
import boto3  
  
s3 = boto3.client("s3")  
url = s3.generate_presigned_url(  
    "get_object",  
    Params={"Bucket": "my-bucket", "Key": "report.pdf"},  
    ExpiresIn=3600, # 1 hour  
)
```

Use cases: download links, upload forms, sharing files with external parties.

1.10 21.10 Chapter summary

- Enable **Block Public Access** at the account level; only disable for intentional public buckets.
- Use **bucket policies** and **IAM policies** together for fine-grained access control.
- Encrypt with **SSE-KMS** for audit trails; enable **Bucket Key** to reduce KMS costs.
- Host static websites behind **CloudFront + OAC**, not direct S3 public access.
- Monitor with **CloudTrail data events**, **access logging**, and **Macie**.

1.11 Lab 21.1 — Secure bucket

1. Create a bucket with default SSE-KMS encryption and block public access.
2. Add a bucket policy denying unencrypted uploads and insecure transport.
3. Attempt to upload without encryption — verify denial.
4. Upload with `--server-side-encryption aws:kms` — verify success.

1.12 Lab 21.2 — Static website with CloudFront

1. Create an S3 bucket with static website files.
 2. Create a CloudFront distribution with OAC (bucket stays private).
 3. Configure Route 53 alias record and ACM certificate.
 4. Verify HTTPS access through CloudFront.
-

1.13 Review questions

1. What are the four S3 Block Public Access settings?
 2. When would you use SSE-KMS over SSE-S3?
 3. Why should production static sites use CloudFront OAC instead of public bucket policies?
 4. How does a pre-signed URL provide temporary access?
 5. What is the purpose of S3 Bucket Key?
-

Next: [Chapter 22 — EBS & EFS](#)

AWS Cloud Handbook — Complete Edition —

Chapter 22

1 Chapter 22: EBS & EFS

1.1 22.1 Block vs file storage on AWS

AWS provides two primary storage services for compute workloads:

Service	Type	Attachment	Use case
Amazon EBS	Block storage	Single EC2 instance (per volume)	Boot volumes, databases, single-instance apps
Amazon EFS	File storage (NFS)	Multiple instances simultaneously	Shared files, content management, web serving

Understanding when to use each—and their performance, durability, and cost characteristics—is essential for architecting reliable applications.

1.2 22.2 Amazon EBS deep dive

Elastic Block Store (EBS) provides persistent block-level storage volumes for EC2 instances. Think of EBS as a virtual hard drive.

1.2.1 Volume types

Type	Storage	IOPS	Throughput	Use case
gp3	SSD	3,000–16,000 (configurable)	125–1,000 MB/s	General purpose (default)
gp2	SSD	3–16,000 (burst/scaled)	128–250 MB/s	Legacy general purpose
io2 Block Express	SSD	Up to 256,000	Up to 4,000 MB/s	Mission-critical databases
io1/io2	SSD	Up to 64,000	Up to 1,000 MB/s	High-IOPS databases
st1	HDD	500 baseline	Up to 500 MB/s	Throughput-intensive (big data)
sc1	HDD	250 baseline	Up to 250 MB/s	Cold data, infrequent access

Recommendation: Use **gp3** for most workloads. It offers better price-performance than gp2 with independently configurable IOPS and throughput.

1.2.2 Key characteristics

- **AZ-bound** — A volume exists in one AZ; snapshot and restore to another AZ.
- **Durability** — 99.999% (replicated within AZ).
- **Encryption** — All volume types support encryption (default with account setting).

- **Size** — 1 GB to 64 TB per volume.
- **Multi-attach** — io1/io2 support attachment to multiple instances (cluster file systems only).

1.2.3 Creating and attaching volumes

```
# Create a gp3 volume
aws ec2 create-volume \
  --availability-zone us-east-1a \
  --size 100 \
  --volume-type gp3 \
  --iops 3000 \
  --throughput 125 \
  --encrypted \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=data-vol}]'

# Attach to instance
aws ec2 attach-volume \
  --volume-id vol-0abc123 \
  --instance-id i-0def456 \
  --device /dev/sdf

# Format and mount (on the instance)
sudo mkfs -t xfs /dev/xvdf
sudo mkdir /data && sudo mount /dev/xvdf /data
echo '/dev/xvdf /data xfs defaults,nofail 0 2' | sudo tee -a /etc/fstab
```

1.2.4 Terraform

```
resource "aws_ebs_volume" "data" {
  availability_zone = "us-east-1a"
  size             = 100
  type             = "gp3"
  iops             = 3000
  throughput       = 125
  encrypted        = true
  kms_key_id       = aws_kms_key.ebs.arn

  tags = { Name = "app-data" }
}

resource "aws_volume_attachment" "data" {
  device_name = "/dev/sdf"
  volume_id   = aws_ebs_volume.data.id
  instance_id = aws_instance.app.id
}
```

1.3 22.3 EBS snapshots

Snapshots are point-in-time backups stored in S3 (managed by AWS):

Feature	Detail
Incremental	Only changed blocks since last snapshot
Cross-region	Copy snapshots for DR
Cross-account	Share snapshots with other accounts
Fast Snapshot Restore	Pre-warm snapshots for immediate IOPS (costly)
Lifecycle	Automate with Data Lifecycle Manager (DLM)

```
# Create snapshot
aws ec2 create-snapshot \
  --volume-id vol-0abc123 \
  --description "Daily backup" \
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=daily-backup}]'
```

```
# Create AMI from snapshot (for golden images)
aws ec2 register-image \
  --name "app-ami-v2" \
  --block-device-mappings '[{
    "DeviceName": "/dev/xvda",
    "Ebs": {"SnapshotId": "snap-0abc123", "VolumeSize": 20, "VolumeType": "gp3"}
  }]'
```

1.3.1 DLM policy example

```
resource "aws_dlm_lifecycle_policy" "ebs_backup" {
  description      = "Daily EBS snapshots"
  execution_role_arn = aws_iam_role.dlm.arn
  state            = "ENABLED"

  policy_details {
    resource_types = ["VOLUME"]
    schedule {
      name = "daily"
      create_rule { interval = 24; interval_unit = "HOURS"; times = ["03:00"] }
      retain_rule { count = 7 }
      copy_tags = true
    }
    target_tags = { Backup = "true" }
  }
}
```

1.4 22.4 Amazon EFS deep dive

Elastic File System (EFS) provides scalable, elastic NFS file storage accessible from multiple EC2 instances, ECS tasks, and Lambda functions simultaneously.

1.4.1 Performance modes

Mode	Throughput	Use case
General Purpose	Low latency	Web serving, CMS, dev environments
Max I/O	Higher aggregate throughput	Big data, media processing

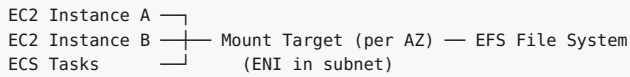
1.4.2 Throughput modes

Mode	Behavior
Bursting	Scales with storage size; burst credits
Provisioned	Set throughput independent of storage size
Elastic (default)	Automatically scales throughput up and down

1.4.3 Storage classes

Class	Description
Standard	Frequently accessed files
Infrequent Access (IA)	Lifecycle policy moves files after N days
Archive	Rarely accessed; lowest cost
One Zone	Single AZ; lower cost, less resilience

1.4.4 EFS architecture



1.4.5 Creating EFS

```

# Create file system
aws efs create-file-system \
  --performance-mode generalPurpose \
  --throughput-mode elastic \
  --encrypted \
  --tags Key=Name,Value=shared-data

# Create mount targets in each AZ
aws efs create-mount-target \
  --file-system-id fs-0abc123 \
  --subnet-id subnet-0aaa \
  --security-groups sg-0efs

# Mount on EC2 (Amazon Linux)
sudo yum install -y amazon-efs-utils
sudo mount -t efs -o tls fs-0abc123:/ /mnt/efs
  
```

1.4.6 Terraform

```

resource "aws_efs_file_system" "shared" {
  creation_token   = "shared-data"
  performance_mode = "generalPurpose"
  throughput_mode  = "elastic"
  encrypted        = true

  lifecycle_policy {
    transition_to_ia = "AFTER_30_DAYS"
  }

  tags = { Name = "shared-efs" }
}

resource "aws_efs_mount_target" "az" {
  count                = length(var.private_subnet_ids)
  file_system_id       = aws_efs_file_system.shared.id
  subnet_id            = var.private_subnet_ids[count.index]
  security_groups      = [aws_security_group.efs.id]
}
  
```

1.5 22.5 EBS vs EFS comparison

Feature	EBS	EFS
Type	Block	File (NFS)
Access	Single instance (multi-attach for io1/io2)	Multiple instances
AZ	Single AZ	Regional (multi-AZ)
Max size	64 TB	Petabyte scale
Latency	Sub-millisecond	Low (NFS overhead)
Cost	Lower per GB	Higher per GB
Boot volume	Yes	No
Backup	Snapshots	AWS Backup

1.6 22.6 Instance Store (ephemeral)

Instance store volumes are physically attached to the host:

- **Highest IOPS** and lowest latency.
 - **Ephemeral** — data lost on stop/terminate or hardware failure.
 - Use for temporary data: caches, buffers, scratch space.
 - Cannot be detached and reattached.
-

1.7 22.7 EBS optimization and monitoring

Metric	What to watch
VolumeQueueLength	I/O wait; sustained > 1 may need more IOPS
VolumeReadBytes / VolumeWriteBytes	Throughput utilization
BurstBalance (gp2)	Burst credit depletion
PercentIOLimit (io1/io2)	IOPS utilization

1.7.1 Right-sizing tips

- Start with gp3; increase IOPS/throughput independently.
 - Use CloudWatch metrics to identify I/O-bound volumes.
 - Consider io2 Block Express for databases requiring > 64K IOPS.
-

1.8 22.8 Chapter summary

- **EBS** provides block storage for single-instance workloads; **gp3** is the default choice.
 - **Snapshots** enable backup, AMI creation, and cross-region DR.
 - **EFS** provides shared NFS file storage across multiple instances and AZs.
 - Use **DLM** for automated snapshot lifecycle management.
 - Choose EBS for databases and boot volumes; EFS for shared file access.
-

1.9 Lab 22.1 — EBS volume lifecycle

1. Launch an EC2 instance and attach a 20 GB gp3 volume.
2. Format, mount, and write test data.
3. Create a snapshot, terminate the instance, launch a new instance, and restore from snapshot.
4. Verify data persistence.

1.10 Lab 22.2 — Shared EFS

1. Create an EFS file system with mount targets in two AZs.
 2. Launch EC2 instances in both AZs and mount the EFS.
 3. Write a file from Instance A and read it from Instance B.
 4. Configure a lifecycle policy to move files to IA after 30 days.
-

1.11 Review questions

1. What is the key difference between EBS and EFS access patterns?
 2. Why is gp3 preferred over gp2 for new workloads?
 3. What happens to instance store data when an EC2 instance is stopped?
 4. How do EBS snapshots support disaster recovery?
 5. When would you use EFS Infrequent Access storage class?
-

Next: [*Chapter 23 — AWS Backup & DR*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 23

1 Chapter 23: AWS Backup & Disaster Recovery Patterns

1.1 23.1 Why backup and disaster recovery matter

Data loss—from accidental deletion, ransomware, regional outages, or application bugs—can destroy businesses. AWS provides native backup capabilities and architectural patterns to protect workloads with defined **Recovery Time Objective (RTO)** and **Recovery Point Objective (RPO)** targets.

Metric	Definition	Example
RPO	Maximum acceptable data loss (time)	1 hour = lose at most 1 hour of data
RTO	Maximum acceptable downtime	4 hours = back online within 4 hours

1.2 23.2 AWS Backup

AWS Backup is a centralized backup service that automates and manages backups across AWS services:

Supported resource	Backup method
EC2 / EBS	EBS snapshots
RDS / Aurora	DB snapshots
DynamoDB	On-demand or continuous backup
EFS	EFS backup
FSx	FSx backup
S3	S3 backup (newer)
Storage Gateway	Volume backup

1.2.1 Backup components

Component	Description
Backup vault	Container for recovery points; encryption with KMS
Backup plan	Schedule, lifecycle, and rules
Backup rule	Frequency, retention, target vault, copy actions
Recovery point	A restorable backup at a point in time
Backup selection	Resources matched by tag or ID

1.2.2 Terraform backup plan

```
resource "aws_backup_vault" "primary" {
  name          = "primary-vault"
  kms_key_arn   = aws_kms_key.backup.arn
}

resource "aws_backup_plan" "daily" {
  name = "daily-backup-plan"

  rule {
    rule_name          = "daily-ec2-rds"
    target_vault_name = aws_backup_vault.primary.name
    schedule            = "cron(0 5 * * ? *)" # 5 AM UTC daily

    lifecycle {
      delete_after = 30 # days
    }

    copy_action {
      destination_vault_arn = aws_backup_vault.dr.arn
      lifecycle {
        delete_after = 90
      }
    }
  }
}

resource "aws_backup_selection" "tagged" {
  name          = "production-resources"
  plan_id       = aws_backup_plan.daily.id
  iam_role_arn  = aws_iam_role.backup.arn

  selection_tag {
    type = "STRINGEQUALS"
    key  = "Backup"
    value = "daily"
  }
}
```

1.2.3 CLI

```
# Start on-demand backup
aws backup start-backup-job \
  --backup-vault-name primary-vault \
  --resource-arn arn:aws:ec2:us-east-1:123456789012:volume/vol-0abc123 \
  --iam-role-arn arn:aws:iam::123456789012:role/AWSBackupDefaultServiceRole

# List recovery points
aws backup list-recovery-points-by-backup-vault \
  --backup-vault-name primary-vault

# Restore
aws backup start-restore-job \
  --recovery-point-arn arn:aws:backup:us-east-1:123456789012:recovery-point:abc123 \
  --iam-role-arn arn:aws:iam::123456789012:role/AWSBackupDefaultServiceRole \
  --metadata file://restore-metadata.json
```

1.3 23.3 DR strategies

AWS defines four disaster recovery strategies, ordered by increasing RTO/RPO and cost:

Cost / Complexity	→		
Backup & Restore	Pilot Light	Warm Standby	Multi-Site Active/Active
(highest RTO)			(lowest RTO)
(highest RPO)			(lowest RPO)

1.3.1 Backup and Restore

- **RPO:** Hours to days | **RTO:** Hours to days
- Back up data to S3/EBS snapshots; restore in DR region when needed.
- Lowest cost; acceptable for non-critical workloads.

1.3.2 Pilot Light

- **RPO:** Minutes to hours | **RTO:** Hours
- Core infrastructure (database replica, AMIs) running at minimal scale in DR region.
- Scale up on failover.

1.3.3 Warm Standby

- **RPO:** Minutes | **RTO:** Minutes to hours
- Reduced-capacity version of production running in DR region.
- Scale up ASG/ECS on failover.

1.3.4 Multi-Site Active/Active

- **RPO:** Near zero | **RTO:** Near zero
- Full production in multiple regions simultaneously.
- Route 53 health checks shift traffic on failure.
- Highest cost and complexity.

1.4 23.4 Cross-region and cross-account backup

1.4.1 Cross-region copy

Protect against regional failures by copying backups to a DR region:

```
copy_action {
  destination_vault_arn = aws_backup_vault.dr_region.arn
  lifecycle {
    cold_storage_after = 30
    delete_after      = 365
  }
}
```

1.4.2 Cross-account backup

Protect against account compromise (ransomware, insider threat):

1. Create backup vault in a separate **log archive** or **security** account.
2. Configure cross-account copy in the backup plan.
3. Apply vault access policies restricting who can delete recovery points.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::PROD_ACCOUNT:root"},
    "Action": "backup:CopyIntoBackupVault",
    "Resource": "*"
  }]
}
```

1.5 23.5 Service-specific DR patterns

1.5.1 RDS / Aurora

Feature	RPO	Use case
Automated backups	5 minutes (transaction logs)	Point-in-time recovery
Read replica (cross-region)	Seconds	DR with promotion
Aurora Global Database	< 1 second	Multi-region with fast failover
Snapshot copy	Hours	Long-term retention

```
# Create cross-region read replica
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-replica-dr \
  --source-db-instance-identifier arn:aws:rds:us-east-1:123456789012:db:prod \
  --region us-west-2
```

1.5.2 S3

Feature	Purpose
Cross-Region Replication (CRR)	Replicate objects to DR region
S3 Versioning	Recover from accidental overwrites/deletes
MFA Delete	Prevent deletion without MFA
Object Lock	WORM compliance; ransomware protection

1.5.3 DynamoDB

Feature	RPO
On-demand backup	Point-in-time
Continuous backup (PITR)	35 days, per-second recovery
Global Tables	Near-zero; multi-region active-active

1.6 23.6 Failover automation

1.6.1 Route 53 failover

Health checks on primary region; automatic DNS failover to DR region ALB/CloudFront.

1.6.2 Database failover

```
# Promote Aurora Global Database secondary
aws rds failover-global-cluster \
  --global-cluster-identifier my-global-cluster \
  --target-db-cluster-identifier dr-cluster
```

1.6.3 Application failover

- ASG/ECS desired count increase in DR region (via Lambda + CloudWatch alarm).
- Step Functions orchestrate multi-step failover runbooks.

1.7 23.7 Testing DR

Untested backups are not backups. Regular DR testing is required:

Test type	Frequency	Scope
Backup verification	Weekly (automated)	Restore random recovery point
Tabletop exercise	Quarterly	Walk through failover runbook
Partial failover	Semi-annually	Fail over one service to DR
Full DR drill	Annually	Complete regional failover

Document RTO/RPO achieved during each test and update runbooks.

1.8 23.8 Compliance and retention

Requirement	AWS feature
Immutable backups	S3 Object Lock, Backup Vault Lock
Long-term retention	Glacier via lifecycle; Backup cold storage
Audit trail	CloudTrail logs all backup API calls
Encryption	KMS keys per vault; separate keys per account

1.8.1 Backup Vault Lock

Prevent deletion of recovery points before their retention period expires—even by root account.

```
aws backup put-backup-vault-lock-configuration \
  --backup-vault-name compliance-vault \
  --min-retention-days 90 \
  --changeable-for-days 3
```

1.9 23.9 Chapter summary

- **AWS Backup** centralizes backup policies across EC2, RDS, EFS, DynamoDB, and more.
- DR strategies range from **backup & restore** (cheapest) to **active/active** (fastest recovery).
- Use **cross-region** and **cross-account** copies for comprehensive protection.
- Define **RPO** and **RTO** targets and test DR plans regularly.
- Enable **Vault Lock** and **Object Lock** for ransomware-resistant backups.

1.10 Lab 23.1 — AWS Backup plan

1. Create a backup vault with KMS encryption.
2. Create a backup plan with daily backups and 14-day retention.
3. Tag an EC2 instance with `Backup=daily` and verify it is included.
4. Trigger an on-demand backup and restore to a new volume.

1.11 Lab 23.2 — Cross-region replication

1. Enable S3 versioning and CRR on a production bucket.

2. Upload objects and verify replication to the DR region.
 3. Simulate primary region failure by blocking access and reading from DR.
-

1.12 Review questions

1. What is the difference between RPO and RTO?
 2. Which DR strategy offers the lowest RTO and highest cost?
 3. Why is cross-account backup important for ransomware protection?
 4. How does Aurora Global Database achieve sub-second RPO?
 5. Why should DR plans be tested regularly?
-

Next: [*Chapter 24 — RDS & Aurora*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 24

1 Chapter 24: RDS & Aurora

1.1 24.1 Managed relational databases

Amazon RDS provides managed relational databases (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server) with automated patching, backups, monitoring, and Multi-AZ failover. **Amazon Aurora** is AWS's cloud-native database engine—MySQL and PostgreSQL compatible—with superior performance, storage auto-scaling, and global distribution.

For most new workloads, **Aurora PostgreSQL** or **Aurora MySQL** is the recommended choice over standard RDS.

1.2 24.2 RDS fundamentals

1.2.1 Supported engines

Engine	Versions	Licensing
PostgreSQL	12-16	Open source
MySQL	5.7, 8.0	Open source
MariaDB	10.x	Open source
Oracle	19c, 21c	BYOL or License Included
SQL Server	2019, 2022	License Included
Aurora	MySQL 5.7/8.0, PostgreSQL compatible	AWS proprietary

1.2.2 Instance classes

Class	Use case
db.t3/t4g	Dev/test, burstable
db.m6g/m7g	General purpose, Graviton
db.r6g/r7g	Memory-optimized, large datasets
db.x2g	Extreme memory (Aurora)

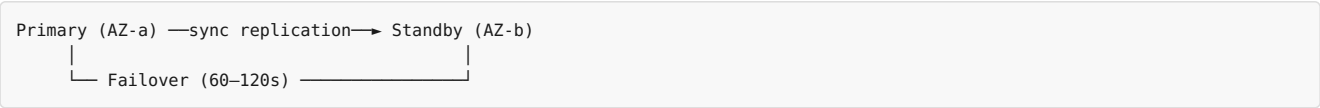
1.2.3 Storage types

Type	IOPS	Use case
gp3	3,000–16,000	General purpose (default)
io2	Up to 256,000	I/O-intensive workloads
magnetic	Legacy	Deprecated

1.3 24.3 High availability

1.3.1 Multi-AZ deployment

RDS synchronously replicates to a **standby instance** in another AZ:



- Automatic failover on primary failure.
- Same endpoint DNS (points to new primary after failover).
- Standby not readable (except Aurora).

1.3.2 Read replicas

Asynchronous replication for **read scaling** and DR:

Feature	Multi-AZ	Read Replica
Purpose	HA / failover	Read scaling, DR
Replication	Synchronous	Asynchronous
Endpoint	Same	Separate read endpoint
Cross-region	No	Yes
Promotion	Automatic	Manual

```
aws rds create-db-instance-read-replica \
  --db-instance-identifier mydb-replica-1 \
  --source-db-instance-identifier mydb \
  --db-instance-class db.t3.medium
```

1.4 24.4 Amazon Aurora

1.4.1 Aurora architecture

Unlike RDS (local storage per instance), Aurora uses a **distributed, shared storage layer** across 3 AZs:



1.4.2 Aurora advantages over RDS

Feature	Aurora	RDS
Storage scaling	Automatic (10 GB-128 TB)	Manual (max 64 TB)
Read replicas	Up to 15, < 10ms lag	Up to 5
Failover	~30 seconds	60-120 seconds
Storage copies	6 across 3 AZs	2 (Multi-AZ)

Serverless v2	Yes (auto-scaling ACUs)	No
Global Database	< 1 second cross-region	Manual replica setup
Backtrack (MySQL)	Rewind to point in time	No

1.4.3 Aurora Serverless v2

Automatically scales compute capacity (ACUs) based on demand:

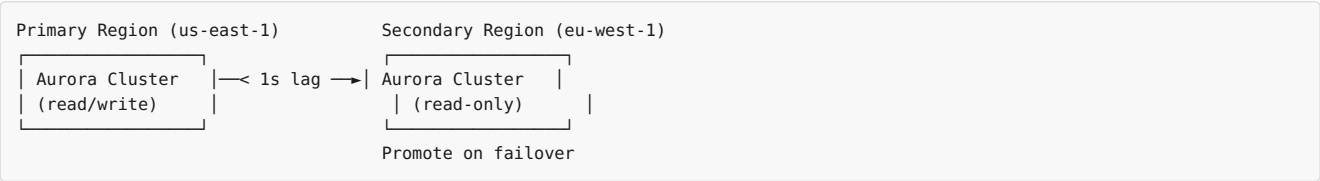
```
resource "aws_rds_cluster" "aurora" {
  engine          = "aurora-postgresql"
  engine_mode     = "provisioned"
  engine_version  = "15.4"
  database_name   = "appdb"
  master_username = "admin"
  master_password = var.db_password

  serverlessv2_scaling_configuration {
    min_capacity = 0.5 # ACUs
    max_capacity = 16
  }
}

resource "aws_rds_cluster_instance" "aurora" {
  cluster_identifier = aws_rds_cluster.aurora.id
  instance_class     = "db.serverless"
  engine             = aws_rds_cluster.aurora.engine
}
```

1.4.4 Aurora Global Database

Cross-region replication with < 1 second lag:



1.5 24.5 Backups and recovery

Feature	RDS	Aurora
Automated backups	1-35 days retention	1-35 days retention
Manual snapshots	Yes	Yes (cluster snapshot)
Point-in-time recovery	5-minute granularity	5-minute granularity
Backtrack (MySQL)	No	Rewind without restore

```
# Restore to point in time
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier mydb \
  --target-db-instance-identifier mydb-restored \
  --restore-time 2024-06-15T10:30:00Z
```

1.6 24.6 Security

--	--

Control	Implementation
Network	Deploy in private subnets; security groups
Encryption at rest	KMS (enabled by default on new instances)
Encryption in transit	SSL/TLS connections (force via parameter group)
IAM authentication	Token-based auth (no password in connection string)
Secrets Manager	Auto-rotate database credentials
Audit	PostgreSQL log exports, MySQL audit plugin, CloudTrail

1.6.1 IAM database authentication

```
import boto3

rds = boto3.client("rds")
token = rds.generate_db_auth_token(
    DBHostname="mydb.abc123.us-east-1.rds.amazonaws.com",
    Port=5432,
    DBUsername="iam_user",
    Region="us-east-1",
)
# Use token as password in connection string (valid 15 minutes)
```

1.7 24.7 Parameter and option groups

Group	Purpose
Parameter group	Engine configuration (max_connections, shared_buffers)
Option group	Optional features (Oracle OEM, SQL Server SSIS)

```
aws rds modify-db-parameter-group \
  --db-parameter-group-name custom-pg \
  --parameters "ParameterName=max_connections,ParameterValue=200,ApplyMethod=immediate"
```

1.8 24.8 Monitoring

Tool	Metrics
CloudWatch	CPU, connections, IOPS, replica lag, free storage
Performance Insights	Top SQL, wait events, database load
Enhanced Monitoring	OS-level metrics (1-second granularity)
Event subscriptions	SNS notifications for failover, maintenance

Key alarms: - CPUUtilization > 80% - FreeableMemory < 256 MB - DatabaseConnections approaching max_connections - ReplicaLag > 30 seconds

1.9 24.9 Terraform example

```
resource "aws_db_subnet_group" "main" {
  name      = "main"
```

```

    subnet_ids = aws_subnet.private[*].id
}

resource "aws_rds_cluster" "aurora" {
  cluster_identifier = "app-aurora"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  database_name      = "appdb"
  master_username    = "admin"
  manage_master_user_password = true # Secrets Manager

  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  backup_retention_period = 14
  preferred_backup_window = "03:00-04:00"
  storage_encrypted       = true
  kms_key_id              = aws_kms_key.rds.arn

  enabled_cloudwatch_logs_exports = ["postgresql"]

  tags = { Name = "app-aurora" }
}

resource "aws_rds_cluster_instance" "aurora" {
  count          = 2
  identifier     = "app-aurora-${count.index}"
  cluster_identifier = aws_rds_cluster.aurora.id
  instance_class = "db.r6g.large"
  engine         = aws_rds_cluster.aurora.engine
  publicly_accessible = false
}

```

1.10 24.10 Chapter summary

- **RDS** provides managed relational databases; **Aurora** offers superior performance and scaling.
 - **Multi-AZ** for HA; **read replicas** for read scaling and cross-region DR.
 - **Aurora Serverless v2** auto-scales compute; **Global Database** for multi-region.
 - Encrypt at rest and in transit; use **IAM authentication** and **Secrets Manager**.
 - Monitor with **Performance Insights** and CloudWatch alarms.
-

1.11 Lab 24.1 — Aurora cluster

1. Create an Aurora PostgreSQL cluster with one writer and one reader.
2. Connect via psql from a private EC2 instance.
3. Insert test data and query from the reader endpoint.
4. Simulate failover and verify application reconnects.

1.12 Lab 24.2 — Point-in-time recovery

1. Enable automated backups (7-day retention).
 2. Create and delete a test table.
 3. Restore to a point before the deletion.
 4. Verify the table exists in the restored instance.
-

1.13 Review questions

1. What is the difference between Multi-AZ and read replicas?
2. How does Aurora storage differ from RDS storage?

3. What is an ACU in Aurora Serverless v2?
 4. How does Aurora Global Database achieve sub-second cross-region replication?
 5. Why should databases be deployed in private subnets?
-

Next: [*Chapter 25 — DynamoDB*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 25

1 Chapter 25: DynamoDB — Tables, Indexes, and Streams

1.1 25.1 What is DynamoDB?

Amazon DynamoDB is a fully managed, serverless NoSQL database providing single-digit millisecond performance at any scale. Unlike RDS, there are no servers to provision, patch, or scale—DynamoDB handles capacity automatically (on-demand mode) or lets you provision read/write capacity units.

DynamoDB excels at key-value and document data models with predictable access patterns.

1.2 25.2 Core concepts

Concept	Description
Table	Collection of items (like a table without schema enforcement)
Item	A record (up to 400 KB)
Attribute	A field within an item (name-value pair)
Primary key	Uniquely identifies each item
Partition key	Hash key (required)
Sort key	Range key (optional; enables queries on range)

1.2.1 Primary key types

Type	Structure	Example
Simple	Partition key only	UserId
Composite	Partition key + sort key	UserId + OrderDate

1.3 25.3 Capacity modes

Mode	Billing	Use case
On-demand	Per request	Unpredictable traffic, new apps
Provisioned	Per RCU/WCU per hour	Predictable, cost-optimized at scale
Provisioned + Auto Scaling	Provisioned with automatic adjustment	Variable but bounded traffic

1.3.1 Capacity units

Unit	Definition
RCU (Read Capacity Unit)	1 strongly consistent read/sec for 4 KB item
WCU (Write Capacity Unit)	1 write/sec for 1 KB item

Eventually consistent reads cost half the RCUs of strongly consistent reads.

1.4 25.4 Indexes

1.4.1 Global Secondary Index (GSI)

- Different partition key and sort key from the base table.
- Eventually consistent (by default).
- Separate provisioned or on-demand capacity.
- Up to 20 GSIs per table.

Base table: PK=UserId, SK=OrderDate
 GSI: PK=ProductId, SK=OrderDate → Query orders by product

1.4.2 Local Secondary Index (LSI)

- Same partition key, different sort key.
- Must be created at table creation time.
- Shares table capacity.
- Up to 5 LSIs per table.

1.4.3 When to use which

Need	Index type
Query by different partition key	GSI
Query by different sort key (same partition)	LSI
Add index after table creation	GSI only
Strongly consistent reads on index	LSI only

1.5 25.5 Query and Scan

1.5.1 Query (efficient)

Access items by partition key (and optionally sort key condition):

```
import boto3

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table("Orders")

response = table.query(
    KeyConditionExpression="UserId = :uid AND OrderDate > :date",
    ExpressionAttributeValues={
        ":uid": "user-123",
        ":date": "2024-01-01",
    },
)
```

1.5.2 Scan (expensive)

Reads every item in the table. Avoid in production; use GSIs instead.

1.5.3 Best practices

- Design access patterns first, then schema.
- Use composite keys to model one-to-many relationships.
- Prefer Query over Scan.
- Use `ProjectionExpression` to return only needed attributes.
- Paginate with `LastEvaluatedKey`.

1.6 25.6 DynamoDB Streams

Streams capture item-level changes (insert, update, delete) in near real-time:

View type	Data captured
KEYS_ONLY	Partition and sort key only
NEW_IMAGE	Entire item after change
OLD_IMAGE	Entire item before change
NEW_AND_OLD_IMAGES	Both (most common)

1.6.1 Stream consumers

Consumer	Use case
Lambda	Event-driven processing, CDC
Kinesis Data Streams	Extended retention, fan-out
DynamoDB (Kinesis adapter)	Legacy integration

1.6.2 Lambda trigger example

```
resource "aws_lambda_event_source_mapping" "dynamo" {
  event_source_arn = aws_dynamodb_table.orders.stream_arn
  function_name    = aws_lambda_function.processor.arn
  starting_position = "LATEST"
  batch_size       = 100

  filter_criteria {
    filter {
      pattern = jsonencode({
        eventName = ["INSERT", "MODIFY"]
      })
    }
  }
}
```

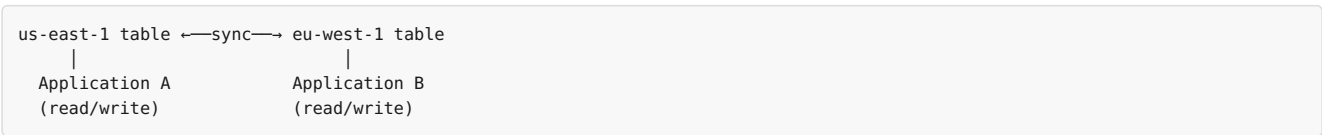
1.7 25.7 Advanced features

1.7.1 DynamoDB Accelerator (DAX)

In-memory cache for microsecond read latency. Transparent to application (drop-in replacement for DynamoDB endpoint).

1.7.2 Global Tables

Multi-region, multi-active replication:



- Conflict resolution: last writer wins.
- RPO: sub-second; RTO: near-zero (DNS failover).

1.7.3 Transactions

`TransactWriteItems` and `TransactGetItems` for ACID operations across up to 100 items (same or different tables).

1.7.4 TTL (Time to Live)

Automatically delete items after a specified timestamp attribute. No WCU consumed for TTL deletions.

```
aws dynamodb update-time-to-live \
  --table-name Sessions \
  --time-to-live-specification "Enabled=true,AttributeName=ExpiresAt"
```

1.7.5 Point-in-time recovery (PITR)

Continuous backup with per-second restore capability (35-day window).

1.8 25.8 Single-table design

DynamoDB best practice: model multiple entity types in one table using composite keys:

PK	SK	Attributes
USER#123	PROFILE	name, email
USER#123	ORDER#2024-001	total, status
USER#123	ORDER#2024-002	total, status
PRODUCT#ABC	METADATA	name, price
PRODUCT#ABC	REVIEW#001	rating, text

GSI for alternate access patterns:

```
GSI1PK=PRODUCT#ABC GSI1SK=REVIEW#001 → All reviews for a product
```

1.9 25.9 Terraform example

```
resource "aws_dynamodb_table" "orders" {
  name           = "Orders"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "UserId"
  range_key      = "OrderDate"

  attribute {
    name = "UserId"
    type = "S"
  }
  attribute {
    name = "OrderDate"
    type = "S"
  }
  attribute {
    name = "ProductId"
    type = "S"
  }
}
```

```

}

global_secondary_index {
  name      = "ProductIndex"
  hash_key  = "ProductId"
  range_key = "OrderDate"
  projection_type = "ALL"
}

stream_enabled   = true
stream_view_type = "NEW_AND_OLD_IMAGES"

point_in_time_recovery { enabled = true }
server_side_encryption { enabled = true }

ttl {
  attribute_name = "ExpiresAt"
  enabled        = true
}

tags = { Name = "orders-table" }
}

```

1.10 25.10 Monitoring

Metric	Alarm threshold idea
ConsumedReadCapacityUnits	Approaching provisioned limit
ConsumedWriteCapacityUnits	Approaching provisioned limit
ThrottledRequests	Any throttling → scale capacity
SuccessfulRequestLatency	p99 > 10ms
UserErrors	4xx errors from client

Enable **Contributor Insights** for per-item access pattern analysis.

1.11 25.11 Chapter summary

- **DynamoDB** is serverless NoSQL with single-digit ms latency at any scale.
- Design **access patterns first**; use composite keys and **GSIs** for query flexibility.
- **Streams** enable event-driven architectures and change data capture.
- **Global Tables** provide multi-region active-active replication.
- Prefer **on-demand** for unpredictable workloads; **single-table design** for complex models.

1.12 Lab 25.1 — Orders table

1. Create a DynamoDB table with composite key (UserId + OrderDate).
2. Add a GSI on ProductId.
3. Insert 20 orders across 5 users and 3 products.
4. Query orders for one user; query orders for one product via GSI.

1.13 Lab 25.2 — Streams + Lambda

1. Enable DynamoDB Streams on your table.
2. Create a Lambda function triggered by stream events.

3. Insert/update items and verify Lambda processes changes.
 4. Add a filter for INSERT events only.
-

1.14 Review questions

1. What is the difference between a GSI and an LSI?
 2. Why should you avoid Scan operations in production?
 3. How do DynamoDB Streams enable change data capture?
 4. What is single-table design and why is it recommended?
 5. When would you choose provisioned capacity over on-demand?
-

Next: [Chapter 26 — ElastiCache](#)

AWS Cloud Handbook — Complete Edition —

Chapter 26

1 Chapter 26: ElastiCache & Caching Patterns

1.1 26.1 Why caching?

Every database query that hits disk (or even memory-optimized databases) adds latency and load. **Caching** stores frequently accessed data in a fast, in-memory layer, reducing database pressure and improving response times from milliseconds to microseconds.

Amazon ElastiCache provides fully managed in-memory caching with two engines: **Redis** and **Memcached**.

1.2 26.2 Redis vs Memcached

Feature	Redis	Memcached
Data structures	Strings, lists, sets, sorted sets, hashes, streams, bitmaps	Strings only (key-value)
Persistence	RDB snapshots, AOF	None (pure cache)
Replication	Primary + up to 5 replicas	None (client-side sharding)
Multi-AZ	Automatic failover	No
Clustering	Cluster mode (sharding)	Client-side consistent hashing
Pub/Sub	Yes	No
Lua scripting	Yes	No
Backup/restore	Yes	No
Use case	Sessions, leaderboards, real-time analytics, queues	Simple object caching

Recommendation: Choose **Redis** unless you need only simple key-value caching with horizontal scaling via client-side sharding.

1.3 26.3 ElastiCache deployment options

Option	Description
Single node	Dev/test only; no HA
Multi-AZ with replica	Primary + replica; automatic failover
Cluster mode disabled	One shard; up to 5 read replicas
Cluster mode enabled	Multiple shards; horizontal scaling

Serverless	Auto-scaling memory; pay per use
------------	----------------------------------

1.3.1 Node types

Family	Use case
cache.t3/t4g	Dev/test, burstable
cache.m6g/m7g	General purpose
cache.r6g/r7g	Memory-optimized, large datasets

1.4 26.4 Common caching patterns

1.4.1 Cache-aside (lazy loading)

Application checks cache first; on miss, reads from database and populates cache:

1. App → Cache: GET key
2. Cache miss → App → DB: SELECT
3. App → Cache: SET key (with TTL)
4. App returns data

```
import redis
import json

cache = redis.Redis(host="my-cluster.abc123.cache.amazonaws.com", port=6379)

def get_user(user_id):
    cached = cache.get(f"user:{user_id}")
    if cached:
        return json.loads(cached)

    user = db.query("SELECT * FROM users WHERE id = %s", user_id)
    cache.setex(f"user:{user_id}", 3600, json.dumps(user)) # TTL 1 hour
    return user
```

1.4.2 Write-through

Application writes to cache and database simultaneously:

- App → Cache: SET key
- App → DB: INSERT/UPDATE

Ensures cache is always up to date but adds write latency.

1.4.3 Write-behind (write-back)

Application writes to cache; cache asynchronously writes to database:

- App → Cache: SET key
- Cache → DB: async write (batched)

Highest write performance but risk of data loss if cache fails before DB write.

1.4.4 Read-through

Cache itself loads data from database on miss (requires cache library support):

- App → Cache: GET key
- Cache miss → Cache → DB: load and return

1.5 26.5 Redis data structure use cases

Structure	Use case	Example
String	Simple cache, counters	Session token, page view count
Hash	Object storage	User profile fields
List	Queues, timelines	Recent activity feed
Set	Unique items	Online users, tags
Sorted Set	Leaderboards, rankings	Game scores, trending
Stream	Event log	Activity stream, messaging
Bitmap	Feature flags, analytics	Daily active users
HyperLogLog	Cardinality estimation	Unique visitor count

1.5.1 Leaderboard example

```
# Add score
cache.zadd("leaderboard:2024", {"player-1": 1500, "player-2": 2300})

# Top 10
top_10 = cache.zrevrange("leaderboard:2024", 0, 9, withscores=True)

# Player rank
rank = cache.zrevrank("leaderboard:2024", "player-1")
```

1.6 26.6 Session management

Redis is the standard for distributed session storage:

```
import uuid

def create_session(user_id):
    session_id = str(uuid.uuid4())
    cache.setex(f"session:{session_id}", 86400, json.dumps({
        "user_id": user_id,
        "created": time.time(),
    }))
    return session_id

def get_session(session_id):
    data = cache.get(f"session:{session_id}")
    return json.loads(data) if data else None
```

Benefits over sticky sessions: - Any app server can handle any request. - Sessions survive server restarts. - Easy session invalidation (delete key).

1.7 26.7 ElastiCache security

Control	Implementation
Network	Deploy in private subnets; security groups
Encryption in transit	TLS (required for compliance)

Encryption at rest	KMS
AUTH token	Redis password (required with encryption in transit)
RBAC	Redis ACLs for fine-grained access (Redis 6+)

```
resource "aws_elasticache_replication_group" "redis" {
  replication_group_id      = "app-cache"
  description               = "Application Redis cluster"
  node_type                 = "cache.r6g.large"
  num_cache_clusters        = 2
  automatic_failover_enabled = true
  multi_az_enabled          = true

  engine           = "redis"
  engine_version   = "7.0"
  port             = 6379
  parameter_group_name = "default.redis7"

  subnet_group_name = aws_elasticache_subnet_group.main.name
  security_group_ids = [aws_security_group.redis.id]

  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  auth_token                 = var.redis_auth_token

  snapshot_retention_limit = 7
  snapshot_window          = "03:00-05:00"
}
```

1.8 26.8 Monitoring and maintenance

Metric	Action
CPUUtilization	Scale up node type if > 75%
DatabaseMemoryUsagePercentage	Scale up or evict; risk of OOM
CacheHitRate	Low hit rate → review TTLs and key design
Evictions	Memory pressure; scale up or reduce TTLs
ReplicationLag	Replica falling behind; check network/load

1.8.1 Maintenance windows

ElastiCache applies engine patches during maintenance windows. Enable **auto minor version upgrade** for security patches.

1.9 26.9 Caching anti-patterns

Anti-pattern	Problem	Solution
Cache everything	Memory waste, stale data	Cache only hot data with TTLs
No TTL	Stale data forever	Always set expiration
Cache stampede	Many requests hit DB on simultaneous expiry	Jittered TTLs, request coalescing
Large objects	Memory pressure, slow serialization	Compress or split large values
Caching errors	Error responses cached	Don't cache 4xx/5xx responses

No cache invalidation strategy	Stale data after updates	Event-driven invalidation via streams/SNS
--------------------------------	--------------------------	---

1.10 26.10 Chapter summary

- **ElastiCache** provides managed Redis and Memcached for in-memory caching.
- **Redis** supports rich data structures, persistence, replication, and pub/sub.
- **Cache-aside** is the most common pattern; always set **TTLs**.
- Use Redis for sessions, leaderboards, and real-time features.
- Deploy in **private subnets** with encryption in transit and at rest.

1.11 Lab 26.1 — Redis cache-aside

1. Create an ElastiCache Redis cluster (single node for lab).
2. Write a Python app that implements cache-aside for a DynamoDB table.
3. Measure response time with and without cache.
4. Verify cache hit rate in CloudWatch.

1.12 Lab 26.2 — Leaderboard

1. Use Redis sorted sets to build a game leaderboard.
2. Add scores for 20 players.
3. Query top 10 and individual player rank.
4. Implement a “scores in last hour” feature using TTL keys.

1.13 Review questions

1. When would you choose Memcached over Redis?
2. How does the cache-aside pattern handle cache misses?
3. What is cache stampede and how do you prevent it?
4. Why is Redis preferred for session management over sticky sessions?
5. What CloudWatch metric indicates your cache is too small?

Next: [Chapter 27 — DMS & DR](#)

AWS Cloud Handbook — Complete Edition —

Chapter 27

1 Chapter 27: Database Migration (DMS) & DR

1.1 27.1 Database migration challenges

Migrating databases to AWS—whether from on-premises, another cloud, or between AWS engines—is one of the highest-risk activities in cloud adoption. **AWS Database Migration Service (DMS)** simplifies heterogeneous and homogeneous migrations with minimal downtime.

Combined with **Schema Conversion Tool (SCT)** for schema translation and **native replication features** for ongoing DR, AWS provides a complete migration toolkit.

1.2 27.2 AWS DMS overview

DMS replicates data between source and target databases continuously:

Migration type	Description
Full load	One-time copy of all data
Full load + CDC	Initial copy + ongoing change data capture
CDC only	Ongoing replication (source already loaded)

1.2.1 Supported sources and targets

Source	Target
On-premises Oracle, SQL Server, MySQL, PostgreSQL, MongoDB	RDS, Aurora, Redshift, S3, DynamoDB, Kinesis
EC2-hosted databases	Same as above
RDS / Aurora	Cross-engine (e.g., Oracle → PostgreSQL)
S3	RDS, Redshift

1.2.2 DMS components

Component	Description
Replication instance	EC2 instance running DMS engine
Source endpoint	Connection to source database
Target endpoint	Connection to target database
Replication task	Defines what to migrate and how
Table mappings	Rules for schema selection and transformation

1.3 27.3 Migration workflow

1. Assess → SCT analyzes source schema; identifies incompatibilities
2. Convert → SCT converts schema to target engine (if heterogeneous)
3. Provision → Create target RDS/Aurora instance
4. Replicate → DMS full load + CDC
5. Cutover → Switch application to target; stop DMS task
6. Validate → Data validation and application testing

1.3.1 Heterogeneous vs homogeneous

Type	Example	SCT needed?
Homogeneous	MySQL → Aurora MySQL	No
Heterogeneous	Oracle → Aurora PostgreSQL	Yes
Homogeneous	SQL Server → RDS SQL Server	No
Heterogeneous	MongoDB → DynamoDB	Partial

1.4 27.4 DMS replication instance

Size	vCPU	Memory	Use case
dms.t3.medium	2	4 GB	Small databases (< 100 GB)
dms.r5.large	2	16 GB	Medium workloads
dms.r5.4xlarge	16	128 GB	Large, high-throughput migrations
dms.c5.9xlarge	36	72 GB	Maximum throughput

Multi-AZ replication instances provide automatic failover for ongoing CDC tasks.

1.4.1 Terraform example

```
resource "aws_dms_replication_subnet_group" "main" {
  replication_subnet_group_id      = "dms-subnet-group"
  replication_subnet_group_description = "DMS subnet group"
  subnet_ids                       = aws_subnet.private[*].id
}

resource "aws_dms_replication_instance" "main" {
  replication_instance_id      = "dms-replication"
  replication_instance_class    = "dms.r5.large"
  allocated_storage             = 100
  multi_az                     = true
  replication_subnet_group_id   = aws_dms_replication_subnet_group.main.id
  vpc_security_group_ids       = [aws_security_group.dms.id]
}

resource "aws_dms_endpoint" "source" {
  endpoint_id      = "source-oracle"
  endpoint_type    = "source"
  engine_name      = "oracle"
  server_name      = "on-prem-oracle.example.com"
  port             = 1521
  database_name    = "ORCL"
  username         = "dms_user"
  password         = var.source_password
}

resource "aws_dms_endpoint" "target" {
  endpoint_id      = "target-aurora-pg"
  endpoint_type    = "target"
  engine_name      = "aurora-postgresql"
}
```

```

server_name = aws_rds_cluster.aurora.endpoint
port        = 5432
database_name = "appdb"
username     = "admin"
password     = var.target_password
}

resource "aws_dms_replication_task" "migration" {
  replication_task_id      = "oracle-to-aurora-pg"
  migration_type           = "full-load-and-cdc"
  replication_instance_arn = aws_dms_replication_instance.main.replication_instance_arn
  source_endpoint_arn      = aws_dms_endpoint.source.endpoint_arn
  target_endpoint_arn      = aws_dms_endpoint.target.endpoint_arn

  table_mappings = file("${path.module}/table-mappings.json")

  replication_task_settings = jsonencode({
    TargetMetadata = { TargetSchema = "public" }
    FullLoadSettings = { TargetTablePrepMode = "DROP_AND_CREATE" }
    ChangeProcessingDdlHandlingPolicy = {
      HandleSourceTableDropped = true
      HandleSourceTableTruncated = true
    }
  })
}

```

1.5 27.5 Schema Conversion Tool (SCT)

SCT automates schema conversion for heterogeneous migrations:

1. Connect to source database.
2. Analyze schema objects (tables, views, stored procedures, triggers).
3. Convert to target engine syntax.
4. Report incompatible objects for manual review.
5. Apply converted schema to target.

1.5.1 Common conversion challenges

Source (Oracle)	Target (PostgreSQL)	SCT action
VARCHAR2	VARCHAR	Auto-convert
NUMBER	NUMERIC	Auto-convert
PL/SQL procedures	PL/pgSQL	Partial auto-convert
Packages	Schemas + functions	Manual review
Sequences	SERIAL / sequences	Auto-convert

1.6 27.6 Migration strategies

1.6.1 Big bang

Migrate everything in one maintenance window:

- **Pros:** Simple, clean cutover.
- **Cons:** Extended downtime; high risk.
- **When:** Small databases, tolerant downtime windows.

1.6.2 Trickle (parallel run)

DMS replicates continuously; application reads from both during transition:

Phase 1: DMS full load + CDC running
 Phase 2: Application writes to source; reads from source
 Phase 3: Application writes to source; reads from target (validate)
 Phase 4: Application writes to target; reads from target
 Phase 5: Decommission source

- **Pros:** Minimal downtime (minutes for cutover).
- **Cons:** Complex application changes; dual-write period.

1.6.3 Zero-ETL integrations

AWS-native integrations that eliminate DMS for supported paths:

Integration	Source → Target
Aurora zero-ETL to Redshift	Aurora → Redshift analytics
S3 zero-ETL to Redshift	S3 → Redshift
OpenSearch zero-ETL	Aurora/DMS → OpenSearch

1.7 27.7 Data validation

DMS provides built-in data validation:

```
{
  "ValidationSettings": {
    "EnableValidation": true,
    "ValidationMode": "ROW_LEVEL",
    "ThreadCount": 5,
    "FailureMaxCount": 10000
  }
}
```

Additionally: - Row count comparison between source and target. - Checksum validation for critical tables. - Application-level integration testing.

1.8 27.8 Ongoing replication for DR

DMS is not just for migration—it enables ongoing replication for DR:

On-premises Oracle —CDC→ Aurora PostgreSQL (DR region)
 |
 Promote on failover

Feature	Benefit
CDC	Near-real-time replication
Multi-AZ instance	DMS failover protection
Table filtering	Replicate only critical tables
Transformation rules	Mask PII in DR copy

For AWS-native DR, prefer **Aurora Global Database** or **RDS cross-region read replicas** over DMS when source is already on AWS.

1.9 27.9 Troubleshooting

Issue	Cause	Fix
Full load slow	Undersized replication instance	Scale up instance class
CDC lag increasing	Network or target write bottleneck	Scale instance; optimize target
LOB columns failing	LOB settings too small	Increase <code>LobMaxSize</code> in task settings
FK constraint errors	Load order wrong	Disable FK checks during load
Connection timeout	Security group or network ACL	Verify SG rules and route tables

```
aws dms describe-replication-tasks \
  --filters Name=replication-task-id,Values=oracle-to-aurora-pg

aws dms describe-table-statistics \
  --replication-task-arn arn:aws:dms:us-east-1:123456789012:task:abc123
```

1.10 27.10 Chapter summary

- **DMS** migrates databases with full load and ongoing CDC for minimal downtime.
- **SCT** converts schemas for heterogeneous migrations (Oracle → PostgreSQL, etc.).
- Use **trickle migration** for near-zero-downtime cutovers.
- DMS also supports **ongoing replication** for DR scenarios.
- Validate data thoroughly before and after cutover.

1.11 Lab 27.1 — Homogeneous migration

1. Create a source MySQL RDS instance with sample data.
2. Create a target Aurora MySQL cluster.
3. Configure DMS endpoints and a full-load + CDC replication task.
4. Verify data replication; insert new rows on source and confirm CDC delivery.

1.12 Lab 27.2 — Migration assessment

1. Install SCT on an EC2 instance.
2. Connect to a sample database and run an assessment report.
3. Review conversion warnings and manual action items.
4. Document a migration plan with estimated downtime.

1.13 Review questions

1. What is the difference between full load and CDC in DMS?
2. When is SCT required for a database migration?
3. What is trickle migration and how does it minimize downtime?
4. How does DMS support ongoing DR replication?
5. What factors determine the size of a DMS replication instance?

Next: [Chapter 28 — API Gateway](#)

AWS Cloud Handbook — Complete Edition —

Chapter 28

1 Chapter 28: API Gateway

1.1 28.1 What is Amazon API Gateway?

Amazon API Gateway is a fully managed service for creating, publishing, securing, and monitoring HTTP, REST, and WebSocket APIs at any scale. It acts as the front door for serverless backends (Lambda), container services (ECS/EKS), and legacy applications.

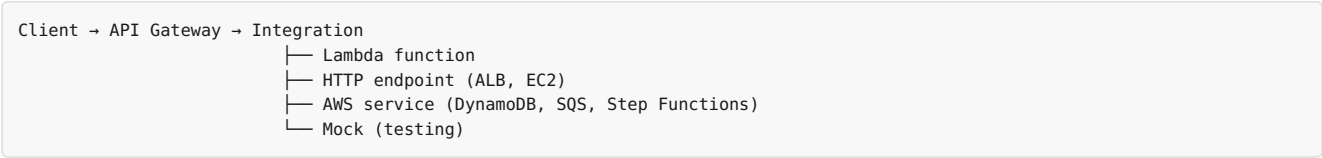
API Gateway handles request routing, authentication, throttling, caching, and API versioning—freeing backend services from these cross-cutting concerns.

1.2 28.2 API types

Type	Protocol	Use case
REST API	HTTP/REST	Full-featured; request/response transformation
HTTP API	HTTP	Simpler, cheaper, lower latency; Lambda/proxy integrations
WebSocket API	WebSocket	Real-time bidirectional communication
Private REST API	HTTP/REST	Internal APIs via VPC endpoint

Recommendation: Use **HTTP API** for new Lambda-based APIs (70% cheaper, lower latency). Use **REST API** when you need request validation, API keys, or advanced transformation.

1.3 28.3 REST API architecture



1.3.1 Key components

Component	Description
Resource	URL path segment (/users , /users/{id})
Method	HTTP verb (GET, POST, PUT, DELETE)
Integration	Backend that processes the request
Stage	Deployment environment (dev, staging, prod)
Deployment	Snapshot of API configuration

1.4 28.4 HTTP API with Lambda

1.4.1 Terraform example

```
resource "aws_apigatewayv2_api" "api" {
  name           = "app-api"
  protocol_type  = "HTTP"

  cors_configuration {
    allow_origins = ["https://app.example.com"]
    allow_methods = ["GET", "POST", "PUT", "DELETE"]
    allow_headers = ["Content-Type", "Authorization"]
    max_age      = 3600
  }
}

resource "aws_apigatewayv2_integration" "lambda" {
  api_id           = aws_apigatewayv2_api.api.id
  integration_type = "AWS_PROXY"
  integration_uri  = aws_lambda_function.api.invoke_arn
  payload_format_version = "2.0"
}

resource "aws_apigatewayv2_route" "get_users" {
  api_id = aws_apigatewayv2_api.api.id
  route_key = "GET /users"
  target = "integrations/${aws_apigatewayv2_integration.lambda.id}"
}

resource "aws_apigatewayv2_route" "post_users" {
  api_id = aws_apigatewayv2_api.api.id
  route_key = "POST /users"
  target = "integrations/${aws_apigatewayv2_integration.lambda.id}"
}

resource "aws_apigatewayv2_stage" "prod" {
  api_id = aws_apigatewayv2_api.api.id
  name = "prod"
  auto_deploy = true

  access_log_settings {
    destination_arn = aws_cloudwatch_log_group.api.arn
    format = jsonencode({
      requestId = "$context.requestId"
      ip = "$context.identity.sourceIp"
      method = "$context.httpMethod"
      path = "$context.path"
      status = "$context.status"
      responseTime = "$context.responseLatency"
    })
  }

  default_route_settings {
    throttling_burst_limit = 100
    throttling_rate_limit = 50
  }
}

resource "aws_lambda_permission" "api" {
  statement_id = "AllowAPIGateway"
  action = "lambda:InvokeFunction"
  function_name = aws_lambda_function.api.function_name
  principal = "apigateway.amazonaws.com"
  source_arn = "${aws_apigatewayv2_api.api.execution_arn}/*/*"
}
```

1.5 28.5 Authentication and authorization

Method	Description
--------	-------------

IAM	SigV4 signing; service-to-service
Cognito User Pools	JWT tokens for user-facing APIs
Lambda authorizer	Custom auth logic (any token format)
JWT authorizer (HTTP API)	Validate JWT from any OIDC provider
API keys	Simple key-based access (REST API only)
Resource policy	IP-based or VPC-based restrictions

1.5.1 Lambda authorizer example

```
def handler(event, context):
    token = event["authorizationToken"]
    if not validate_token(token):
        raise Exception("Unauthorized") # Returns 401

    return {
        "principalId": "user-123",
        "policyDocument": {
            "Version": "2012-10-17",
            "Statement": [{
                "Action": "execute-api:Invoke",
                "Effect": "Allow",
                "Resource": event["methodArn"],
            }],
        },
        "context": {
            "userId": "user-123",
            "role": "admin",
        },
    }
```

1.6 28.6 Throttling and quotas

API Gateway protects backends with built-in throttling:

Level	Default
Account	10,000 requests/second (can increase)
Per API/stage	Configurable burst and rate limits
Per method	Usage plans with API keys

1.6.1 Usage plans

```
resource "aws_api_gateway_usage_plan" "basic" {
    name = "basic-plan"

    api_stages {
        api_id = aws_api_gateway_rest_api.api.id
        stage = aws_api_gateway_stage.prod.stage_name
    }

    throttle_settings {
        burst_limit = 50
        rate_limit = 25
    }

    quota_settings {
        limit = 10000
        period = "MONTH"
    }
}
```

1.7 28.7 Request/response transformation

REST API supports **mapping templates** (VTL) for request/response transformation:

```
Client sends JSON → API Gateway transforms → DynamoDB PutItem format
DynamoDB response → API Gateway transforms → Client JSON response
```

HTTP API passes the request directly to Lambda (proxy integration) with less transformation capability but better performance.

1.7.1 Request validation (REST API)

```
{
  "type": "object",
  "required": ["name", "email"],
  "properties": {
    "name": { "type": "string", "minLength": 1 },
    "email": { "type": "string", "format": "email" }
  }
}
```

1.8 28.8 Caching

API Gateway can cache responses at the stage level:

Setting	Value
TTL	0–3600 seconds
Cache key	Query parameters, headers
Encryption	In-transit and at-rest
Cost	Per-hour cache capacity

Enable caching for read-heavy endpoints with infrequently changing data. Invalidate cache on deployments.

1.9 28.9 Custom domains and TLS

```
resource "aws_apigatewayv2_domain_name" "api" {
  domain_name = "api.example.com"

  domain_name_configuration {
    certificate_arn = aws_acm_certificate.api.arn
    endpoint_type   = "REGIONAL"
    security_policy = "TLS_1_2"
  }
}

resource "aws_apigatewayv2_api_mapping" "api" {
  api_id      = aws_apigatewayv2_api.api.id
  domain_name = aws_apigatewayv2_domain_name.api.id
  stage       = aws_apigatewayv2_stage.prod.id
}
```

Route 53 alias record points `api.example.com` to the API Gateway domain.

1.10 28.10 WebSocket API

For real-time applications (chat, gaming, live dashboards):

```
Client ← WebSocket → API Gateway ← Lambda/DynamoDB
    $connect           Route by route key
    $disconnect        $default, custom routes
    $default
```

```
resource "aws_apigatewayv2_api" "websocket" {
  name           = "chat-api"
  protocol_type  = "WEBSOCKET"
  route_selection_expression = "$request.body.action"
}
```

1.11 28.11 Monitoring

Tool	Data
CloudWatch Metrics	Count, Latency, 4XXError, 5XXError, IntegrationLatency
CloudWatch Logs	Access logs, execution logs
X-Ray	Distributed tracing through API Gateway → Lambda → DynamoDB

Key alarms: - 5XXError > 1% of requests - Latency p99 > 3000ms - Count approaching throttle limits

1.12 28.12 Chapter summary

- **API Gateway** is the managed front door for HTTP, REST, and WebSocket APIs.
- Use **HTTP API** for new Lambda backends; **REST API** for advanced features.
- Secure with **Cognito**, **Lambda authorizers**, or **IAM**.
- Configure **throttling**, **caching**, and **custom domains** for production.
- Monitor with CloudWatch metrics, access logs, and X-Ray tracing.

1.13 Lab 28.1 — HTTP API + Lambda CRUD

1. Create an HTTP API with Lambda integration for a DynamoDB-backed CRUD API.
2. Implement GET /items, POST /items, GET /items/{id}, DELETE /items/{id}.
3. Add a JWT authorizer (Cognito or custom).
4. Configure access logging and test with `curl`.

1.14 Lab 28.2 — Throttling and usage plans

1. Create a REST API with API key requirement.
2. Create a usage plan with 100 requests/day quota.
3. Exceed the quota and verify 429 responses.
4. Review CloudWatch metrics for throttled requests.

1.15 Review questions

1. When should you choose REST API over HTTP API?
2. How does a Lambda authorizer work?
3. What is the difference between burst limit and rate limit?
4. How do you attach a custom domain with TLS to API Gateway?
5. What CloudWatch metric indicates backend integration problems?

Next: [*Chapter 29 — SQS, SNS, and EventBridge*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 29

1 Chapter 29: SQS, SNS, and EventBridge

1.1 29.1 Event-driven architecture on AWS

Modern applications decouple components using asynchronous messaging. AWS provides three core services for event-driven patterns:

Service	Pattern	Delivery
SQS	Message queuing	Pull-based; consumer polls
SNS	Pub/sub fan-out	Push-based; subscribers receive
EventBridge	Event bus routing	Push-based; rules route events

Understanding when to use each—and how to combine them—is fundamental to building scalable, resilient systems.

1.2 29.2 Amazon SQS

Simple Queue Service (SQS) is a fully managed message queue. Producers send messages; consumers poll and process them.

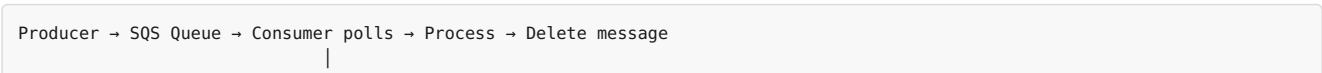
1.2.1 Queue types

Type	Throughput	Delivery	Use case
Standard	Unlimited	At-least-once; best-effort ordering	Most workloads
FIFO	3,000 msg/s (with batching: 30,000)	Exactly-once; strict ordering	Order processing, financial transactions

1.2.2 Key concepts

Concept	Description
Message	Up to 256 KB (larger via S3 pointer)
Visibility timeout	Message hidden after receive; must delete or it reappears
Retention	1 minute to 14 days (default 4 days)
Dead letter queue (DLQ)	Captures messages that fail processing repeatedly
Long polling	Wait up to 20 seconds for messages (reduces empty receives)

1.2.3 Standard queue workflow



(fail after N retries)

▼
DLQ

1.2.4 Terraform

```
resource "aws_sqs_queue" "dlq" {
  name = "orders-dlq"
}

resource "aws_sqs_queue" "orders" {
  name                = "orders-queue"
  visibility_timeout_seconds = 300
  message_retention_seconds = 1209600 # 14 days
  receive_wait_time_seconds = 20      # long polling

  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.dlq.arn
    maxReceiveCount     = 3
  })
}
```

1.2.5 Lambda + SQS integration

```
resource "aws_lambda_event_source_mapping" "sqs" {
  event_source_arn = aws_sqs_queue.orders.arn
  function_name    = aws_lambda_function.processor.arn
  batch_size       = 10
  maximum_batching_window_in_seconds = 5

  scaling_config {
    maximum_concurrency = 50
  }
}
```

1.2.6 CLI

```
aws sqs send-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/123456789012/orders-queue \
  --message-body '{"orderId": "12345", "action": "process"}'

aws sqs receive-message \
  --queue-url https://sqs.us-east-1.amazonaws.com/123456789012/orders-queue \
  --wait-time-seconds 20 \
  --max-number-of-messages 10
```

1.3 29.3 Amazon SNS

Simple Notification Service (SNS) is a pub/sub messaging service. Publishers send messages to a **topic**; subscribers (SQS, Lambda, HTTP, email, SMS) receive copies.

1.3.1 SNS + SQS fan-out pattern

```

      ┌─ SQS Queue A → Lambda (process orders)
Publisher → SNS Topic ┤─ SQS Queue B → Lambda (send email)
      └─ SQS Queue C → Lambda (update analytics)
```

Benefits: - Decouple publisher from consumers. - Each consumer processes at its own pace. - Add/remove consumers without changing the publisher.

```
resource "aws_sns_topic" "orders" {
  name = "order-events"
}
```

```

resource "aws_sns_topic_subscription" "email_queue" {
  topic_arn = aws_sns_topic.orders.arn
  protocol  = "sqs"
  endpoint  = aws_sqs_queue.email.arn
}

resource "aws_sqs_queue_policy" "email" {
  queue_url = aws_sqs_queue.email.id
  policy = jsonencode({
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "sns.amazonaws.com" }
      Action    = "sqs:SendMessage"
      Resource  = aws_sqs_queue.email.arn
      Condition = {
        ArnEquals = { "aws:SourceArn" = aws_sns_topic.orders.arn }
      }
    }]
  })
}

```

1.3.2 Message filtering

SNS subscription filter policies route only matching messages:

```

{
  "eventType": ["order.created", "order.updated"],
  "amount": [{ "numeric": [">", 100] }]
}

```

1.4 29.4 Amazon EventBridge

EventBridge is a serverless event bus that routes events from AWS services, SaaS applications, and custom applications.

1.4.1 Event buses

Bus	Events
Default	All AWS service events in the account
Custom	Application-specific events
Partner	SaaS events (Zendesk, Auth0, etc.)

1.4.2 Rules and targets

Event source → Rule (pattern match) → Target(s)

- ├─ Lambda
- ├─ SQS
- ├─ SNS
- ├─ Step Functions
- ├─ API Gateway
- └─ 20+ other targets

1.4.3 Event pattern example

```

{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": {
    "state": ["stopped", "terminated"]
  }
}

```

1.4.4 Terraform

```
resource "aws_cloudwatch_event_rule" "ec2_state" {
  name       = "ec2-state-change"
  description = "Capture EC2 state changes"

  event_pattern = jsonencode({
    source       = ["aws.ec2"]
    detail-type  = ["EC2 Instance State-change Notification"]
    detail = {
      state = ["stopped", "terminated"]
    }
  })
}

resource "aws_cloudwatch_event_target" "lambda" {
  rule       = aws_cloudwatch_event_rule.ec2_state.name
  target_id  = "notify-slack"
  arn        = aws_lambda_function.notify.arn
}

resource "aws_cloudwatch_event_bus" "app" {
  name = "app-events"
}

resource "aws_cloudwatch_event_rule" "order_created" {
  name       = "order-created"
  event_bus_name = aws_cloudwatch_event_bus.app.name

  event_pattern = jsonencode({
    source       = ["app.orders"]
    detail-type  = ["OrderCreated"]
  })
}
```

1.4.5 Custom events

```
import boto3, json

events = boto3.client("events")
events.put_events(Entries=[{
  "Source": "app.orders",
  "DetailType": "OrderCreated",
  "Detail": json.dumps({"orderId": "12345", "amount": 99.99}),
  "EventBusName": "app-events",
}])
```

1.5 29.5 Choosing the right service

Scenario	Service
Decouple two services; consumer polls	SQS
Fan-out to multiple consumers	SNS (+ SQS for each)
React to AWS service events	EventBridge
Custom application events with routing rules	EventBridge
Guaranteed ordering	SQS FIFO
Exactly-once processing	SQS FIFO
Cross-account event routing	EventBridge
Real-time push to HTTP endpoints	SNS (HTTP subscription)
Scheduled events	EventBridge Scheduler

1.5.1 EventBridge vs SNS

Feature	EventBridge	SNS
Event filtering	Content-based rules	Subscription filter policies
Targets per rule	5 (default)	Many subscribers per topic
Schema registry	Yes	No
Archive and replay	Yes	No
Cross-account	Native	Via topic policy
Delivery	At-least-once	At-least-once

1.6 29.6 Error handling patterns

1.6.1 SQS retry with DLQ

Receive → Process (fail) → Visibility timeout expires → Re-deliver
→ Fail 3 times → Move to DLQ → Alert + manual review

1.6.2 SNS retry

SNS retries failed deliveries to HTTP/Lambda targets with exponential backoff. Configure DLQ on the subscription for persistent failures.

1.6.3 EventBridge retry

EventBridge retries failed target invocations for 24 hours with exponential backoff. Configure DLQ or retry policy per target.

1.7 29.7 Monitoring

Service	Key metrics
SQS	ApproximateNumberOfMessagesVisible , ApproximateAgeOfOldestMessage
SNS	NumberOfMessagesPublished , NumberOfNotificationsFailed
EventBridge	FailedInvocations , Invocations , ThrottledRules

Alarm on DLQ message count > 0 and ApproximateAgeOfOldestMessage > threshold (processing lag).

1.8 29.8 Chapter summary

- **SQS** provides reliable message queuing with standard and FIFO options.
- **SNS** enables pub/sub fan-out to multiple subscribers.
- **EventBridge** routes events from AWS services and custom applications with content-based filtering.
- Combine **SNS + SQS** for fan-out with independent consumer scaling.
- Configure **DLQs** and monitor queue depth and message age.

1.9 Lab 29.1 — Order processing pipeline

1. Create an SNS topic for order events.
2. Subscribe two SQS queues (email notifications, inventory update).
3. Publish order events and verify both queues receive messages.
4. Process messages with Lambda and configure DLQs.

1.10 Lab 29.2 — EventBridge automation

1. Create a rule that triggers on S3 `ObjectCreated` events.
2. Route matching events to a Lambda that processes uploaded files.
3. Create a custom event bus and publish application events.
4. Archive events and replay from a specific timestamp.

1.11 Review questions

1. What is the difference between SQS standard and FIFO queues?
2. How does the SNS + SQS fan-out pattern work?
3. When would you use EventBridge instead of SNS?
4. What is a dead letter queue and when should you configure one?
5. What does the SQS visibility timeout control?

Next: [Chapter 30 — Step Functions](#)

AWS Cloud Handbook — Complete Edition — Chapter 30

1 Chapter 30: Step Functions Workflows

1.1 30.1 What are Step Functions?

AWS Step Functions is a serverless orchestration service that coordinates multiple AWS services into visual workflows. Instead of writing complex retry/error logic in application code, you define workflows as state machines that handle sequencing, parallel execution, error handling, and human approval steps.

Step Functions is ideal for order processing, ETL pipelines, ML workflows, and multi-step business processes.

1.2 30.2 Workflow types

Type	Language	Use case
Standard	Amazon States Language (JSON/YAML)	Long-running (up to 1 year), exactly-once
Express	ASL	High-volume, short-duration (< 5 min), at-least-once

1.2.1 Standard vs Express

Feature	Standard	Express
Duration	Up to 1 year	Up to 5 minutes
Execution model	Exactly-once	At-least-once
Pricing	Per state transition	Per execution + duration
Use case	Business workflows, ETL	IoT, streaming, high-throughput

1.3 30.3 State types

State	Purpose
Task	Execute work (Lambda, ECS, DynamoDB, etc.)
Choice	Branch based on conditions
Parallel	Execute branches concurrently
Map	Iterate over array items (dynamic parallelism)
Wait	Delay for fixed time or until timestamp
Pass	Pass input to output (transform)
Succeed	Terminal success
Fail	Terminal failure

1.4 30.4 Example: Order processing workflow

```
Start → ValidateOrder → Choice (valid?)
├── Yes → ProcessPayment → Choice (paid?)
│   ├── Yes → FulfillOrder → Succeed
│   └── No → NotifyFailure → Fail
└── No → NotifyInvalid → Fail
```

1.4.1 ASL definition

```
{
  "Comment": "Order processing workflow",
  "StartAt": "ValidateOrder",
  "States": {
    "ValidateOrder": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:validate-order",
      "Next": "IsOrderValid",
      "Retry": [{
        "ErrorEquals": ["Lambda.ServiceException", "Lambda.TooManyRequestsException"],
        "IntervalSeconds": 2,
        "MaxAttempts": 3,
        "BackoffRate": 2
      }],
      "Catch": [{
        "ErrorEquals": ["ValidationError"],
        "Next": "NotifyInvalid"
      }]
    },
    "IsOrderValid": {
      "Type": "Choice",
      "Choices": [{
        "Variable": "$.valid",
        "BooleanEquals": true,
        "Next": "ProcessPayment"
      }],
      "Default": "NotifyInvalid"
    },
    "ProcessPayment": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName": "process-payment",
        "Payload.$": "$"
      },
      "ResultPath": "$.payment",
      "Next": "IsPaymentSuccessful"
    },
    "IsPaymentSuccessful": {
      "Type": "Choice",
      "Choices": [{
        "Variable": "$.payment.Payload.success",
        "BooleanEquals": true,
        "Next": "FulfillOrder"
      }],
      "Default": "NotifyFailure"
    },
    "FulfillOrder": {
      "Type": "Task",
      "Resource": "arn:aws:states:::sqs:sendMessage.waitForTaskToken",
      "Parameters": {
        "QueueUrl": "https://sqs.us-east-1.amazonaws.com/123456789012/fulfillment",
        "MessageBody": {
          "orderId.$": "$.orderId",
          "taskToken.$": "$$.Task.Token"
        }
      },
      "End": true
    },
    "NotifyInvalid": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:notify-invalid",
      "Next": "OrderFailed"
    }
  }
}
```

```

    "NotifyFailure": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:function:notify-failure",
      "Next": "OrderFailed"
    },
    "OrderFailed": {
      "Type": "Fail",
      "Error": "OrderProcessingFailed",
      "Cause": "Order could not be processed"
    }
  }
}

```

1.5 30.5 Service integrations

Step Functions integrates with 220+ AWS services via **optimized integrations** (no Lambda wrapper needed):

Integration pattern	Example
Request Response	Invoke Lambda, call DynamoDB GetItem
Run a Job	Submit Batch job, start ECS task, start Glue job
Wait for Callback	Send SQS message with task token; resume when callback received

1.5.1 Direct DynamoDB integration

```

{
  "Type": "Task",
  "Resource": "arn:aws:states:::dynamodb:putItem",
  "Parameters": {
    "TableName": "Orders",
    "Item": {
      "OrderId": { "S.$": "$.orderId" },
      "Status": { "S": "PROCESSING" },
      "Amount": { "N.$": "$.amount" }
    }
  },
  "Next": "ProcessPayment"
}

```

1.5.2 ECS/Fargate task

```

{
  "Type": "Task",
  "Resource": "arn:aws:states:::ecs:runTask.sync",
  "Parameters": {
    "LaunchType": "FARGATE",
    "Cluster": "processing-cluster",
    "TaskDefinition": "data-processor",
    "NetworkConfiguration": {
      "AwsVpcConfiguration": {
        "Subnets": ["subnet-aaa", "subnet-bbb"],
        "SecurityGroups": ["sg-processor"]
      }
    }
  },
  "Next": "CheckResults"
}

```

1.6 30.6 Error handling

1.6.1 Retry

```
"Retry": [{
  "ErrorEquals": ["States.TaskFailed"],
  "IntervalSeconds": 1,
  "MaxAttempts": 3,
  "BackoffRate": 2.0,
  "JitterStrategy": "FULL"
}]
```

1.6.2 Catch

```
"Catch": [{
  "ErrorEquals": ["PaymentDeclined"],
  "ResultPath": "$.error",
  "Next": "HandlePaymentFailure"
}]
```

1.6.3 Error types

Error	Source
States.Timeout	Task exceeded timeout
States.TaskFailed	Task returned error
States.Permissions	IAM permission denied
Custom errors	Application-defined (via <code>Fail</code> state or Lambda throw)

1.7 30.7 Parallel and Map states

1.7.1 Parallel

Execute multiple branches simultaneously:

```
{
  "Type": "Parallel",
  "Branches": [
    { "StartAt": "SendEmail", "States": { "SendEmail": { "Type": "Task", "Resource": "...", "End": true } } },
    { "StartAt": "UpdateInventory", "States": { "UpdateInventory": { "Type": "Task", "Resource": "...", "End": true } } },
    { "StartAt": "LogAnalytics", "States": { "LogAnalytics": { "Type": "Task", "Resource": "...", "End": true } } }
  ],
  "Next": "OrderComplete"
}
```

1.7.2 Map (dynamic parallelism)

Process each item in an array concurrently:

```
{
  "Type": "Map",
  "ItemsPath": "$.orders",
  "MaxConcurrency": 10,
  "Iterator": {
    "StartAt": "ProcessSingleOrder",
    "States": {
      "ProcessSingleOrder": {
        "Type": "Task",
        "Resource": "arn:aws:lambda:...:function:process-order",
        "End": true
      }
    }
  },
  "Next": "AllOrdersProcessed"
}
```

```
}
```

Distributed Map (newer) supports large-scale processing with S3 item sources and child executions.

1.8 30.8 Terraform

```
resource "aws_sfn_state_machine" "order_processing" {
  name      = "order-processing"
  role_arn = aws_iam_role.step_functions.arn

  definition = templatefile("${path.module}/state-machine.json", {
    validate_lambda_arn = aws_lambda_function.validate.arn
    payment_lambda_arn  = aws_lambda_function.payment.arn
  })

  logging_configuration {
    level          = "ALL"
    include_execution_data = true
    log_destination = "${aws_cloudwatch_log_group.sfn.arn}:*"
  }

  tracing_configuration {
    enabled = true
  }
}
```

1.9 30.9 Monitoring and debugging

Tool	Purpose
Execution history	Visual trace of each state transition
CloudWatch Logs	Detailed execution data
X-Ray	Distributed tracing across services
CloudWatch Metrics	ExecutionsStarted, ExecutionsFailed, ExecutionTime

```
aws stepfunctions start-execution \
  --state-machine-arn arn:aws:states:us-east-1:123456789012:stateMachine:order-processing \
  --input '{"orderId": "12345", "amount": 99.99}'

aws stepfunctions describe-execution \
  --execution-arn arn:aws:states:us-east-1:123456789012:execution:order-processing:abc123
```

1.10 30.10 Chapter summary

- **Step Functions** orchestrates multi-step workflows with built-in error handling and retries.
- **Standard workflows** for long-running processes; **Express** for high-volume, short tasks.
- Use **service integrations** to call AWS services directly without Lambda wrappers.
- **Map** and **Parallel** states enable dynamic and static concurrency.
- Monitor with execution history, CloudWatch Logs, and X-Ray tracing.

1.11 Lab 30.1 — Order workflow

1. Create a Step Functions state machine with validate → pay → fulfill steps.

2. Implement Lambda functions for each step.
3. Test successful and failed order paths.
4. Add retry logic for transient failures.

1.12 Lab 30.2 — Parallel processing

1. Create a Map state that processes an array of 10 items concurrently.
 2. Set `MaxConcurrency` to 3 and observe throttled execution.
 3. Review execution history in the Step Functions console.
-

1.13 Review questions

1. What is the difference between Standard and Express workflows?
 2. How do Retry and Catch differ in Step Functions error handling?
 3. What is the purpose of the Map state?
 4. How does the “wait for callback” pattern work with task tokens?
 5. When would you use a direct service integration vs a Lambda task?
-

Next: [Chapter 31 — CloudFormation](#)

AWS Cloud Handbook — Complete Edition —

Chapter 31

1 Chapter 31: CloudFormation Deep Dive

1.1 31.1 Infrastructure as Code with CloudFormation

AWS CloudFormation is AWS’s native Infrastructure as Code (IaC) service. You define resources in YAML or JSON templates; CloudFormation provisions, updates, and deletes them as a single **stack**. Changes are tracked, rollback is automatic on failure, and drift detection identifies manual modifications.

For AWS-only environments, CloudFormation provides the deepest service integration and zero licensing cost.

1.2 31.2 Template structure

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Web application stack

Parameters:
  Environment:
    Type: String
    Default: dev
    AllowedValues: [dev, staging, prod]

Mappings:
  RegionMap:
    us-east-1:
      AMI: ami-0c55b159cbfafa1f0
    eu-west-1:
      AMI: ami-0d71ea30463e0ff8d

Conditions:
  IsProd: !Equals [!Ref Environment, prod]

Resources:
  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !FindInMap [RegionMap, !Ref "AWS::Region", AMI]
      InstanceType: !If [IsProd, t3.medium, t3.micro]

Outputs:
  PublicIP:
    Value: !GetAtt WebServer.PublicIp
    Export:
      Name: !Sub "${AWS::StackName}-PublicIP"
```

1.2.1 Intrinsic functions

Function	Purpose	Example
!Ref	Reference parameter or resource	!Ref WebServer
!GetAtt	Get resource attribute	!GetAtt WebServer.PublicIp
!Sub	String substitution	!Sub "arn:aws:s3:::\${Bucket}"
!Join	Concatenate strings	!Join ["-", [a, b]]

<code>!Select</code>	Pick from list	<code>!Select [0, !GetAZs ""]</code>
<code>!If</code>	Conditional value	<code>!If [IsProd, t3.medium, t3.micro]</code>
<code>!FindInMap</code>	Lookup in mapping	<code>!FindInMap [RegionMap, !Ref "AWS::Region", AMI]</code>

1.3 31.3 Stack operations

```
# Validate template
aws cloudformation validate-template --template-body file://template.yaml

# Create stack
aws cloudformation create-stack \
  --stack-name web-app-dev \
  --template-body file://template.yaml \
  --parameters ParameterKey=Environment,ParameterValue=dev \
  --capabilities CAPABILITY_IAM

# Update stack
aws cloudformation update-stack \
  --stack-name web-app-dev \
  --template-body file://template.yaml \
  --parameters ParameterKey=Environment,ParameterValue=staging

# Delete stack (removes all resources)
aws cloudformation delete-stack --stack-name web-app-dev

# Describe events (troubleshooting)
aws cloudformation describe-stack-events --stack-name web-app-dev
```

1.3.1 Change sets

Preview changes before applying:

```
aws cloudformation create-change-set \
  --stack-name web-app-dev \
  --template-body file://template-v2.yaml \
  --change-set-name update-to-v2

aws cloudformation describe-change-set \
  --stack-name web-app-dev \
  --change-set-name update-to-v2

aws cloudformation execute-change-set \
  --stack-name web-app-dev \
  --change-set-name update-to-v2
```

1.4 31.4 Nested stacks

Break large templates into reusable modules:

```
Resources:
  NetworkStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://s3.amazonaws.com/templates/network.yaml
      Parameters:
        VpcCidr: 10.0.0.0/16

  AppStack:
    Type: AWS::CloudFormation::Stack
    DependsOn: NetworkStack
    Properties:
```

```

TemplateURL: https://s3.amazonaws.com/templates/app.yaml
Parameters:
  VpcId: !GetAtt NetworkStack.Outputs.VpcId
  SubnetIds: !GetAtt NetworkStack.Outputs.PrivateSubnetIds

```

1.4.1 Cross-stack references

Export values from one stack and import in another:

```

# Stack A (network)
Outputs:
  VpcId:
    Value: !Ref MyVpc
    Export:
      Name: !Sub "${AWS::StackName}-VpcId"

# Stack B (application)
Resources:
  WebServer:
    Properties:
      VpcId: !ImportValue network-stack-VpcId

```

1.5 31.5 StackSets

Deploy stacks across multiple accounts and regions from a single template:

```

aws cloudformation create-stack-set \
  --stack-set-name baseline-security \
  --template-body file://security-baseline.yaml \
  --permission-model SERVICE_MANAGED \
  --auto-deployment Enabled=true

aws cloudformation create-stack-instances \
  --stack-set-name baseline-security \
  --deployment-targets OrganizationalUnitIds=["ou-abc-123"] \
  --regions us-east-1 eu-west-1

```

Use cases: organization-wide guardrails, logging, security baselines.

1.6 31.6 IAM in CloudFormation

1.6.1 CAPABILITY flags

Flag	When required
CAPABILITY_IAM	Template creates IAM resources
CAPABILITY_NAMED_IAM	Template creates named IAM resources
CAPABILITY_AUTO_EXPAND	Template uses macros (SAM, CDK)

1.6.2 Least-privilege IAM example

```

Resources:
  LambdaRole:
    Type: AWS::IAM::Role
    Properties:
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
            Principal:
              Service: lambda.amazonaws.com

```

```

    Action: sts:AssumeRole
Policies:
  - PolicyName: DynamoDBAccess
    PolicyDocument:
      Version: "2012-10-17"
      Statement:
        - Effect: Allow
          Action:
            - dynamodb:GetItem
            - dynamodb:PutItem
          Resource: !GetAtt OrdersTable.Arn

```

1.7 31.7 Drift detection

Manual changes outside CloudFormation create **drift**:

```

aws cloudformation detect-stack-drift --stack-name web-app-dev

aws cloudformation describe-stack-drift-detection-status \
  --stack-drift-detection-id abc-123

aws cloudformation describe-stack-resource-drifts \
  --stack-name web-app-dev

```

Best practice: All infrastructure changes go through CloudFormation. Use SCPs to deny direct resource creation where possible.

1.8 31.8 Custom resources

When CloudFormation lacks native support, use **custom resources** backed by Lambda:

```

Resources:
  CustomDNS:
    Type: Custom::DNSRecord
    Properties:
      ServiceToken: !GetAtt DNSLambda.Arn
      HostedZoneId: Z1234567890
      Name: app.example.com
      Value: !GetAtt WebServer.PublicIp

```

The Lambda handles Create, Update, and Delete requests and sends a response to CloudFormation's pre-signed S3 URL.

1.9 31.9 CloudFormation Hooks

Hooks validate resources before provisioning (proactive compliance):

Hook	Purpose
Guard Hooks	Evaluate templates against policy rules
Lambda Hooks	Custom validation logic

```

aws cloudformation activate-type \
  --type HOOK \
  --type-name MyOrg::S3::EncryptionCheck \
  --publisher-id 123456789012

```

1.10 31.10 Best practices

Practice	Rationale
Use parameters for environment-specific values	Reuse templates across environments
Use mappings for region-specific constants	AMI IDs, instance types
Use conditions for optional resources	Prod-only resources (WAF, Multi-AZ)
Tag all resources	Cost allocation, automation
Use nested stacks for modularity	Manageable template sizes
Enable termination protection on prod stacks	Prevent accidental deletion
Use change sets for production updates	Preview before apply
Store templates in S3 with versioning	Audit trail, rollback
Run drift detection regularly	Catch manual changes

1.11 31.11 Chapter summary

- **CloudFormation** provisions AWS infrastructure from declarative YAML/JSON templates.
- **Stacks** group related resources; **nested stacks** and **StackSets** enable modularity and multi-account deployment.
- Use **change sets** to preview updates; **drift detection** catches manual changes.
- **Custom resources** extend CloudFormation with Lambda-backed logic.
- Follow IaC best practices: parameters, conditions, tagging, and termination protection.

1.12 Lab 31.1 — Multi-tier stack

1. Write a CloudFormation template with VPC, subnets, security groups, ALB, and EC2.
2. Deploy with parameters for environment (dev/prod).
3. Update the template to add an RDS instance (prod only, using conditions).
4. Create a change set and review before applying.

1.13 Lab 31.2 — Drift detection

1. Deploy a stack and manually modify a security group in the console.
2. Run drift detection and identify the drifted resource.
3. Update the stack to reconcile the drift.

1.14 Review questions

1. What is the purpose of CloudFormation change sets?
2. How do nested stacks improve template organization?
3. What capability flag is required when creating IAM roles in a template?
4. What is stack drift and how do you detect it?
5. When would you use StackSets instead of individual stacks?

Next: [Chapter 32 — AWS CDK](#)

AWS Cloud Handbook — Complete Edition —

Chapter 32

1 Chapter 32: AWS CDK

1.1 32.1 What is the AWS CDK?

The **AWS Cloud Development Kit (CDK)** lets you define cloud infrastructure using familiar programming languages (TypeScript, Python, Java, C#, Go). CDK synthesizes your code into CloudFormation templates, combining the expressiveness of a general-purpose language with CloudFormation's deployment engine.

CDK is ideal for teams that find YAML/JSON templates limiting and want reusable constructs, unit tests, and IDE support.

1.2 32.2 CDK concepts

Concept	Description
App	Root of the CDK application
Stack	Unit of deployment (maps to CloudFormation stack)
Construct	Reusable component (L1, L2, L3)
Synthesis	Generate CloudFormation template from code
Deployment	<code>cdk deploy</code> creates/updates CloudFormation stack

1.2.1 Construct levels

Level	Description	Example
L1 (Cfn)	Direct CloudFormation resource	<code>CfnBucket</code>
L2	Curated construct with sensible defaults	<code>s3.Bucket</code>
L3	Opinionated patterns combining multiple resources	<code>ApplicationLoadBalancedFargateService</code>

1.3 32.3 Getting started (TypeScript)

```
npm install -g aws-cdk
cdk init app --language typescript
npm install @aws-cdk/aws-lambda @aws-cdk/aws-apigatewayv2-integrations
```

1.3.1 Basic stack

```
import * as cdk from "aws-cdk-lib";
import * as s3 from "aws-cdk-lib/aws-s3";
import * as lambda from "aws-cdk-lib/aws-lambda";
import * as apigwv2 from "aws-cdk-lib/aws-apigatewayv2";
```

```

import * as integrations from "aws-cdk-lib/aws-apigatewayv2-integrations";

export class AppStack extends cdk.Stack {
  constructor(scope: cdk.App, id: string, props?: cdk.StackProps) {
    super(scope, id, props);

    const bucket = new s3.Bucket(this, "DataBucket", {
      encryption: s3.BucketEncryption.S3_MANAGED,
      blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
      versioned: true,
      removalPolicy: cdk.RemovalPolicy.RETAIN,
    });

    const fn = new lambda.Function(this, "ApiHandler", {
      runtime: lambda.Runtime.PYTHON_3_12,
      handler: "app.handler",
      code: lambda.Code.fromAsset("lambda"),
      environment: { BUCKET_NAME: bucket.bucketName },
    });

    bucket.grantReadWrite(fn);

    const api = new apigwv2.HttpApi(this, "Api", {
      defaultIntegration: new integrations.HttpLambdaIntegration(
        "LambdaIntegration", fn
      ),
    });

    new cdk.CfnOutput(this, "ApiUrl", { value: api.url! });
  }
}

const app = new cdk.App();
new AppStack(app, "AppStack", { env: { account: "123456789012", region: "us-east-1" } });

```

1.3.2 Python equivalent

```

from aws_cdk import (
    App, Stack, CfnOutput, RemovalPolicy,
    aws_s3 as s3,
    aws_lambda as lambda_,
)
from constructs import Construct

class AppStack(Stack):
    def __init__(self, scope: Construct, id: str, **kwargs):
        super().__init__(scope, id, **kwargs)

        bucket = s3.Bucket(
            self, "DataBucket",
            encryption=s3.BucketEncryption.S3_MANAGED,
            block_public_access=s3.BlockPublicAccess.BLOCK_ALL,
            versioned=True,
            removal_policy=RemovalPolicy.RETAIN,
        )

        fn = lambda_.Function(
            self, "Processor",
            runtime=lambda_.Runtime.PYTHON_3_12,
            handler="app.handler",
            code=lambda_.Code.from_asset("lambda"),
            environment={"BUCKET_NAME": bucket.bucket_name},
        )
        bucket.grant_read_write(fn)

app = App()
AppStack(app, "AppStack")
app.synth()

```

1.4 32.4 CDK workflow

```
cdk synth # Generate CloudFormation template
```

```

cdk diff      # Compare deployed stack with current code
cdk deploy    # Deploy stack(s)
cdk destroy   # Delete stack(s)
cdk ls        # List stacks in the app
cdk bootstrap # Prepare account/region (one-time)

```

1.4.1 Bootstrapping

CDK requires an S3 bucket and ECR repo for assets (Lambda code, Docker images):

```
cdk bootstrap aws://123456789012/us-east-1
```

1.5 32.5 Reusable constructs

1.5.1 Creating a construct

```

import { Construct } from "constructs";
import * as ec2 from "aws-cdk-lib/aws-ec2";
import * as rds from "aws-cdk-lib/aws-rds";

interface DatabaseProps {
  vpc: ec2.IVpc;
  instanceType: ec2.InstanceType;
}

export class Database extends Construct {
  public readonly endpoint: string;

  constructor(scope: Construct, id: string, props: DatabaseProps) {
    super(scope, id);

    const cluster = new rds.DatabaseCluster(this, "Cluster", {
      engine: rds.DatabaseClusterEngine.auroraPostgres({
        version: rds.AuroraPostgresEngineVersion.VER_15_4,
      }),
      vpc: props.vpc,
      writer: rds.ClusterInstance.provisioned("Writer", {
        instanceType: props.instanceType,
      }),
      readers: [
        rds.ClusterInstance.provisioned("Reader", {
          instanceType: props.instanceType,
        }),
      ],
      storageEncrypted: true,
    });

    this.endpoint = cluster.clusterEndpoint.hostname;
  }
}

```

1.5.2 Using the construct

```

const db = new Database(this, "AppDatabase", {
  vpc,
  instanceType: ec2.InstanceType.of(ec2.InstanceClass.R6G, ec2.InstanceSize.LARGE),
});

```

1.6 32.6 CDK Pipelines

CDK Pipelines is a high-level construct for CI/CD:

```
import { pipelines } from "aws-cdk-lib";
```

```
const pipeline = new pipelines.CodePipeline(this, "Pipeline", {
  pipelineName: "AppPipeline",
  synth: new pipelines.ShellStep("Synth", {
    input: pipelines.CodePipelineSource.gitHub("org/repo", "main"),
    commands: ["npm ci", "npm run build", "npx cdk synth"],
  }),
});

pipeline.addStage(new AppStage(this, "Prod", { env: prodEnv }));
```

Stages deploy to different environments with automatic approvals between them.

1.7 32.7 Testing CDK

1.7.1 Snapshot tests

```
import { Template } from "aws-cdk-lib/assertions";

test("S3 bucket is encrypted", () => {
  const stack = new AppStack(app, "TestStack");
  const template = Template.fromStack(stack);

  template.hasResourceProperties("AWS::S3::Bucket", {
    BucketEncryption: {
      ServerSideEncryptionConfiguration: [{
        ServerSideEncryptionByDefault: { SSEAlgorithm: "AES256" },
      }],
    },
  });
});
```

1.7.2 Fine-grained assertions

```
template.resourceCountIs("AWS::S3::Bucket", 1);
template.hasResource("AWS::IAM::Role", {
  Properties: { AssumeRolePolicyDocument: { /* ... */ } },
});
```

1.8 32.8 CDK vs CloudFormation vs Terraform

Feature	CDK	CloudFormation	Terraform
Language	TypeScript, Python, etc.	YAML/JSON	HCL
AWS integration	Deepest	Native	Via provider
Multi-cloud	No	No	Yes
State management	CloudFormation	CloudFormation	Remote state
Constructs/libraries	Rich ecosystem	Modules (limited)	Modules (extensive)
Learning curve	Moderate	Low	Moderate

1.9 32.9 Best practices

Practice	Rationale
----------	-----------

Use L2 constructs over L1	Sensible defaults, less boilerplate
Create reusable L3 constructs	Consistency across teams
Use <code>cdk diff</code> before deploy	Preview changes
Set <code>removalPolicy</code> explicitly	Prevent accidental data loss
Use aspects for cross-cutting concerns	Tagging, encryption, compliance
Write snapshot tests	Catch unintended infrastructure changes
Pin CDK version	Reproducible builds
Use environments for multi-account	<code>env: { account, region }</code>

1.10 32.10 Chapter summary

- **CDK** defines infrastructure in TypeScript, Python, or other languages; synthesizes to CloudFormation.
- **Constructs** (L1/L2/L3) provide reusable building blocks.
- **CDK Pipelines** enables CI/CD for infrastructure deployments.
- Test with **snapshot assertions** and fine-grained resource checks.
- Bootstrap accounts once; use `cdk diff` and `cdk deploy` for safe updates.

1.11 Lab 32.1 — CDK web application

1. Initialize a CDK TypeScript project.
2. Create a stack with S3, Lambda, and HTTP API.
3. Run `cdk synth` and inspect the generated CloudFormation template.
4. Deploy with `cdk deploy` and test the API endpoint.

1.12 Lab 32.2 — Reusable construct

1. Create a `VpcStack` construct with public/private subnets.
2. Create an `AppStack` that imports the VPC and deploys ECS Fargate.
3. Write a snapshot test verifying the VPC has 2 AZs and 4 subnets.

1.13 Review questions

1. What is the relationship between CDK and CloudFormation?
2. What is the difference between L1, L2, and L3 constructs?
3. Why is `cdk bootstrap` required before the first deployment?
4. How do CDK snapshot tests help prevent infrastructure regressions?
5. When would you choose CDK over raw CloudFormation templates?

Next: [Chapter 33 — CodePipeline, CodeBuild, CodeDeploy](#)

AWS Cloud Handbook — Complete Edition —

Chapter 33

1 Chapter 33: CodePipeline, CodeBuild, and CodeDeploy

1.1 33.1 CI/CD on AWS

Continuous Integration and Continuous Delivery automate the path from code commit to production deployment. AWS provides a native CI/CD suite:

Service	Role
CodeCommit	Git repository (being deprecated; use GitHub/GitLab)
CodeBuild	Build and test code
CodeDeploy	Deploy to EC2, ECS, Lambda, on-premises
CodePipeline	Orchestrate the full pipeline
CodeArtifact	Package repository (npm, Maven, PyPI)

1.2 33.2 CodePipeline

CodePipeline orchestrates stages from source to deployment:

Source → Build → Test → Deploy (Staging) → Approval → Deploy (Production)

1.2.1 Pipeline stages

Stage	Actions
Source	GitHub, CodeCommit, S3, ECR
Build	CodeBuild, Jenkins, custom
Test	CodeBuild, third-party
Deploy	CodeDeploy, CloudFormation, ECS, Lambda, S3
Approval	Manual approval gate

1.2.2 Terraform pipeline

```
resource "aws_codepipeline" "app" {
  name      = "app-pipeline"
  role_arn = aws_iam_role.pipeline.arn

  artifact_store {
    location = aws_s3_bucket.artifacts.bucket
    type     = "S3"
  }
}
```

```

stage {
  name = "Source"
  action {
    name           = "GitHub"
    category       = "Source"
    owner          = "AWS"
    provider       = "CodeStarSourceConnection"
    version        = "1"
    output_artifacts = ["source_output"]
    configuration = {
      ConnectionArn = aws_codestarconnections_connection.github.arn
      FullRepositoryId = "org/app-repo"
      BranchName     = "main"
    }
  }
}

stage {
  name = "Build"
  action {
    name           = "Build"
    category       = "Build"
    owner          = "AWS"
    provider       = "CodeBuild"
    version        = "1"
    input_artifacts = ["source_output"]
    output_artifacts = ["build_output"]
    configuration = {
      ProjectName = aws_codebuild_project.app.name
    }
  }
}

stage {
  name = "Deploy"
  action {
    name           = "DeployECS"
    category       = "Deploy"
    owner          = "AWS"
    provider       = "ECS"
    version        = "1"
    input_artifacts = ["build_output"]
    configuration = {
      ClusterName = aws_ecs_cluster.main.name
      ServiceName = aws_ecs_service.web.name
      FileName     = "imagedefinitions.json"
    }
  }
}

```

1.3 33.3 CodeBuild

CodeBuild compiles source code, runs tests, and produces deployment artifacts:

1.3.1 buildspec.yml

```

version: 0.2

env:
  variables:
    NODE_ENV: production
  parameter-store:
    DB_PASSWORD: /app/db-password
  secrets-manager:
    API_KEY: app/api-key

phases:
  install:
    runtime-versions:
      nodejs: 20
    commands:

```

```

- npm ci

pre_build:
  commands:
    - npm run lint
    - npm run test:unit

build:
  commands:
    - npm run build
    - docker build -t $IMAGE_REPO_NAME:$IMAGE_TAG .

post_build:
  commands:
    - aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
    - docker push $IMAGE_REPO_NAME:$IMAGE_TAG
    - printf ' [{"name":"web","imageUri":"%s"}]' $IMAGE_REPO_NAME:$IMAGE_TAG > imagedefinitions.json

artifacts:
  files:
    - imagedefinitions.json
    - dist/**/*
  discard-paths: no

cache:
  paths:
    - node_modules/**/*
    - .npm/**/*

```

1.3.2 CodeBuild features

Feature	Purpose
Custom images	Docker-based build environments
VPC support	Access private resources during build
Caching	S3 or local cache for dependencies
Reports	Test and coverage reports
Batch builds	Matrix builds across configurations
Arm builds	Graviton-based build environments

```

resource "aws_codebuild_project" "app" {
  name           = "app-build"
  service_role   = aws_iam_role.codebuild.arn
  build_timeout  = 15

  artifacts { type = "CODEPIPELINE" }
  source {
    type      = "CODEPIPELINE"
    buildspec = "buildspec.yml"
  }

  environment {
    compute_type      = "BUILD_GENERAL1_MEDIUM"
    image             = "aws/codebuild/amazonlinux2-x86_64-standard:5.0"
    type              = "LINUX_CONTAINER"
    privileged_mode    = true # required for Docker builds
    image_pull_credentials_type = "CODEBUILD"

    environment_variable {
      name = "ECR_URI"
      value = aws_ecr_repository.app.repository_url
    }
  }

  cache {
    type      = "S3"
    location = "${aws_s3_bucket.cache.bucket}/codebuild-cache"
  }
}

```

1.4 33.4 CodeDeploy

CodeDeploy automates application deployments to various compute platforms:

Target	Deployment type
EC2 / On-premises	In-place or blue/green
ECS	Blue/green with ALB traffic shifting
Lambda	Canary or linear traffic shifting

1.4.1 ECS blue/green deployment

ALB Target Group A (blue) ← current production
ALB Target Group B (green) ← new version

1. Deploy new task definition to green target group
2. Shift traffic: 10% → 50% → 100% to green
3. Monitor CloudWatch alarms
4. Auto-rollback if alarms trigger
5. Terminate blue tasks

1.4.2 appspec.yml (ECS)

```
version: 0.0
Resources:
  - TargetService:
    Type: AWS::ECS::Service
    Properties:
      TaskDefinition: arn:aws:ecs:us-east-1:123456789012:task-definition/web:5
      LoadBalancerInfo:
        ContainerName: web
        ContainerPort: 8080
      PlatformVersion: LATEST
      NetworkConfiguration:
        AwsVpcConfiguration:
          Subnets: [subnet-aaa, subnet-bbb]
          SecurityGroups: [sg-ecs]
          AssignPublicIp: DISABLED
```

1.4.3 Lambda canary deployment

```
version: 0.0
Resources:
  - MyFunction:
    Type: AWS::Lambda::Function
    Properties:
      Name: api-handler
      Alias: live
      CurrentVersion: "3"
      TargetVersion: "4"
```

Traffic shifting: 10% every 5 minutes to the new version, with automatic rollback on alarm.

1.5 33.5 Deployment strategies comparison

Strategy	Downtime	Rollback speed	Complexity
In-place	Brief (rolling)	Slow (redeploy)	Low
Blue/green	None	Fast (switch ALB)	Medium
Canary	None	Fast (shift back)	Medium

Canary	None	Fast (SNIT BACK)	Medium
Linear	None	Fast	Medium
Rolling (ECS)	None	Moderate	Low

1.6 33.6 GitHub Actions alternative

Many teams use GitHub Actions with OIDC federation instead of CodePipeline:

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
          aws-region: us-east-1
      - run: npm ci && npm test && npm run build
      - run: |
          aws ecr get-login-password | docker login --username AWS --password-stdin $ECR_URI
          docker build -t $ECR_URI:$GITHUB_SHA .
          docker push $ECR_URI:$GITHUB_SHA
          aws ecs update-service --cluster app --service web --force-new-deployment
```

1.7 33.7 Security best practices

Practice	Implementation
Least-privilege IAM roles	Separate roles for pipeline, build, deploy
Encrypted artifacts	S3 SSE-KMS on artifact bucket
No long-term credentials	OIDC federation for GitHub/GitLab
Secrets in Parameter Store/Secrets Manager	Never in buildspec or code
VPC for builds accessing private resources	CodeBuild VPC configuration
Manual approval for production	CodePipeline approval stage

1.8 33.8 Chapter summary

- **CodePipeline** orchestrates CI/CD from source to deployment.
- **CodeBuild** compiles, tests, and packages applications with `buildspec.yml`.
- **CodeDeploy** automates deployments to EC2, ECS, and Lambda with blue/green and canary strategies.
- Use **OIDC federation** instead of long-term credentials for external Git providers.
- Choose deployment strategy based on downtime tolerance and rollback requirements.

1.9 Lab 33.1 — CodePipeline for ECS

1. Create a CodeBuild project with a Docker buildspec.
2. Create a CodePipeline: GitHub source → CodeBuild → ECS deploy.
3. Push a code change and watch the pipeline execute.
4. Verify the new container version is running in ECS.

1.10 Lab 33.2 — Blue/green ECS deployment

1. Configure CodeDeploy for ECS blue/green with ALB.
 2. Deploy a new task definition version.
 3. Observe traffic shifting from blue to green target group.
 4. Trigger a CloudWatch alarm and verify automatic rollback.
-

1.11 Review questions

1. What is the role of each service in the CodePipeline → CodeBuild → CodeDeploy flow?
 2. What is the purpose of `buildspec.yml`?
 3. How does ECS blue/green deployment minimize downtime?
 4. Why should CI/CD use OIDC instead of access keys?
 5. What deployment strategy provides the fastest rollback?
-

Next: [Chapter 34 — Systems Manager & Parameter Store](#)

AWS Cloud Handbook — Complete Edition —

Chapter 34

1 Chapter 34: Systems Manager & Parameter Store

1.1 34.1 AWS Systems Manager overview

AWS Systems Manager is a unified operations hub for managing AWS and hybrid resources. It eliminates the need for SSH bastion hosts, manual patching, and ad-hoc scripting by providing centralized visibility, automation, and configuration management.

Key capabilities include **Session Manager** (browser-based shell access), **Patch Manager**, **Run Command**, **State Manager**, and integration with **Parameter Store** and **Secrets Manager**.

1.2 34.2 Session Manager

Session Manager provides secure, auditable shell access to EC2 instances and on-premises servers without opening inbound SSH ports:

Engineer → AWS Console/CLI → Session Manager → SSM Agent → EC2 instance

1.2.1 Advantages over SSH

Feature	SSH	Session Manager
Inbound port 22	Required	Not required
Bastion host	Often needed	Not needed
Key management	SSH keys	IAM permissions
Audit logging	Manual	CloudWatch Logs + S3 automatically
Access control	Security groups + keys	IAM policies

1.2.2 Requirements

1. **SSM Agent** installed on the instance (pre-installed on Amazon Linux, Windows AMIs).
2. **IAM instance profile** with `AmazonSSMManagedInstanceCore` policy.
3. **Network path** to SSM endpoints (internet, NAT, or VPC endpoints).

```
# Start a session
aws ssm start-session --target i-0abc123

# Port forwarding (access RDS through instance)
aws ssm start-session \
  --target i-0abc123 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["mydb.abc.rds.amazonaws.com"],"portNumber":["5432"],"localPortNumber":["5432"]}'
```

1.2.3 Session logging

```
resource "aws_ssm_document" "session_prefs" {
```

```

name           = "SSM-SessionManagerRunShell"
document_type  = "Session"
document_format = "JSON"
content = jsonencode({
  schemaVersion = "1.0"
  sessionType   = "Standard_Stream"
  inputs = {
    s3BucketName      = aws_s3_bucket.session_logs.bucket
    s3KeyPrefix       = "session-logs/"
    cloudWatchLogGroupName = aws_cloudwatch_log_group.sessions.name
    cloudWatchEncryptionEnabled = true
  }
})
}

```

1.3 34.3 Run Command

Execute commands across fleets without SSH:

```

# Run command on tagged instances
aws ssm send-command \
  --document-name "AWS-RunShellScript" \
  --targets "Key=tag:Environment,Values=production" \
  --parameters 'commands=["yum update -y", "systemctl restart httpd"]' \
  --comment "Patch and restart web servers"

# Check command status
aws ssm list-command-invocations \
  --command-id abc-123 \
  --details

```

1.3.1 Common documents

Document	Purpose
AWS-RunShellScript	Run shell commands (Linux)
AWS-RunPowerShellScript	Run PowerShell (Windows)
AWS-UpdateSSMAgent	Update SSM Agent
AWS-ConfigureAWSPackage	Install SSM packages

1.4 34.4 Patch Manager

Automate OS patching across your fleet:

Maintenance Window → Patch Baseline → Scan → Install → Reboot (if needed)

```

resource "aws_ssm_patch_baseline" "amazon_linux" {
  name           = "amazon-linux-baseline"
  operating_system = "AMAZON_LINUX_2"

  approval_rule {
    approve_after_days = 7
    patch_filter {
      key    = "CLASSIFICATION"
      values = ["Security", "Critical"]
    }
  }
}

resource "aws_ssm_maintenance_window" "patch" {
  name = "patch-window"
}

```

```

schedule = "cron(0 2 ? * SUN *)" # Sundays at 2 AM
duration = 4
cutoff   = 1
}

```

1.5 34.5 State Manager

State Manager maintains desired configuration state on instances (similar to Ansible/Chef):

```

# Association: ensure CloudWatch agent is installed
schemaVersion: "2.2"
description: Install and configure CloudWatch agent
mainSteps:
  - action: aws:configurePackage
    name: installCWAgent
    inputs:
      action: Install
      name: AmazonCloudWatchAgent
  - action: aws:runCommand
    name: configureCWAgent
    inputs:
      documentType: SSMDocument
      documentPath: AmazonCloudWatch-ManageAgent
      parameters:
        action: configure
        mode: ec2

```

Associations run on a schedule or at instance launch, ensuring compliance.

1.6 34.6 Parameter Store

Systems Manager Parameter Store provides hierarchical storage for configuration data and secrets:

Type	Cost	Use case
String	Free	Configuration values, AMI IDs
StringList	Free	Comma-separated values
SecureString	Free (KMS charges apply)	Passwords, API keys, tokens

1.6.1 Hierarchy

```

/app/prod/database/host
/app/prod/database/port
/app/prod/database/password (SecureString)
/app/dev/database/host
/shared/ami/amazon-linux-2023

```

1.6.2 Creating parameters

```

aws ssm put-parameter \
  --name "/app/prod/database/host" \
  --value "mydb.abc.us-east-1.rds.amazonaws.com" \
  --type String \
  --tags "Key=Environment,Value=prod"

aws ssm put-parameter \
  --name "/app/prod/database/password" \
  --value "super-secret" \
  --type SecureString \
  --key-id alias/app-secrets

```

1.6.3 Reading parameters

```
# Single parameter
aws ssm get-parameter \
  --name "/app/prod/database/password" \
  --with-decryption

# By path (all under /app/prod/)
aws ssm get-parameters-by-path \
  --path "/app/prod/" \
  --recursive \
  --with-decryption
```

1.6.4 In application code

```
import boto3

ssm = boto3.client("ssm")
response = ssm.get_parameters_by_path(
    Path="/app/prod/",
    Recursive=True,
    WithDecryption=True,
)
config = {p["Name"].split("/")[1]: p["Value"] for p in response["Parameters"]}
```

1.6.5 Terraform

```
resource "aws_ssm_parameter" "db_host" {
  name = "/app/${var.environment}/database/host"
  type = "String"
  value = aws_rds_cluster.main.endpoint
}

resource "aws_ssm_parameter" "db_password" {
  name = "/app/${var.environment}/database/password"
  type = "SecureString"
  key_id = aws_kms_key.app.id
  value = var.db_password
}
```

1.7 34.7 Parameter Store vs Secrets Manager

Feature	Parameter Store	Secrets Manager
Cost	Free (Standard); KMS for SecureString	\$0.40/secret/month + API calls
Rotation	Manual or Lambda	Built-in rotation for RDS
Cross-account	Via RAM	Via resource policy
Max size	4 KB (advanced: 8 KB)	64 KB
Versioning	Yes	Yes
Use case	Config, non-rotating secrets	Database credentials, API keys with rotation

Guideline: Use Parameter Store for configuration and non-rotating secrets. Use Secrets Manager for credentials requiring automatic rotation.

1.8 34.8 Inventory and Compliance

Feature	Purpose
---------	---------

Inventory	Collect OS, application, and custom metadata from instances
Compliance	Track patch compliance against baselines
OpsCenter	Investigate and remediate operational issues

```
aws ssm get-inventory \
  --filters '[{"Key":"AWS:InstanceInformation.PlatformName","Values":["Amazon Linux"],"Type":"Equal"}]'
```

1.9 34.9 VPC endpoints for SSM

For instances without internet access, create VPC interface endpoints:

Endpoint	Purpose
<code>com.amazonaws.region.ssm</code>	Systems Manager API
<code>com.amazonaws.region.ssmmessages</code>	Session Manager
<code>com.amazonaws.region.ec2messages</code>	SSM Agent communication

```
resource "aws_vpc_endpoint" "ssm" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.region}.ssm"
  vpc_endpoint_type = "Interface"
  subnet_ids       = aws_subnet.private[*].id
  security_group_ids = [aws_security_group.vpc_endpoints.id]
  private_dns_enabled = true
}
```

1.10 34.10 Chapter summary

- **Session Manager** replaces SSH with IAM-controlled, auditable shell access.
- **Run Command** and **Patch Manager** automate fleet operations.
- **Parameter Store** provides hierarchical configuration and secret storage.
- Use **Secrets Manager** for credentials requiring automatic rotation.
- Deploy **VPC endpoints** for SSM in private subnets without internet access.

1.11 Lab 34.1 — Session Manager access

1. Launch an EC2 instance with SSM instance profile (no SSH key, no port 22).
2. Connect via Session Manager in the console.
3. Enable session logging to S3 and CloudWatch Logs.
4. Review the session log after disconnecting.

1.12 Lab 34.2 — Parameter Store configuration

1. Create a hierarchy of parameters under `/app/dev/`.
2. Write a script that reads all parameters and prints configuration.
3. Update a Lambda function to read database credentials from Parameter Store.
4. Rotate a SecureString parameter and verify the application picks up the new value.

1.13 Review questions

1. What are the advantages of Session Manager over traditional SSH?
 2. What IAM policy must an EC2 instance have for Session Manager?
 3. When should you use Secrets Manager instead of Parameter Store?
 4. How does State Manager differ from Run Command?
 5. Which VPC endpoints are required for Session Manager in a private subnet?
-

Next: [*Chapter 35 — CloudWatch Fundamentals*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 35

1 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards

1.1 35.1 Amazon CloudWatch overview

CloudWatch is AWS’s observability service for **metrics**, **logs**, **alarms**, and **dashboards**. Nearly every AWS service emits CloudWatch metrics automatically.

1.2 35.2 Metrics

A **metric** is a time-series datapoint (e.g., `CPUUtilization` for EC2).

Concept	Description
Namespace	Grouping (AWS/EC2, AWS/S3, custom)
Metric name	e.g., <code>CPUUtilization</code>
Dimensions	<code>InstanceId</code> , <code>FunctionName</code> , etc.
Resolution	Standard 1 min; high-resolution down to 1 sec

1.2.1 View metrics

```
aws cloudwatch list-metrics --namespace AWS/EC2
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --start-time 2026-09-01T00:00:00Z \
  --end-time 2026-09-01T12:00:00Z \
  --period 300 \
  --statistics Average
```

1.2.2 Custom metrics

```
import boto3
cloudwatch = boto3.client('cloudwatch')
cloudwatch.put_metric_data(
    Namespace='MyApp/Orders',
    MetricData=[{
        'MetricName': 'OrdersProcessed',
        'Value': 42,
        'Unit': 'Count'
    }]
)
```

1.3 35.3 Alarms

Alarms trigger actions when metrics breach thresholds.

```
aws cloudwatch put-metric-alarm \
  --alarm-name high-cpu-web-server \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:ops-alerts \
  --dimensions Name=InstanceId,Value=i-0abc123
```

1.3.1 Alarm states

State	Meaning
OK	Within threshold
ALARM	Breached threshold
INSUFFICIENT_DATA	Not enough datapoints

1.3.2 Composite alarms

Combine multiple alarms with AND/OR logic to reduce alert noise.

1.4 35.4 Dashboards

Visualize metrics in the console or via API. Example Terraform:

```
resource "aws_cloudwatch_dashboard" "main" {
  dashboard_name = "production-overview"
  dashboard_body = jsonencode({
    widgets = [{
      type = "metric"
      width = 12
      height = 6
      properties = {
        metrics = [
          ["AWS/EC2", "CPUUtilization", "InstanceId", "i-0abc123"],
          [".", "NetworkIn", ".", "."]
        ]
        period = 300
        stat = "Average"
        region = "us-east-1"
        title = "EC2 Web Server"
      }
    }]
  })
}
```

1.5 35.5 CloudWatch Logs (overview)

Feature	Purpose
Log groups / streams	Organize application logs
Logs Insights	SQL-like query language
Subscription filters	Stream to Lambda, Kinesis, OpenSearch
Metric filters	Turn log patterns into metrics

```
aws logs tail /aws/lambda/my-function --follow
```

1.6 35.6 Observability best practices on AWS

1. **Alarm on symptoms** — Error rate, latency p99, not just CPU
 2. **SNS → PagerDuty/Slack** for on-call routing
 3. **Dashboards per service** + one executive overview
 4. **Log retention** — Set retention policies (cost control)
 5. **X-Ray** for distributed tracing (complements CloudWatch)
 6. Use **Container Insights** / **Lambda Insights** for managed runtimes
-

1.7 35.7 Chapter summary

- CloudWatch collects **metrics** from AWS and custom apps.
 - **Alarms** notify via SNS when thresholds breach.
 - **Dashboards** and **Logs** complete the operational picture.
-

1.8 Lab 35.1

1. Create SNS topic `ops-alerts` (email subscription).
 2. Create CPU alarm on an EC2 instance.
 3. Build a dashboard with EC2 CPU, ALB request count, and RDS connections.
 4. Run a Logs Insights query on a Lambda log group.
-

Continue: Chapter 36 — CloudWatch Logs Deep Dive (outline in TOC)

AWS Cloud Handbook — Complete Edition —

Chapter 36

1 Chapter 36: CloudWatch Logs & Insights

1.1 36.1 CloudWatch Logs overview

Amazon CloudWatch Logs centralizes log data from AWS services, applications, and on-premises servers. Every Lambda function, API Gateway stage, ECS task, and VPC Flow Log can stream to CloudWatch Logs for search, analysis, alerting, and long-term retention.

Effective log management is the foundation of operational troubleshooting and security investigation.

1.2 36.2 Log groups and streams

Concept	Description
Log group	Container for related log streams (e.g., <code>/aws/lambda/api-handler</code>)
Log stream	Sequence of events from a single source (e.g., one Lambda instance)
Log event	Single log entry with timestamp and message
Retention	1 day to 10 years (default: never expire)

1.2.1 Common log group paths

Service	Log group pattern
Lambda	<code>/aws/lambda/<function-name></code>
API Gateway	<code>/aws/apigateway/<api-name></code>
ECS	<code>/ecs/<task-definition></code>
RDS	<code>/aws/rds/instance/<id>/error</code>
VPC Flow Logs	Custom name
CloudTrail	Custom name

```
# Set retention policy
aws logs put-retention-policy \
  --log-group-name /aws/lambda/api-handler \
  --retention-in-days 30

# Create log group
aws logs create-log-group --log-group-name /app/production
```

1.3 36.3 Ingesting logs

1.3.1 AWS service integration

Most AWS services send logs automatically when enabled:

```
resource "aws_cloudwatch_log_group" "lambda" {
  name           = "/aws/lambda/api-handler"
  retention_in_days = 30
  kms_key_id     = aws_kms_key.logs.arn
}

resource "aws_lambda_function" "api" {
  # ...
  logging_config {
    log_format = "JSON"
    application_log_level = "INFO"
    system_log_level = "WARN"
  }
}
```

1.3.2 CloudWatch Agent (EC2/on-premises)

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/application.log",
            "log_group_name": "/app/production",
            "log_stream_name": "{instance_id}/application",
            "timezone": "UTC"
          }
        ]
      }
    }
  }
}
```

1.3.3 Subscription filters

Stream logs to Lambda, Kinesis, or Firehose for real-time processing:

```
resource "aws_cloudwatch_log_subscription_filter" "errors" {
  name           = "error-filter"
  log_group_name = aws_cloudwatch_log_group.app.name
  filter_pattern = "ERROR"
  destination_arn = aws_lambda_function.alert.arn
}
```

1.4 36.4 CloudWatch Logs Insights

Logs Insights is a purpose-built query language for analyzing log data at scale without provisioning servers.

1.4.1 Query syntax

```
fields @timestamp, @message, @logStream
| filter @message like /ERROR/
| sort @timestamp desc
| limit 50
```

1.4.2 Common queries

Error rate by function (Lambda):

```
fields @timestamp, @message
| filter @message like /ERROR|Exception|error/
| stats count() as errorCount by bin(5m)
```

Slow API requests:

```
fields @timestamp, @message
| filter @type = "REPORT"
| filter @duration > 1000
| sort @duration desc
| limit 20
```

Top IP addresses (VPC Flow Logs):

```
fields srcAddr, dstAddr, bytes
| filter action = "ACCEPT"
| stats sum(bytes) as totalBytes by srcAddr
| sort totalBytes desc
| limit 10
```

Parse structured JSON logs:

```
fields @timestamp, level, message, userId, duration
| filter level = "ERROR"
| sort @timestamp desc
| limit 100
```

1.4.3 CLI query

```
aws logs start-query \
  --log-group-names /aws/lambda/api-handler \
  --start-time $(date -d '1 hour ago' +%s) \
  --end-time $(date +%s) \
  --query-string 'fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc | limit 20'
```

1.5 36.5 Metric filters

Convert log patterns into CloudWatch metrics for alarming:

```
resource "aws_cloudwatch_log_metric_filter" "errors" {
  name           = "api-errors"
  log_group_name = aws_cloudwatch_log_group.api.name
  pattern        = "ERROR"

  metric_transformation {
    name       = "ApiErrorCount"
    namespace = "App/Errors"
    value      = "1"
    default_value = "0"
  }
}

resource "aws_cloudwatch_metric_alarm" "error_rate" {
  alarm_name         = "high-api-errors"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 2
  metric_name        = "ApiErrorCount"
  namespace          = "App/Errors"
  period             = 300
  statistic           = "Sum"
  threshold          = 10
  alarm_actions      = [aws_sns_topic.alerts.arn]
}
```

1.6 36.6 Log encryption and access control

Control	Implementation
Encryption at rest	KMS key on log group
Encryption in transit	TLS for all API calls
Resource policies	Cross-account and service access
IAM policies	logs:CreateLogStream, logs:PutLogEvents, logs:FilterLogEvents

1.6.1 Cross-account log sharing

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "AWS": "arn:aws:iam::AUDIT_ACCOUNT:root" },
    "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
    "Resource": "arn:aws:logs:us-east-1:123456789012:log-group:/security/*"
  }]
}
```

1.7 36.7 Log export and archival

Method	Destination	Use case
Export to S3	S3 bucket	Long-term archival, Athena analysis
Subscription filter	Kinesis Firehose → S3/OpenSearch	Real-time streaming
Logs Insights	Query results	Ad-hoc analysis

```
aws logs create-export-task \
  --log-group-name /aws/lambda/api-handler \
  --from $(date -d '7 days ago' +%s000) \
  --to $(date +%s000) \
  --destination my-log-archive-bucket \
  --destination-prefix lambda-logs/
```

1.8 36.8 Best practices

Practice	Rationale
Set retention policies	Control costs; 30-90 days for most workloads
Use structured JSON logging	Enables field-based queries in Insights
Include correlation IDs	Trace requests across services
Create metric filters for errors	Proactive alarming
Encrypt sensitive log groups	KMS for compliance
Avoid logging secrets/PII	Redact or mask sensitive data
Use log levels appropriately	DEBUG in dev, INFO/WARN in prod
Centralize logs in a dedicated account	Security team access, tamper resistance

1.8.1 Structured logging example

```
import json, logging
```

```
logger = logging.getLogger()
logger.setLevel(logging.INFO)

def handler(event, context):
    logger.info(json.dumps({
        "level": "INFO",
        "message": "Processing order",
        "orderId": event["orderId"],
        "requestId": context.aws_request_id,
        "duration_ms": None,
    }))
```

1.9 36.9 Chapter summary

- **CloudWatch Logs** centralizes log data from AWS services and applications.
- **Logs Insights** provides a powerful query language for log analysis.
- **Metric filters** convert log patterns into alarmable CloudWatch metrics.
- Use **structured JSON logging** and **retention policies** for effective operations.
- **Subscription filters** enable real-time log processing and alerting.

1.10 Lab 36.1 — Logs Insights queries

1. Deploy a Lambda function that logs structured JSON (level, message, requestId).
2. Invoke it multiple times with success and error paths.
3. Write Insights queries to find errors, calculate average duration, and count by log level.

1.11 Lab 36.2 — Error alerting pipeline

1. Create a metric filter for ERROR log entries.
2. Create a CloudWatch alarm on the metric (threshold > 5 in 5 minutes).
3. Connect the alarm to an SNS topic.
4. Trigger errors and verify the alert fires.

1.12 Review questions

1. What is the difference between a log group and a log stream?
2. How do metric filters bridge logs and alarms?
3. Why is structured JSON logging recommended over plain text?
4. What is a subscription filter used for?
5. How do you control CloudWatch Logs costs?

Next: [Chapter 37 — X-Ray Distributed Tracing](#)

AWS Cloud Handbook — Complete Edition —

Chapter 37

1 Chapter 37: X-Ray & Distributed Tracing

1.1 37.1 Why distributed tracing?

In microservice architectures, a single user request traverses multiple services: API Gateway → Lambda → DynamoDB → SQS → another Lambda → SNS. When latency spikes or errors occur, traditional logging cannot show the full request path.

AWS X-Ray provides end-to-end tracing, showing the complete journey of a request with timing, errors, and dependencies visualized as a service map.

1.2 37.2 X-Ray concepts

Concept	Description
Trace	Complete path of a request through services
Segment	Work done by a single service (e.g., Lambda execution)
Subsegment	Downstream call within a segment (e.g., DynamoDB query)
Annotation	Indexed key-value pairs for filtering (e.g., <code>userId=123</code>)
Metadata	Non-indexed data attached to segments
Service map	Visual graph of service dependencies
Trace ID	Unique identifier propagated across services

1.3 37.3 How X-Ray works

```
Request → API Gateway (segment) → Lambda (segment)
                                     ├── DynamoDB (subsegment)
                                     ├── SQS (subsegment)
                                     └── Lambda 2 (segment)
                                           └── SNS (subsegment)
```

1. First service generates a **trace ID** and **segment**.
2. Trace ID propagates via HTTP header (`X-Amzn-Trace-Id`) or message attribute.
3. Each downstream service creates its own segment linked to the trace.
4. Segments are sent to the X-Ray daemon (or SDK) and stored by the X-Ray service.

1.4 37.4 Enabling X-Ray

1.4.1 AWS service integration

Service	Enable method
Lambda	<code>tracing_config { mode = "Active" }</code>
API Gateway	Stage tracing enabled
ECS	X-Ray daemon sidecar container
EKS	ADOT (AWS Distro for OpenTelemetry) collector
Elastic Beanstalk	Enable in environment config
Step Functions	<code>tracing_configuration { enabled = true }</code>

1.4.2 Lambda with active tracing

```
resource "aws_lambda_function" "api" {
  # ...
  tracing_config {
    mode = "Active"
  }
}

resource "aws_iam_role_policy" "xray" {
  role = aws_iam_role.lambda.id
  policy = jsonencode({
    Statement = [{
      Effect   = "Allow"
      Action   = ["xray:PutTraceSegments", "xray:PutTelemetryRecords"]
      Resource = "*"
    }]
  })
}
```

1.4.3 API Gateway

```
resource "aws_apigatewayv2_stage" "prod" {
  # ...
  default_route_settings {
    detailed_metrics_enabled = true
    throttling_burst_limit  = 100
    throttling_rate_limit   = 50
  }
}
```

1.5 37.5 SDK instrumentation

For custom applications, use the X-Ray SDK:

1.5.1 Python

```
from aws_xray_sdk.core import xray_recorder, patch_all
from aws_xray_sdk.core import lambda_launcher

patch_all() # auto-instrument boto3, requests, etc.

@xray_recorder.capture("process_order")
def process_order(order_id):
    xray_recorder.put_annotation("orderId", order_id)
    xray_recorder.put_metadata("order", {"id": order_id, "source": "api"})

    # Subsegment for DynamoDB call (auto-patched by patch_all)
    table.put_item(Item={"orderId": order_id, "status": "processing"})

def handler(event, context):
    process_order(event["orderId"])
    return {"statusCode": 200}
```

1.5.2 Node.js

```
const AWSXRay = require("aws-xray-sdk");
const AWS = AWSXRay.captureAWS(require("aws-sdk"));

exports.handler = async (event) => {
  const segment = AWSXRay.getSegment();
  segment.addAnnotation("orderId", event.orderId);

  const subsegment = segment.addNewSubsegment("validateOrder");
  try {
    // business logic
    subsegment.close();
  } catch (err) {
    subsegment.addError(err);
    subsegment.close();
    throw err;
  }
};
```

1.6 37.6 Service map

The **service map** visualizes dependencies and health:

```
[Client] → [API Gateway] → [Lambda: api-handler] → [DynamoDB: Orders]
                                   ↓ (error rate 2%)
                               [SQS: orders-queue] → [Lambda: processor]
```

- **Green** edges: healthy (< 5% error rate).
- **Red/yellow** edges: elevated errors or latency.
- **Node size**: proportional to request volume.

Use the service map to identify bottlenecks, error hotspots, and unexpected dependencies.

1.7 37.7 Trace analysis

1.7.1 Filtering traces

Filter	Example
By annotation	annotation.orderId = "12345"
By response time	responsetime > 3
By error	error = true
By service	service("api-handler")
Combined	service("api-handler") { error = true }

1.7.2 Trace detail view

Each trace shows: - **Timeline** — waterfall of segments with duration. - **Segments** — per-service breakdown with metadata. - **Exceptions** — stack traces for errors. - **Annotations** — searchable key-value pairs.

```
aws xray get-trace-summaries \
  --start-time $(date -d '1 hour ago' +%s) \
  --end-time $(date +%s) \
  --filter-expression 'service("api-handler") { error = true }'
```

1.8 37.8 X-Ray with OpenTelemetry

AWS supports **OpenTelemetry (OTel)** as the modern standard for instrumentation:

Component	AWS implementation
SDK	OpenTelemetry SDK (language-specific)
Collector	ADOT (AWS Distro for OpenTelemetry)
Backend	X-Ray, CloudWatch, Prometheus, Jaeger

ADOT collector on ECS/EKS/EC2 receives OTLP traces and exports to X-Ray:

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317

exporters:
  awsxray:
    region: us-east-1

service:
  pipelines:
    traces:
      receivers: [otlp]
      exporters: [awsxray]
```

1.9 37.9 Sampling

X-Ray uses sampling to control costs—by default, 1 request/second plus 5% of additional requests are traced.

1.9.1 Custom sampling rules

```
{
  "version": 2,
  "rules": [{
    "description": "Trace all errors",
    "host": "*",
    "http_method": "*",
    "url_path": "*",
    "fixed_target": 0,
    "rate": 1.0,
    "priority": 100,
    "service_name": "*",
    "service_type": "*",
    "resource_arn": "*"
  }, {
    "description": "Sample 10% of API calls",
    "host": "*",
    "http_method": "*",
    "url_path": "/api/*",
    "fixed_target": 1,
    "rate": 0.1,
    "priority": 200,
    "service_name": "api-handler",
    "service_type": "AWS::Lambda::Function",
    "resource_arn": "*"
  }]
}
```

1.10 37.10 X-Ray groups and insights

Feature	Purpose
Groups	Filter traces by criteria; track error rates and latency
Insights	Automatically detect anomalies in trace data
Encryption	KMS encryption for trace data

1.11 37.11 Chapter summary

- **X-Ray** provides end-to-end distributed tracing across AWS services and custom applications.
- **Segments** and **subsegments** capture timing and errors at each service boundary.
- Enable active tracing on Lambda, API Gateway, and ECS; use SDKs for custom instrumentation.
- The **service map** visualizes dependencies and identifies bottlenecks.
- Use **sampling rules** to balance observability with cost; consider **OpenTelemetry** for modern instrumentation.

1.12 Lab 37.1 — Lambda tracing

1. Enable active X-Ray tracing on a Lambda function behind API Gateway.
2. Invoke the API with success and error scenarios.
3. View the service map and trace timeline in the X-Ray console.
4. Filter traces by error and identify the failing subsegment.

1.13 Lab 37.2 — Custom annotations

1. Add X-Ray SDK annotations (`userId` , `orderId`) to a Lambda function.
2. Generate traffic with different user IDs.
3. Filter traces by annotation in the X-Ray console.
4. Create an X-Ray group for high-latency traces (> 2 seconds).

1.14 Review questions

1. What is the difference between a segment and a subsegment?
2. How does the trace ID propagate between services?
3. Why is sampling important for X-Ray cost management?
4. What does the X-Ray service map show?
5. How does OpenTelemetry relate to X-Ray?

Next: [Chapter 38 — CloudTrail & AWS Config](#)

AWS Cloud Handbook — Complete Edition —

Chapter 38

1 Chapter 38: CloudTrail & AWS Config

1.1 38.1 Audit and compliance foundations

Two services form the backbone of AWS governance and compliance:

Service	Purpose	Question answered
CloudTrail	API activity logging	Who did what, when, and from where?
AWS Config	Resource configuration tracking	What is the current/desired state of resources?

Together, they provide the audit trail and configuration baseline required for SOC 2, PCI DSS, HIPAA, and internal security policies.

1.2 38.2 AWS CloudTrail

CloudTrail records AWS API calls and delivers log files to S3 (and optionally CloudWatch Logs):

1.2.1 Event types

Type	Description	Example
Management events	Control plane operations	CreateVpc, RunInstances, DeleteBucket
Data events	Data plane operations (opt-in)	S3 GetObject, Lambda Invoke, DynamoDB GetItem
Insights events	Unusual API activity	Spike in DeleteSecurityGroup calls

1.2.2 Trail types

Type	Scope
Organization trail	All accounts in the organization (deploy from management account)
Account trail	Single account

Best practice: One organization trail in the management account, logging to a centralized S3 bucket in the log archive account.

1.2.3 Terraform organization trail

```
resource "aws_cloudtrail" "org" {
  name                = "organization-trail"
  s3_bucket_name      = aws_s3_bucket.trail.id
  include_global_service_events = true
  is_multi_region_trail = true
}
```

```

is_organization_trail      = true
enable_log_file_validation = true
kms_key_id                = aws_kms_key.trail.arn

event_selector {
  read_write_type      = "All"
  include_management_events = true

  data_resource {
    type = "AWS::S3::Object"
    values = ["arn:aws:s3:::sensitive-bucket/"]
  }
}

insight_selector {
  insight_type = "ApiCallRateInsight"
}
}

```

1.2.4 CloudTrail log entry structure

```

{
  "eventTime": "2024-06-15T10:30:00Z",
  "eventName": "RunInstances",
  "eventSource": "ec2.amazonaws.com",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "203.0.113.50",
  "userAgent": "aws-cli/2.15.0",
  "userIdentity": {
    "type": "AssumedRole",
    "principalId": "AROAI23:session-name",
    "arn": "arn:aws:sts::123456789012:assumed-role/AdminRole/session-name"
  },
  "requestParameters": {
    "instanceType": "t3.micro",
    "minCount": 1,
    "maxCount": 1
  },
  "responseElements": {
    "instancesSet": { "items": [{ "instanceId": "i-0abc123" }] }
  }
}

```

1.2.5 Querying CloudTrail

```

# Recent events
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteBucket \
  --max-results 10

# CloudTrail Lake (SQL queries)
aws cloudtrail create-event-data-store \
  --name security-events \
  --retention-period 365

# Athena (S3-based trail logs)
# Create Athena table on trail S3 bucket and run SQL queries

```

1.3 38.3 CloudTrail security

Control	Purpose
Log file validation	Detect tampering with digest files
S3 bucket policy	Restrict access to security team
KMS encryption	Encrypt trail logs at rest

S3 Object Lock	Prevent deletion (WORM compliance)
Organization trail	Ensure all accounts are logged (cannot be disabled by members)

1.3.1 Protecting the trail bucket

```
{
  "Sid": "DenyTrailDeletion",
  "Effect": "Deny",
  "Principal": "*",
  "Action": ["s3:DeleteObject", "s3:DeleteBucket"],
  "Resource": [
    "arn:aws:s3:::org-cloudtrail-logs",
    "arn:aws:s3:::org-cloudtrail-logs/*"
  ],
  "Condition": {
    "StringNotEquals": {
      "aws:PrincipalArn": "arn:aws:iam::LOG_ACCOUNT:role/CloudTrailAdmin"
    }
  }
}
```

1.4 38.4 AWS Config

AWS Config continuously records resource configuration changes and evaluates compliance against rules:

1.4.1 How Config works

Resource change → Config records configuration item → Rules evaluate → Compliance status
 ↓
 Non-compliant → SNS/SSM remediation

1.4.2 Configuration items

Each CI includes: - Resource type, ID, and ARN. - Current configuration (JSON). - Relationships to other resources. - Configuration change timeline.

1.4.3 Config rules

Type	Example
AWS managed	s3-bucket-public-read-prohibited, encrypted-volumes
Custom (Lambda)	Company-specific policies
Custom (Guard)	Policy-as-code (Cedar language)

```
resource "aws_config_config_rule" "s3_public_read" {
  name = "s3-bucket-public-read-prohibited"

  source {
    owner      = "AWS"
    source_identifier = "S3_BUCKET_PUBLIC_READ_PROHIBITED"
  }
}

resource "aws_config_config_rule" "encrypted_volumes" {
  name = "encrypted-volumes"

  source {
    owner      = "AWS"
    source_identifier = "ENCRYPTED_VOLUMES"
  }
}
```

```
scope {
  compliance_resource_types = ["AWS::EC2::Volume"]
}
}
```

1.4.4 Remediation

Automatically fix non-compliant resources:

```
resource "aws_config_remediation_configuration" "encrypt_volume" {
  config_rule_name = aws_config_config_rule.encrypted_volumes.name
  target_type      = "SSM_DOCUMENT"
  target_identifier = "AWS-EnableEBSEncryption"
  automatic        = true
  maximum_automatic_attempts = 3

  parameter {
    name      = "AutomationAssumeRole"
    static_value = aws_iam_role.remediation.arn
  }
}
```

1.5 38.5 Config aggregators

For multi-account environments, **Config aggregators** collect configuration and compliance data from multiple accounts and regions:

```
resource "aws_config_configuration_aggregator" "org" {
  name = "organization-aggregator"

  organization_aggregation_source {
    all_regions = true
    role_arn    = aws_iam_role.config_aggregator.arn
  }
}
```

Deploy the aggregator in the security/audit account for centralized compliance dashboards.

1.6 38.6 CloudTrail vs Config

Aspect	CloudTrail	Config
Records	API calls (actions)	Resource configurations (state)
Question	Who deleted the bucket?	Is the bucket encrypted?
Trigger	Every API call	Configuration change
Retention	S3 lifecycle policies	Config retention period
Compliance rules	No (use CloudTrail Lake + Athena)	Yes (managed and custom rules)

Use both: CloudTrail for “who did it” and Config for “is it compliant.”

1.7 38.7 Compliance reporting

Tool	Output
Config conformance packs	Pre-built rule collections (PCI, CIS, NIST)

Config compliance timeline	Historical compliance status
Security Hub	Aggregates Config, GuardDuty, Inspector findings
Athena on CloudTrail	Custom compliance queries

```
resource "aws_config_conformance_pack" "cis" {
  name = "CIS-AWS-Foundations-Benchmark"
  template_body = file("cis-conformance-pack.yaml")
}
```

1.8 38.8 Chapter summary

- **CloudTrail** logs all API activity; deploy an **organization trail** to a centralized, protected S3 bucket.
- **AWS Config** tracks resource configurations and evaluates **compliance rules**.
- Use **Config aggregators** for multi-account compliance visibility.
- Enable **log file validation**, **KMS encryption**, and **Object Lock** on trail buckets.
- Combine CloudTrail (who) and Config (what state) for complete governance.

1.9 Lab 38.1 — CloudTrail investigation

1. Enable a multi-region CloudTrail with S3 and CloudWatch Logs delivery.
2. Perform several API actions (create/delete security group).
3. Use `lookup-events` and CloudWatch Logs Insights to find the events.
4. Identify the IAM principal that performed each action.

1.10 Lab 38.2 — Config compliance

1. Enable AWS Config with the `s3-bucket-public-read-prohibited` managed rule.
2. Create a bucket with public read access.
3. Verify the rule reports non-compliance.
4. Fix the bucket and confirm compliance is restored.

1.11 Review questions

1. What is the difference between CloudTrail management events and data events?
2. Why should organization trails be deployed from the management account?
3. How does AWS Config differ from CloudTrail?
4. What is a Config conformance pack?
5. How do you protect CloudTrail logs from tampering?

Next: [Chapter 39 — KMS, Secrets Manager, ACM](#)

AWS Cloud Handbook — Complete Edition —

Chapter 39

1 Chapter 39: KMS, Secrets Manager, and ACM

1.1 39.1 Encryption and certificate management

AWS provides three core services for protecting data and communications:

Service	Purpose
KMS	Encryption key management
Secrets Manager	Secret storage and rotation
ACM	TLS/SSL certificate provisioning

Understanding when to use each—and how they integrate—is essential for a secure AWS architecture.

1.2 39.2 AWS Key Management Service (KMS)

KMS creates and manages encryption keys used to encrypt data across AWS services.

1.2.1 Key types

Type	Description	Use case
AWS managed	Created and managed by AWS per service	Default S3, EBS encryption
Customer managed (CMK)	You create and control	Cross-service, custom policies
AWS owned	Shared across accounts; you cannot view	Some default encryption
External (BYOK)	Import your own key material	Regulatory requirements

1.2.2 Key concepts

Concept	Description
CMK	Customer Master Key — never leaves KMS
Data key	Symmetric key generated by CMK; used to encrypt data
Envelope encryption	Encrypt data with data key; encrypt data key with CMK
Key policy	Resource policy controlling key access
Grant	Delegated permission to use a key
Alias	Friendly name for a key (alias/app-secrets)

1.2.3 Creating a CMK

```
aws kms create-key \
```

```
--description "Application encryption key" \
--key-usage ENCRYPT_DECRYPT \
--origin AWS_KMS

aws kms create-alias \
--alias-name alias/app-encryption \
--target-key-id <key-id>
```

1.2.4 Terraform

```
resource "aws_kms_key" "app" {
  description      = "Application encryption key"
  deletion_window_in_days = 30
  enable_key_rotation = true

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "Enable IAM policies"
        Effect   = "Allow"
        Principal = { AWS = "arn:aws:iam::123456789012:root" }
        Action   = "kms:*"
        Resource = "*"
      },
      {
        Sid      = "Allow service use"
        Effect   = "Allow"
        Principal = { Service = ["s3.amazonaws.com", "rds.amazonaws.com"] }
        Action   = ["kms:Encrypt", "kms:Decrypt", "kms:GenerateDataKey*"]
        Resource = "*"
      }
    ]
  })
}

resource "aws_kms_alias" "app" {
  name          = "alias/app-encryption"
  target_key_id = aws_kms_key.app.key_id
}
```

1.2.5 Envelope encryption

```
import boto3

kms = boto3.client("kms")

# Generate a data key
response = kms.generate_data_key(KeyId="alias/app-encryption", KeySpec="AES_256")
plaintext_key = response["Plaintext"] # use to encrypt data
encrypted_key = response["CiphertextBlob"] # store alongside encrypted data

# Decrypt the data key when needed
decrypted = kms.decrypt(CiphertextBlob=encrypted_key)
plaintext_key = decrypted["Plaintext"]
```

1.2.6 Multi-Region Keys (MRK)

Replicate keys across regions for DR without re-encrypting data:

Primary key (us-east-1) ↔ Replica key (eu-west-1)
Same key ID, same key material, regional endpoints

1.3 39.3 AWS Secrets Manager

Secrets Manager stores and rotates secrets (database credentials, API keys, tokens):

1.3.1 Features

Feature	Description
Automatic rotation	Lambda-based rotation on a schedule
Cross-account access	Resource policies for sharing
Versioning	AWSCURRENT and AWSPENDING versions during rotation
Integration	RDS, Redshift, DocumentDB native integration

1.3.2 Creating a secret

```
aws secretsmanager create-secret \  
  --name app/prod/database \  
  --description "Production database credentials" \  
  --secret-string '{"username":"admin","password":"initial-password","host":"mydb.abc.rds.amazonaws.com","port":5432}' \  
  
# Enable rotation (every 30 days)  
aws secretsmanager rotate-secret \  
  --secret-id app/prod/database \  
  --rotation-lambda-arn arn:aws:lambda:us-east-1:123456789012:function:SecretsManagerRDSPostgreSQLRotation \  
  --rotation-rules AutomaticallyAfterDays=30
```

1.3.3 Terraform

```
resource "aws_secretsmanager_secret" "db" {  
  name      = "app/${var.environment}/database"  
  kms_key_id = aws_kms_key.secrets.arn  
}  
  
resource "aws_secretsmanager_secret_version" "db" {  
  secret_id = aws_secretsmanager_secret.db.id  
  secret_string = jsonencode({  
    username = "admin"  
    password = random_password.db.result  
    host      = aws_rds_cluster.main.endpoint  
    port      = 5432  
    engine     = "postgres"  
  })  
}  
  
resource "aws_secretsmanager_secret_rotation" "db" {  
  secret_id      = aws_secretsmanager_secret.db.id  
  rotation_lambda_arn = aws_lambda_function.rotation.arn  
  rotation_rules {  
    automatically_after_days = 30  
  }  
}
```

1.3.4 Retrieving secrets in applications

```
import boto3, json  
  
client = boto3.client("secretsmanager")  
response = client.get_secret_value(SecretId="app/prod/database")  
secret = json.loads(response["SecretString"])  
# secret["username"], secret["password"], secret["host"]
```

1.4 39.4 AWS Certificate Manager (ACM)

ACM provisions, manages, and renews TLS/SSL certificates for free:

Feature	Detail

Cost	Free for use with AWS services
Validation	DNS (recommended) or email
Renewal	Automatic
Integration	ALB, CloudFront, API Gateway, NLB
Private CA	ACM Private CA for internal certificates (\$400/month per CA)

1.4.1 Requesting a certificate

```
aws acm request-certificate \
  --domain-name "*.example.com" \
  --subject-alternative-names "example.com" \
  --validation-method DNS
```

1.4.2 DNS validation (Terraform)

```
resource "aws_acm_certificate" "app" {
  domain_name        = "app.example.com"
  subject_alternative_names = ["*.app.example.com"]
  validation_method   = "DNS"

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.app.domain_validation_options : dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }
  zone_id = aws_route53_zone.main.zone_id
  name     = each.value.name
  type     = each.value.type
  records  = [each.value.record]
  ttl      = 60
}

resource "aws_acm_certificate_validation" "app" {
  certificate_arn      = aws_acm_certificate.app.arn
  validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]
}
```

1.4.3 Certificate limitations

Limitation	Detail
Export	Cannot export private keys (use with AWS services only)
Region	CloudFront requires certificates in us-east-1
Wildcard	*.example.com covers one level only
On EC2	Cannot install ACM certs directly; use ALB/CloudFront or import

1.5 39.5 Choosing the right service

Need	Service
Encrypt EBS, S3, RDS at rest	KMS
Store database password with rotation	Secrets Manager
Store API key (no rotation)	Parameter Store (SecureString) or Secrets Manager
TLS for ALB/CloudFront	ACM
Internal mTLS certificates	ACM Private CA
Application-level encryption	KMS envelope encryption

1.6 39.6 Security best practices

Practice	Rationale
Use customer managed CMKs	Control key policies, enable rotation, audit usage
Enable automatic key rotation	Annual rotation of CMK material
Separate keys per environment	Limit blast radius
Use Secrets Manager for rotating credentials	Automatic rotation reduces exposure window
DNS validation for ACM	More reliable than email validation
Monitor KMS API calls in CloudTrail	Detect unauthorized key usage
Least-privilege key policies	Grant only required principals and actions

1.6.1 CloudTrail KMS events to monitor

- `Decrypt` — unauthorized data access attempts
- `DisableKey` / `ScheduleKeyDeletion` — potential sabotage
- `PutKeyPolicy` — policy tampering

1.7 39.7 Chapter summary

- **KMS** manages encryption keys; use **customer managed CMKs** with automatic rotation.
- **Secrets Manager** stores and rotates credentials; integrates with RDS and Lambda.
- **ACM** provides free TLS certificates with automatic renewal for AWS services.
- Use **envelope encryption** for application-level data protection.
- Monitor key usage via **CloudTrail** and enforce **least-privilege key policies**.

1.8 Lab 39.1 — KMS encryption

1. Create a customer managed CMK with automatic rotation.
2. Encrypt a file using envelope encryption (generate data key, encrypt, store).
3. Decrypt and verify the original content.
4. Review CloudTrail for KMS Decrypt events.

1.9 Lab 39.2 — Secrets Manager rotation

1. Create an RDS instance with Secrets Manager-managed credentials.

2. Enable automatic rotation (30-day cycle).
 3. Retrieve the current secret and connect to the database.
 4. Trigger a manual rotation and verify the password changes.
-

1.10 Review questions

1. What is envelope encryption and why is it used?
 2. When should you use Secrets Manager vs Parameter Store SecureString?
 3. Why must CloudFront certificates be in us-east-1?
 4. What is the difference between AWS managed and customer managed CMKs?
 5. How does Secrets Manager automatic rotation work?
-

Next: [*Chapter 40 — GuardDuty, Security Hub, WAF, Shield*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 40

1 Chapter 40: GuardDuty, Security Hub, WAF, and Shield

1.1 40.1 AWS security services landscape

AWS provides a layered security stack for threat detection, compliance aggregation, and attack mitigation:

Layer	Service	Function
Detection	GuardDuty	ML-based threat detection
Aggregation	Security Hub	Centralized findings dashboard
Prevention	WAF	Application-layer firewall
DDoS protection	Shield	Network/transport layer DDoS mitigation
Vulnerability	Inspector	Software vulnerability scanning
Network firewall	Network Firewall	VPC-level stateful firewall

This chapter focuses on the four most critical services for production workloads.

1.2 40.2 Amazon GuardDuty

GuardDuty is a threat detection service that continuously monitors for malicious activity using machine learning, anomaly detection, and threat intelligence feeds.

1.2.1 Data sources

Source	Detects
CloudTrail	Unusual API calls, credential exfiltration
VPC Flow Logs	Malicious IPs, anomalous traffic patterns
DNS Logs	Compromised instances, crypto-mining domains
S3 Data Events	Unusual data access patterns
EKS Audit Logs	Kubernetes-specific threats
Malware Protection	Malicious files on EC2 and EBS

1.2.2 Finding types (examples)

Severity	Finding	Description
High	UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration	Credentials used from unusual IP
		Instance communicating

High	CryptoCurrency:EC2/BitcoinTool	with mining pool
Medium	Recon:EC2/PortProbeUnprotectedPort	Port scanning detected
Low	Policy:IAMUser/RootCredentialUsage	Root account API usage

1.2.3 Enabling GuardDuty

```
# Enable in a single account
aws guardduty create-detector --enable

# Organization-wide (from delegated admin account)
aws guardduty enable-organization-admin-account \
  --admin-account-id 111111111111

aws organizations enable-aws-service-access \
  --service-principal guardduty.amazonaws.com

aws guardduty create-members \
  --detector-id abc123 \
  --account-details AccountId=222222222222,Email=team@example.com
```

1.2.4 Terraform

```
resource "aws_guardduty_detector" "main" {
  enable = true

  datasources {
    s3_logs { enable = true }
    kubernetes {
      audit_logs { enable = true }
    }
    malware_protection {
      scan_ec2_instance_with_findings {
        ebs_volumes { enable = true }
      }
    }
  }
}
```

1.2.5 Response workflow

```
GuardDuty finding → EventBridge rule → Lambda (auto-remediate) / SNS (alert SOC)
├─ Isolate instance (modify SG)
├─ Snapshot EBS for forensics
└─ Disable compromised IAM credentials
```

1.3 40.3 AWS Security Hub

Security Hub aggregates, organizes, and prioritizes security findings from multiple AWS services and third-party tools:

1.3.1 Integrated services

Service	Findings
GuardDuty	Threat detections
Inspector	Vulnerability findings
Config	Compliance violations
IAM Access Analyzer	External access findings

Macie	Sensitive data exposure
Firewall Manager	WAF rule violations
Third-party	CrowdStrike, Palo Alto, Trend Micro

1.3.2 Security standards

Standard	Framework
AWS Foundational Security Best Practices	AWS-recommended controls
CIS AWS Foundations Benchmark	CIS critical controls
PCI DSS	Payment card industry
NIST 800-53	US government framework

```
resource "aws_securityhub_account" "main" {}

resource "aws_securityhub_standards_subscription" "fsbp" {
  standards_arn = "arn:aws:securityhub:us-east-1::standards/aws-foundational-security-best-practices/v/1.0.0"
  depends_on   = [aws_securityhub_account.main]
}

resource "aws_securityhub_standards_subscription" "cis" {
  standards_arn = "arn:aws:securityhub:::ruleset/cis-aws-foundations-benchmark/v/1.4.0"
  depends_on   = [aws_securityhub_account.main]
}
```

1.3.3 Finding workflow

Status	Meaning
NEW	Just received
NOTIFIED	Sent to integration
SUPPRESSED	Acknowledged/accepted risk
RESOLVED	Remediated

1.3.4 Cross-account aggregation

Designate a **delegated administrator** account to aggregate findings from all organization accounts into a single dashboard.

1.4 40.4 AWS WAF

AWS WAF is a web application firewall that protects HTTP/HTTPS applications from common exploits:

1.4.1 Attach points

Resource	Use case
CloudFront	Global edge protection
ALB	Regional application protection
API Gateway	API-specific rules
App Runner	Containerized app protection

1.4.2 Rule types

Type	Description
Regular rule	Custom conditions (IP, headers, body, URI)
Managed rule group	AWS or marketplace pre-built rules
Rate-based rule	Block IPs exceeding request rate

1.4.3 AWS managed rule groups

Group	Protects against
AWSManagedRulesCommonRuleSet	OWASP Top 10 (SQLi, XSS)
AWSManagedRulesKnownBadInputs	Log4j, Spring4Shell
AWSManagedRulesSQLiRuleSet	SQL injection
AWSManagedRulesBotControlRuleSet	Bot traffic
AWSManagedRulesAmazonIpReputationList	Known malicious IPs

1.4.4 Terraform WAF

```
resource "aws_wafv2_web_acl" "main" {
  name = "app-waf"
  scope = "REGIONAL"

  default_action { allow {} }

  rule {
    name = "RateLimit"
    priority = 1
    action { block {} }
    statement {
      rate_based_statement {
        limit = 2000
        aggregate_key_type = "IP"
      }
    }
    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name = "RateLimit"
      sampled_requests_enabled = true
    }
  }

  rule {
    name = "AWSManagedRulesCommon"
    priority = 2
    override_action { none {} }
    statement {
      managed_rule_group_statement {
        name = "AWSManagedRulesCommonRuleSet"
        vendor_name = "AWS"
      }
    }
    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name = "CommonRules"
      sampled_requests_enabled = true
    }
  }

  visibility_config {
    cloudwatch_metrics_enabled = true
    metric_name = "AppWAF"
    sampled_requests_enabled = true
  }
}

resource "aws_wafv2_web_acl_association" "alb" {
  resource_arn = aws_lb.app.arn
  web_acl_arn = aws_wafv2_web_acl.main.arn
}
```

```
}
```

1.4.5 WAF logging

Send WAF logs to S3, CloudWatch Logs, or Kinesis Firehose for analysis:

```
aws wafv2 put-logging-configuration \
  --logging-configuration '{
    "ResourceArn": "arn:aws:wafv2:us-east-1:123456789012:regional/webacl/app-waf/abc123",
    "LogDestinationConfigs": ["arn:aws:firehose:us-east-1:123456789012:deliverystream/waf-logs"]
  }'
```

1.5 40.5 AWS Shield

Shield provides DDoS protection for AWS resources:

Tier	Cost	Protection
Shield Standard	Free (automatic)	Network/transport layer (L3/L4) for all AWS customers
Shield Advanced	\$3,000/month + data transfer	Enhanced L3/L4 + L7 protection, 24/7 DDoS Response Team, cost protection

1.5.1 Shield Standard (automatic)

- Protects CloudFront, Route 53, ALB, NLB, Global Accelerator.
- No configuration required.
- Mitigates common SYN/UDP floods, reflection attacks.

1.5.2 Shield Advanced

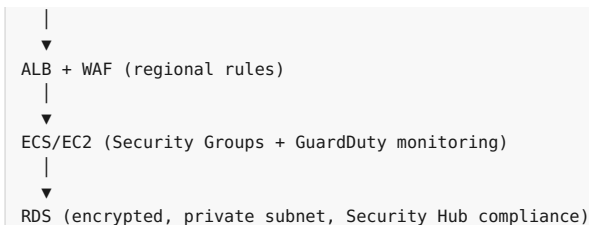
Feature	Benefit
Advanced detection	More sophisticated attack detection
DDoS Response Team (DRT)	24/7 access to AWS DDoS experts
Cost protection	Credits for scaling costs during attacks
WAF included	WAF fees waived for protected resources
Real-time metrics	Enhanced CloudWatch metrics and attack notifications

1.5.3 DDoS mitigation architecture

```
Attacker → Shield Standard/Advanced → CloudFront (WAF) → ALB → Application
          ↓
        Automatic mitigation
        (SYN flood, UDP reflection, etc.)
```

1.6 40.6 Defense-in-depth architecture

```
Internet
  |
  ▼
Route 53 (Shield)
  |
  ▼
CloudFront + WAF (Shield + managed rules)
```



Layer	Service	Protects against
DNS	Shield	DNS amplification
Edge	CloudFront + WAF + Shield	DDoS, OWASP Top 10
Network	Security Groups, NACLs	Unauthorized access
Detection	GuardDuty	Compromised instances, credential theft
Compliance	Security Hub, Config	Misconfigurations
Data	KMS, Macie	Encryption, data exposure

1.7 40.7 Incident response integration

```

resource "aws_cloudwatch_event_rule" "guardduty" {
  name = "guardduty-findings"
  event_pattern = jsonencode({
    source = ["aws.guardduty"]
    detail-type = ["GuardDuty Finding"]
    detail = {
      severity = [7, 7.0, 7.1, 7.2, 7.3, 7.4, 7.5, 7.6, 7.7, 7.8, 7.9,
                  8, 8.0, 8.1, 8.2, 8.3, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9]
    }
  })
}

resource "aws_cloudwatch_event_target" "sns" {
  rule = aws_cloudwatch_event_rule.guardduty.name
  target_id = "SendToSOC"
  arn = aws_sns_topic.security_alerts.arn
}

```

1.8 40.8 Chapter summary

- **GuardDuty** detects threats using ML across CloudTrail, VPC Flow Logs, and DNS logs.
- **Security Hub** aggregates findings and maps to compliance standards (CIS, PCI, NIST).
- **WAF** protects web applications with managed and custom rules at CloudFront and ALB.
- **Shield Standard** is free and automatic; **Shield Advanced** adds DRT and cost protection.
- Integrate findings with **EventBridge** for automated incident response.

1.9 Lab 40.1 — GuardDuty and Security Hub

1. Enable GuardDuty and Security Hub with the AWS Foundational standard.
2. Generate a test finding (e.g., unusual API call from a new IP).
3. Review the finding in GuardDuty and Security Hub.
4. Create an EventBridge rule to alert on high-severity findings.

1.10 Lab 40.2 — WAF protection

1. Create a WAF Web ACL with rate limiting and AWS managed rules.
 2. Associate it with an ALB.
 3. Test with a rate-exceeding script and verify blocking.
 4. Review WAF sampled requests in the console.
-

1.11 Review questions

1. What data sources does GuardDuty analyze?
 2. How does Security Hub differ from GuardDuty?
 3. Where can AWS WAF be attached?
 4. What is the difference between Shield Standard and Advanced?
 5. How would you automate response to a high-severity GuardDuty finding?
-

Next: [*Chapter 41 — Well-Architected Review & Capstone*](#)

AWS Cloud Handbook — Complete Edition —

Chapter 41

1 Chapter 41: Well-Architected Review & Capstone Design

1.1 41.1 The AWS Well-Architected Framework

Throughout this handbook, we have covered individual AWS services and patterns. The **AWS Well-Architected Framework** ties everything together into six pillars that define what “good” cloud architecture looks like. This capstone chapter applies the framework to a realistic design exercise, demonstrating how to evaluate and improve an architecture holistically.

1.2 41.2 The six pillars

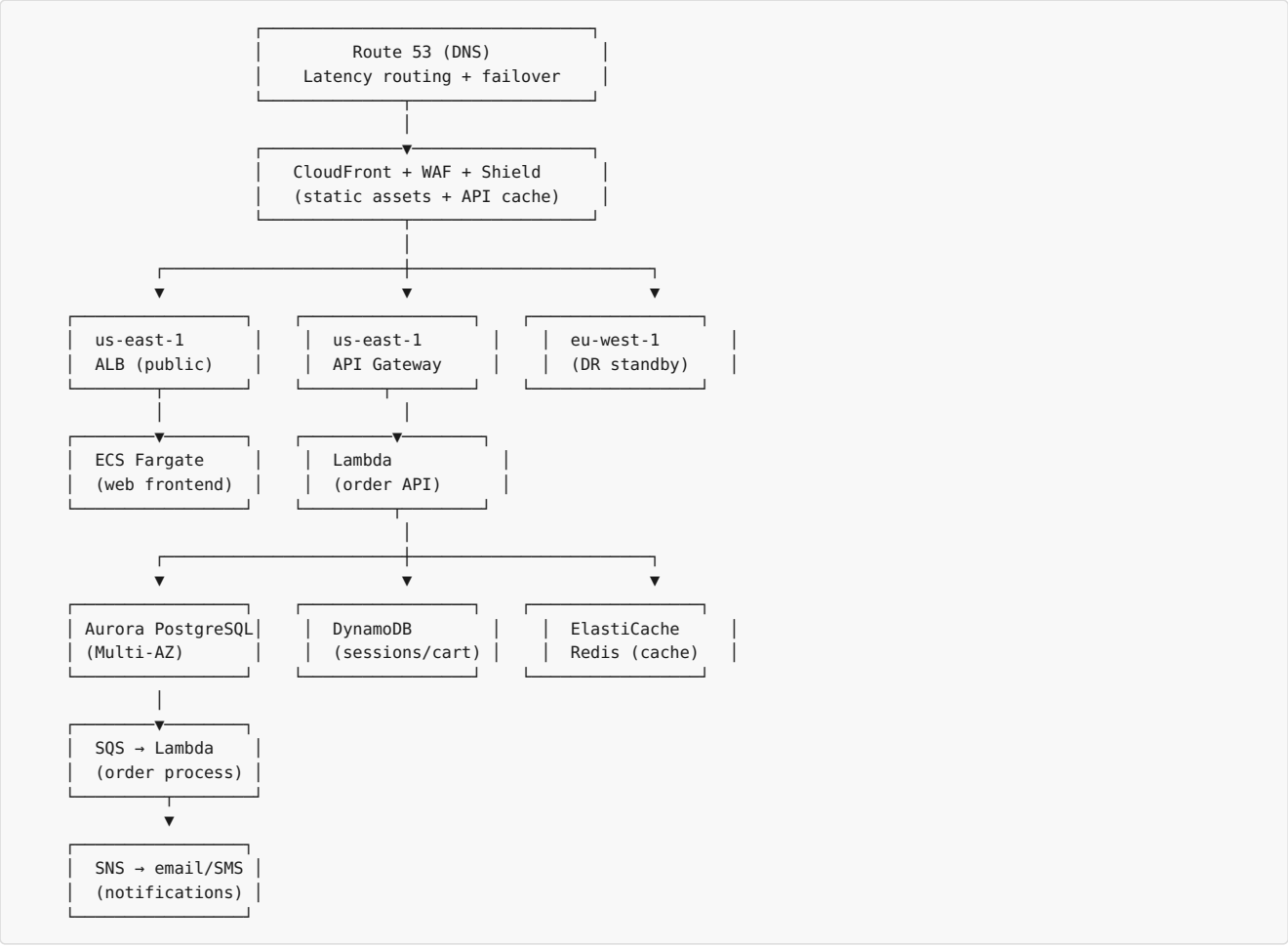
Pillar	Focus	Key questions
Operational Excellence	Run and monitor systems	Can you deploy, respond to events, and improve?
Security	Protect information and systems	Is data protected? Are threats detected?
Reliability	Recover from failures	Can the system recover and meet demand?
Performance Efficiency	Use resources efficiently	Is the right technology used at the right scale?
Cost Optimization	Avoid unnecessary costs	Are you paying only for what you need?
Sustainability	Minimize environmental impact	Are resources used efficiently to reduce carbon?

1.3 41.3 Capstone scenario

Design a **multi-tier e-commerce platform** with these requirements:

Requirement	Detail
Traffic	10,000 concurrent users; 1M orders/month
Availability	99.95% uptime SLA
Data	Customer PII, payment references (PCI scope)
Regions	Primary: us-east-1; DR: eu-west-1
Team	5 developers, 2 DevOps engineers
Budget	Optimize for cost without sacrificing reliability

1.4 41.4 Proposed architecture



1.5 41.5 Pillar-by-pillar review

1.5.1 Operational Excellence

Design decision	Rationale
Infrastructure as Code (CDK)	Reproducible environments; version-controlled
CI/CD via CodePipeline	Automated testing and deployment
CloudWatch dashboards + alarms	Centralized monitoring
Runbooks in Systems Manager	Documented incident response
ECS Exec for debugging	No SSH bastion needed

Review questions: - [] Are deployments automated with rollback capability? - [] Are runbooks documented for common incidents? - [] Is infrastructure defined in code (not console clicks)? - [] Are changes tracked in version control?

1.5.2 Security

Design decision	Rationale
IAM Identity Center for human access	Centralized SSO, MFA, audit

Private subnets for all compute/data	No direct internet exposure
KMS encryption (RDS, S3, DynamoDB, EBS)	Data at rest protection
Secrets Manager for DB credentials	Automatic rotation
WAF + Shield on CloudFront	OWASP protection, DDoS mitigation
GuardDuty + Security Hub	Threat detection and compliance
CloudTrail organization trail	API audit logging
VPC endpoints for AWS services	No internet traversal for API calls
PCI scope reduction	No card data stored; tokenization via payment provider

Review questions: - [] Is all data encrypted at rest and in transit? - [] Are security groups following least-privilege (SG references)? - [] Is GuardDuty enabled with automated response? - [] Are secrets rotated automatically? - [] Is the root account secured with MFA and not used daily?

1.5.3 Reliability

Design decision	Rationale
Multi-AZ Aurora	Automatic failover (< 30 seconds)
ECS Fargate across 3 AZs	Container-level HA
ALB health checks	Unhealthy targets removed automatically
SQS with DLQ	Order processing resilience
Aurora Global Database (DR)	Cross-region failover capability
Auto Scaling (ECS + Aurora Serverless)	Handle traffic spikes
Route 53 health checks + failover	DNS-level DR
AWS Backup for RDS and EBS	Point-in-time recovery

Review questions: - [] Are all tiers deployed across multiple AZs? - [] Is there a tested DR plan with defined RTO/RPO? - [] Are health checks configured on all load balancers? - [] Do async workflows use DLQs? - [] Has failover been tested in the last 6 months?

1.5.4 Performance Efficiency

Design decision	Rationale
CloudFront CDN	Static asset caching at edge
ElastiCache Redis	Database query caching
DynamoDB on-demand	Auto-scaling for cart/session data
Aurora Serverless v2 for variable load	Right-size database compute
Lambda for spiky order processing	Scale to zero, pay per request
Graviton instances (ARM) where supported	20% better price-performance

Review questions: - [] Is caching used at multiple layers (CDN, application, database)? - [] Are instance types right-sized based on metrics? - [] Is serverless used for variable/spiky workloads? - [] Are Graviton instances evaluated for cost savings?

1.5.5 Cost Optimization

--	--

Design decision	Rationale
Fargate (no idle EC2)	Pay per task, not per instance
Aurora Serverless v2	Scale down during off-peak
S3 lifecycle policies	Move old data to IA/Glacier
Reserved capacity for baseline	Savings Plans for predictable compute
CloudWatch cost anomaly detection	Alert on unexpected spend
Right-sizing reviews quarterly	Identify over-provisioned resources

Estimated monthly cost (simplified):

Service	Estimate
ECS Fargate (4 tasks)	\$120
Aurora Serverless v2 (2-8 ACU)	\$200
DynamoDB on-demand	\$50
ElastiCache (cache.r6g.large)	\$150
CloudFront + WAF	\$100
Lambda + API Gateway	\$30
Other (S3, SQS, SNS, monitoring)	\$50
Total	~\$700/month

Review questions: - [] Are unused resources identified and removed? - [] Are Savings Plans or Reserved Instances used for baseline? - [] Are S3 lifecycle policies configured? - [] Is cost allocation tagging enforced?

1.5.6 Sustainability

Design decision	Rationale
Graviton (ARM) instances	Lower energy per computation
Serverless (Lambda, Fargate)	No idle resource waste
S3 lifecycle to IA/Glacier	Reduce storage energy footprint
Right-sizing	Fewer resources = less energy
Region selection	Choose regions with renewable energy commitment

1.6 41.6 Well-Architected Tool review process

AWS provides a free **Well-Architected Tool** in the console:

1. **Create a workload** — Define the architecture scope.
2. **Answer pillar questions** — ~50 questions across six pillars.
3. **Review risks** — High and medium risks identified with remediation guidance.
4. **Improve** — Implement changes and re-review.

```
# Well-Architected Tool is console-based; use the API for automation
aws wellarchitected create-workload \
  --workload-name "E-Commerce Platform" \
  --description "Production e-commerce workload" \
  --environment PRODUCTION \
  --lenses "wellarchitected" \
```

```
--review-owner "platform-team@example.com" \  
--aws-regions us-east-1 eu-west-1
```

1.6.1 Review cadence

Trigger	Action
New workload launch	Initial review before production
Major architecture change	Re-review affected pillars
Quarterly	Scheduled review of all production workloads
Post-incident	Review reliability and operational excellence

1.7 41.7 Anti-patterns to avoid

Anti-pattern	Risk	Fix
Single AZ deployment	AZ failure = outage	Multi-AZ everything
Public RDS/EC2	Data breach	Private subnets + SG references
Root account for daily work	Credential compromise	IAM Identity Center
No monitoring/alarms	Silent failures	CloudWatch alarms on all tiers
Manual deployments	Human error, slow recovery	CI/CD pipeline
Hardcoded secrets	Credential leak	Secrets Manager
No backup testing	Backup exists but doesn't restore	Quarterly DR drills
Over-permissive IAM	Privilege escalation	Least privilege + SCPs
Ignoring costs until bill arrives	Budget overrun	Budgets + anomaly detection

1.8 41.8 Handbook journey recap

Part	Topics covered
I — Foundations	Shared responsibility, global infrastructure, CLI, billing
II — IAM	Users, roles, policies, Identity Center, Organizations, SCPs
III — Networking	VPC, NAT, security groups, NACLs, ALB/NLB, Route 53, TGW
IV — Compute	EC2, Auto Scaling, Lambda, ECS, EKS
V — Storage	S3, EBS, EFS, backup and DR
VI — Databases	RDS, Aurora, DynamoDB, ElastiCache, DMS
VII — Integration	API Gateway, SQS/SNS/EventBridge, Step Functions
VIII — DevOps	CloudFormation, CDK, CodePipeline, Systems Manager

IX — Observability	CloudWatch, Logs Insights, X-Ray, CloudTrail, Config
X — Security	KMS, Secrets Manager, ACM, GuardDuty, WAF, Shield

1.9 41.9 Chapter summary

- The **Well-Architected Framework** provides six pillars for evaluating cloud architectures.
- Apply the framework systematically: design, review, improve, repeat.
- The capstone e-commerce design demonstrates multi-pillar thinking in practice.
- Use the **Well-Architected Tool** for structured reviews with risk identification.
- Avoid common **anti-patterns** that compromise security, reliability, and cost.

1.10 Lab 41.1 — Well-Architected review

1. Open the AWS Well-Architected Tool in the console.
2. Create a workload for a project you have built (or the capstone design).
3. Complete the review for all six pillars.
4. Document the top 5 high-risk items and propose remediation.

1.11 Lab 41.2 — Architecture documentation

1. Draw the architecture diagram for your capstone (or real project).
2. Annotate each component with its pillar contributions.
3. Identify one improvement per pillar.
4. Estimate the cost impact of each improvement.

1.12 Review questions

1. Name the six Well-Architected Framework pillars.
2. What is the target availability of 99.95% in downtime per year?
3. How does the capstone design address the security pillar?
4. Why should Well-Architected reviews be repeated after major changes?
5. What is the most common anti-pattern you have seen in AWS environments?

1.13 Final thoughts

You have completed the AWS Handbook—from cloud foundations through security and compliance. The AWS ecosystem evolves continuously; subscribe to AWS blogs, re:Invent sessions, and service announcements to stay current. The Well-Architected Framework is your compass for every new design decision.

Build securely. Automate everything. Monitor relentlessly. Review regularly.

— *End of AWS Handbook* —